BUKU SOLUSI

Belajar Pemrograman Python

Materi dasar, Data Analisis, Visualisasi data, dan Capstone Project

102 solusi · Edisi 1

```
def solusi(soal):
    return jawaban
```

Python Intensive Bootcamp · 2026

Tentang buku ini

Belajar Pemrograman Python — Buku Solusi memuat solusi lengkap untuk **102 latihan dan mini project** pada buku kerja Hari 1–6, ditambah contoh capstone utuh sebagai lampiran. Setiap solusi dijalankan sungguhan: output dan hasil pemeriksa adalah hasil eksekusi nyata.

Cara membaca setiap latihan

- **Soal** — instruksi latihan persis seperti di buku kerja.
- **Pendekatan (pseudocode)** — kerangka berpikir sebelum menulis kode (kotak kuning).
- **Solusi** — kode jawaban (kotak gelap).
- **Output** — hasil menjalankan solusi (kotak hijau; grafik ditampilkan bila ada).
- **Hasil pemeriksa** — keluaran sel pemeriksa jawaban otomatis pada buku kerja (kotak ungu).

Saran belajar

- **Coba dulu sendiri.** Kerjakan latihan di buku kerja; buka buku ini hanya setelah mencoba, atau untuk membandingkan.
- Solusi di sini adalah **salah satu** cara yang benar. Jika jawaban Anda lolos pemeriksa, itu juga benar.
- Buka petunjuk pseudocode lebih dulu bila buntu, baru lihat kode.
- Bagian **Lampiran** memuat contoh capstone lengkap (kode, grafik, dokumentasi) untuk dibaca, dijalankan, dan dimodifikasi.

Isi per hari

Hari	Topik	Latihan
1	Python Fundamentals	18
2	Functions, Modules & File Handling	22
3	Object-Oriented Programming	12
4	NumPy & Pandas	24
5	Visualisasi, API & Scraping	17
6	Capstone Project	9

© 2026 Python Intensive Bootcamp. Seluruh hak dilindungi. Dilarang memperbanyak atau mendistribusikan ulang tanpa izin tertulis.

Daftar Isi

Hari 1 — Python Fundamentals 6

A. Variabel, Tipe Data, Operator, f-string 7

Latihan L1 — Variabel profil 7

Latihan L2 — Total dan rata-rata 7

Latihan L3 — Celsius ke Fahrenheit 7

Latihan L4 — f-string untuk struk 8

Latihan L5 — Tukar dua variabel 8

Latihan L6 — Grade dari nilai 8

Latihan L7 — Bilangan genap 2–50 9

Latihan L8 — Kasir dengan diskon 9

Latihan L9 — Tebak angka dengan while 10

Latihan L10 — Pola segitiga bintang (nested loop) 10

Latihan L11 — List mata kuliah 11

Latihan L12 — Dictionary profil 11

Latihan L13 — Filter nilai lulus 11

Latihan L14 — Tuple dan unpacking 12

Latihan L15 — Set: irisan, gabungan, deduplikasi 12

Latihan L16 — List of dict 12

Latihan P1 — Mini Project 1 — Sistem Data Mahasiswa 13

Latihan P2 — Mini Project 2 — Sistem Nilai & Statistik 14

Hari 2 — Functions, Modules & File Handling 15

A. Functions 16

Latihan L1 — luas_segitiga 16

Latihan L2 — cek_genap 16

Latihan L3 — rata_rata dengan *args 16

Latihan L4 — info_mahasiswa dengan **kwargs 17

Latihan L5 — konversi_nilai 17

Latihan L6 — hitung_diskon dengan default 17

Latihan L7 — min_max (multiple return) 18

Latihan L8 — Docstring dan hitung_bmi 18

Latihan L9 — Kuadrat 1–20 18

Latihan L10 — Bonus nilai lulus 18

Latihan L11 — filter + lambda 19

Latihan L12 — Dictionary comprehension 19

Latihan L13 — Urutkan dengan lambda 19

Latihan L14 — Module konversi.py 19

Latihan L15 — Module string_utils.py 20

Latihan L16 — bagi_aman 20

Latihan L17 — input_angka (ulangi sampai valid) 21

Latihan L18 — Validasi dengan raise 21

Latihan L19 — Simpan dan muat daftar (teks) 21
Latihan L20 — Simpan dan muat CSV 22
Latihan P1 — Mini Project 1 — Pencatat Kegiatan Akademik 22
Latihan P2 — Mini Project 2 — Perpustakaan Mini 23

Hari 3 — Object-Oriented Programming 25

A. Class, Object, Attribute, Method 26
Latihan L1 — Class Mahasiswa 26
Latihan L2 — Class attribute sebagai penghitung 26
Latihan L3 — RekeningBank 27
Latihan L4 — Kendaraan, Mobil, Motor 27
Latihan L5 — Polimorfisme: Pegawai dan Manager 28
Latihan L6 — Encapsulation — Rekening aman 29
Latihan L7 — Special methods pada Buku 29
Latihan L8 — Titik dan jarak 30
Latihan L9 — Context manager FileCatatan 30
Latihan L10 — Perbaiki mutable default 31
Latihan P1 — Mini Project 1 — Manajemen Tugas Kuliah 31
Latihan P2 — Mini Project 2 — Perpustakaan dengan Inheritance 32

Hari 4 — NumPy & Pandas 34

A. NumPy 35
Latihan L1 — Membuat dan membentuk ulang array 35
Latihan L2 — Indexing dan slicing 35
Latihan L3 — Boolean masking 35
Latihan L4 — np.where 36
Latihan L5 — Broadcasting: nilai akhir berbobot 36
Latihan L6 — Statistik bagian tengah matriks 36
Latihan L7 — Normalisasi min-max 37
Latihan L8 — Korelasi dua variabel 37
Latihan L9 — Isi diagonal 37
Latihan L10 — Series 37
Latihan L11 — Membuat dan memeriksa DataFrame 38
Latihan L12 — Seleksi dan filter 38
Latihan L13 — Kolom baru: status dan grade 38
Latihan L14 — Sorting 39
Latihan L15 — Missing values dan duplikat 39
Latihan L16 — Konversi tipe data 40
Latihan L17 — GroupBy 40
Latihan L18 — Merge 40
Latihan L19 — Pivot table dan CSV 40
Latihan P1 — Project 1 — Kalkulator Nilai Akhir (Dasar) 41
Latihan P2 — Project 2 — Cleaning Dataset Kotor 41

Latihan P3 — Project 3 — Merge Multi-Tabel (Menengah) 42
Latihan P4 — Project 4 — Pivot dan Reporting (Menengah–Tinggi) 42
Latihan P5 — Project 5 — Ringkasan EDA (Tinggi) 42

Hari 5 — Visualisasi, API & Scraping 44

A. Matplotlib 45

Latihan L1 — Line plot dua seri 45
Latihan L2 — Bar chart dengan label nilai 45
Latihan L3 — Histogram dengan garis rata-rata 46
Latihan L4 — Dashboard 2×2 dengan subplot 46
Latihan L5 — Simpan grafik ke file 47
Latihan L6 — Boxplot per hari 47
Latihan L7 — Heatmap korelasi 48
Latihan L8 — Countplot dan barplot 48
Latihan L9 — ambil_json yang tahan error 48
Latihan L10 — json_normalize dan agregasi 49
Latihan L11 — Mengolah respons cuaca 49
Latihan L12 — Scraping HTML kutipan 50
Latihan L13 — Mengurai feed arXiv 50
Latihan P1 — Project 1 — Dashboard Visualisasi Nilai (Dasar) 51
Latihan P2 — Project 2 — API + Visualisasi (Menengah) 52
Latihan P3 — Project 3 — Pipeline Paper arXiv (Menengah–Tinggi) 52
Latihan P4 — Project 4 — EDA Lengkap + Visual (Tinggi) 53

Hari 6 — Capstone Project 54

A. Memilih Jalur dan Topik (Lembar Perencanaan) 55
B. Pertanyaan Analisis dan Struktur Folder 55

Latihan L1 — Menulis pertanyaan analisis 55
Latihan L2 — Membuat struktur folder proyek 55
Latihan L3 — clean(df) 56
Latihan L4 — analyze(df) 56
Latihan L5 — visualize(df, hasil, fig_dir) 57
Latihan L6 — tulis_insight(hasil, path) 57
Latihan L7 — README dan main() — menyatukan pipeline 58
Latihan L8 — Jalur B — ambil data dengan cache dan fallback 59
Latihan L9 — Jalur C — parameter paginasi arXiv 60

Lampiran — Contoh Capstone Lengkap 61

Jalur A — EDA Dataset Publik (Titanic) 62
Jalur B — API PyPI & npm → Analisis → Visualisasi 73
Jalur C — Pipeline Paper arXiv 85
Jalur D — Aplikasi Analisis Nilai Berbasis OOP 94

1

Hari 1 — Python Fundamentals

Variabel, control flow, struktur data, mini project menu mahasiswa

A. Variabel, Tipe Data, Operator, f-string

A. Variabel, Tipe Data, Operator, f-string

Ringkas: variabel adalah nama yang merujuk ke nilai; tipe mengikuti nilainya (`int` , `float` , `str` , `bool`). `input()` selalu menghasilkan `str` , jadi ubah dengan `int()` / `float()` . f-string: `f"Halo {nama}"` , format angka `{x:.2f}` .

Latihan L1 — Variabel profil

Buat lima variabel: `nama` (str), `nim` (str), `semester` (int), `ipk` (float antara 0–4), dan `aktif` (bool).

PENDEKATAN (PSEUDOCODE)

```
nama  ← teks dalam tanda kutip
nim   ← teks (bukan angka)
semester ← bilangan bulat
ipk   ← bilangan desimal
aktif ← True atau False
```

SOLUSI

```
nama = "Rina Kartika"
nim = "2201010234"
semester = 3
ipk = 3.67
aktif = True
```

HASIL PEMERIKSA

```
✅ L1 benar!
```

Latihan L2 — Total dan rata-rata

Diberikan tiga nilai ujian. Hitung `total` dan `rata` (rata-rata).

PENDEKATAN (PSEUDOCODE)

```
total ← n1 + n2 + n3
rata  ← total dibagi 3   (gunakan / agar hasil desimal)
```

SOLUSI

```
n1, n2, n3 = 80, 90, 70
total = n1 + n2 + n3
rata = total / 3
```

HASIL PEMERIKSA

```
✅ L2 benar!
```

Latihan L3 — Celsius ke Fahrenheit

Rumus: `F = C × 9/5 + 32` . Hitung `fahrenheit` dari `celsius = 100` .

PENDEKATAN (PSEUDOCODE)

```
fahrenheit ← celsius × 9 / 5 + 32
```

SOLUSI

```
celsius = 100
fahrenheit = celsius * 9 / 5 + 32
```

HASIL PEMERIKSA

```
✅ L3 benar!
```

Latihan L4 — f-string untuk struk

Dengan `item`, `harga`, dan `qty` di bawah, isi `struk` dengan teks persis: `Fotokopi: 12 x 500 = 6000` memakai f-string.

PENDEKATAN (PSEUDOCODE)

```
struk ← f"{item}: {qty} x {harga} = {hasil kali harga dan qty}"
```

SOLUSI

```
item, harga, qty = "Fotokopi", 500, 12
struk = f"{item}: {qty} x {harga} = {harga * qty}"
```

HASIL PEMERIKSA

```
✅ L4 benar!
```

Latihan L5 — Tukar dua variabel

Tukar isi `a` dan `b` **tanpa** variabel bantu (gunakan penugasan ganda).

PENDEKATAN (PSEUDOCODE)

```
a, b ← b, a
```

SOLUSI

```
a, b = "kiri", "kanan"
a, b = b, a
```

HASIL PEMERIKSA

```
✅ L5 benar!
```

Latihan L6 — Grade dari nilai

Untuk setiap nilai pada `nilai_uji`, tentukan huruf: `A` (≥85), `B` (≥70), `C` (≥60), `D` (≥50), selain itu `E`. Simpan hasilnya (berurutan) ke list `hasil_grade`.

PENDEKATAN (PSEUDOCODE)

```
UNTUK setiap n DI nilai_uji:
    JIKA n >= 85 → "A"
    JIKA TIDAK n >= 70 → "B"
    ... dst (urutan dari batas tertinggi)
    tambahkan huruf ke hasil_grade
```

SOLUSI

```
nilai_uji = [95, 78, 65, 52, 30]
hasil_grade = []
for n in nilai_uji:
    if n >= 85:
        g = "A"
    elif n >= 70:
        g = "B"
    elif n >= 60:
        g = "C"
    elif n >= 50:
        g = "D"
    else:
        g = "E"
    hasil_grade.append(g)
```

HASIL PEMERIKSA

```
✅ L6 benar!
```

Latihan L7 — Bilangan genap 2–50

Isi list `genap` dengan semua bilangan genap dari 2 sampai 50 (inklusif) menggunakan `for` + `range`.

PENDEKATAN (PSEUDOCODE)

```
range(awal, akhir_eksklusif, langkah)
akhir eksklusif = 51 agar 50 ikut
```

SOLUSI

```
genap = []
for i in range(2, 51, 2):
    genap.append(i)
```

HASIL PEMERIKSA

```
✅ L7 benar!
```

Latihan L8 — Kasir dengan diskon

Total belanja ≥ 100.000 mendapat diskon 10%. Untuk tiap nilai di `belanja`, hitung jumlah yang dibayar dan simpan di list `bayar`.

PENDEKATAN (PSEUDOCODE)

```
UNTUK total DI belanja:
    JIKA total >= 100000 → bayar total × 0.9
    SELAIN ITU           → bayar total
```

SOLUSI

```
belanja = [50000, 100000, 250000]
bayar = []
for total in belanja:
    if total >= 100000:
        bayar.append(total * 0.9)
    else:
        bayar.append(total)
```

HASIL PEMERIKSA

```
✅ L8 benar!
```

Latihan L9 — Tebak angka dengan while

Angka rahasia = 7. Tebakan pemain datang berurutan dari `tebakan` (gunakan `next(tebakan)`). Ulangi dengan `while True` dan catat petunjuk ke list `petunjuk`: `"Terlalu kecil"`, `"Terlalu besar"`, atau `"Benar!"` (lalu `break`).

PENDEKATAN (PSEUDOCODE)

```
ULANGI selamanya:
    t ← tebakan berikutnya
    bandingkan t dengan rahasia → tambah petunjuk
    jika benar → hentikan (break)
```

SOLUSI

```python
rahasia = 7
tebakan = iter([3, 9, 7, 5])
petunjuk = []
while True:
    t = next(tebakan)
    if t < rahasia:
        petunjuk.append("Terlalu kecil")
    elif t > rahasia:
        petunjuk.append("Terlalu besar")
    else:
        petunjuk.append("Benar!")
        break
```

HASIL PEMERIKSA

```
✅ L9 benar!
```

Latihan L10 — Pola segitiga bintang (nested loop)

Isi list `baris` dengan lima string: `"*"`, `"**"`, `"***"`, `"****"`, `"*****"` memakai loop bersarang (atau `"*" * i`).

SOLUSI

```python
baris = []
for i in range(1, 6):
    s = ""
    for _ in range(i):
        s += "*"
    baris.append(s)
```

HASIL PEMERIKSA

```
*
**
***
****
*****
✅ L10 benar!
```

Latihan L11 — List mata kuliah

Mulai dari `mk` di bawah: tambahkan `"AI"` di akhir, sisipkan `"Statistika"` di indeks 1, lalu hapus `"Fisika"`.

PENDEKATAN (PSEUDOCODE)

```
append → tambah di akhir
insert(indeks, nilai) → sisip
remove(nilai) → hapus nilai pertama yang cocok
```

SOLUSI

```python
mk = ["Kalkulus", "Fisika", "Pemrograman"]
mk.append("AI")
mk.insert(1, "Statistika")
mk.remove("Fisika")
```

HASIL PEMERIKSA

```
✅ L11 benar!
```

Latihan L12 — Dictionary profil

Tambahkan key `"semester"` bernilai `4` ke `mhs`, ubah `"ipk"` menjadi `3.72`, dan tambahkan `"AI"` ke list di `"mk"`.

PENDEKATAN (PSEUDOCODE)

```
mhs["key"] = nilai   → tambah/ubah
mhs["mk"] adalah list → pakai .append
```

SOLUSI

```python
mhs = {"nama": "Dewi", "nim": "2201", "ipk": 3.5, "mk": ["BD"]}
mhs["semester"] = 4
mhs["ipk"] = 3.72
mhs["mk"].append("AI")
```

HASIL PEMERIKSA

```
✅ L12 benar!
```

Latihan L13 — Filter nilai lulus

Dari `nilai`, buat list `lulus` berisi nilai ≥ 75 (urutan asli) dan simpan jumlahnya di `n_lulus`.

PENDEKATAN (PSEUDOCODE)

```
lulus ← [n untuk n di nilai jika n >= 75]   (atau for + append)
```

SOLUSI

```python
nilai = [75, 80, 90, 65, 88, 70, 95]
lulus = [n for n in nilai if n >= 75]
n_lulus = len(lulus)
```

HASIL PEMERIKSA

```
✅ L13 benar!
```

Latihan L14 — Tuple dan unpacking

`data` berisi `(nama, nim, ipk)`. Pecah menjadi tiga variabel `nama`, `nim`, `ipk` dengan unpacking, lalu isi `terkecil` dan `terbesar` dengan `min` / `max` dari `angka`.

SOLUSI

```
data = ("Ani", "210123", 3.8)
angka = (3, 1, 4, 2)
nama, nim, ipk = data
terkecil, terbesar = min(angka), max(angka)
```

HASIL PEMERIKSA

```
✅ L14 benar!
```

Latihan L15 — Set: irisan, gabungan, deduplikasi

Dari dua himpunan hobi, isi `irisan`, `gabungan`, dan `hanya_a` (ada di A saja). Lalu hapus duplikat `raw` menjadi list `unik` (urutan boleh berbeda; urutkan dengan `sorted`).

PENDEKATAN (PSEUDOCODE)

```
irisan = a & b   gabungan = a | b   selisih = a - b
unik ← sorted(set(raw))
```

SOLUSI

```
a = {"baca", "musik", "olahraga"}
b = {"musik", "gaming", "olahraga"}
raw = [1, 2, 2, 3, 3, 3]
irisan = a & b
gabungan = a | b
hanya_a = a - b
unik = sorted(set(raw))
```

HASIL PEMERIKSA

```
✅ L15 benar!
```

Latihan L16 — List of dict

Dari `mahasiswa`: isi `terbaik` dengan **nama** mahasiswa ber-IPK tertinggi, `if_only` dengan list mahasiswa prodi `"IF"`, dan `rata_if` dengan rata-rata IPK prodi IF.

PENDEKATAN (PSEUDOCODE)

```
terbaik ← nama dari elemen dengan ipk terbesar  (max(..., key=lambda m: m["ipk"]))
if_only ← elemen yang prodi-nya "IF"
rata_if ← jumlah ipk if_only dibagi banyaknya
```

SOLUSI

```
mahasiswa = [
    {"nama": "Ani", "nim": "2201", "ipk": 3.5, "prodi": "IF"},
    {"nama": "Budi", "nim": "2202", "ipk": 3.9, "prodi": "SI"},
    {"nama": "Cici", "nim": "2203", "ipk": 3.2, "prodi": "IF"},
]
terbaik = max(mahasiswa, key=lambda m: m["ipk"])["nama"]
if_only = [m for m in mahasiswa if m["prodi"] == "IF"]
rata_if = sum(m["ipk"] for m in if_only) / len(if_only)
```

HASIL PEMERIKSA

✅ L16 benar!

Latihan P1 — Mini Project 1 — Sistem Data Mahasiswa

Lengkapi empat function pada `data` (list of dict):

- `tambah(data, nama, nim, prodi, ipk)` → tambah mahasiswa; kembalikan `True`. Jika `ipk` di luar 0–4 **atau** `nim` sudah ada, jangan tambah dan kembalikan `False`.
- `cari(data, nim)` → kembalikan dict mahasiswa atau `None`.
- `hapus(data, nim)` → hapus jika ada; kembalikan `True` / `False`.
- `ipk_tertinggi(data)` → kembalikan dict dengan IPK tertinggi, atau `None` bila data kosong.

PENDEKATAN (PSEUDOCODE)

```
tambah: JIKA ipk tidak 0..4 → False; JIKA nim sudah ada → False; selain itu append dict → True
cari  : untuk setiap m, jika m["nim"] == nim → kembalikan m; akhir loop → None
hapus : untuk (i, m), jika nim cocok → data.pop(i), True; akhir → False
ipk_tertinggi: data kosong → None; selain itu max(..., key=...)
```

SOLUSI

```
def tambah(data, nama, nim, prodi, ipk):
    if not 0 <= ipk <= 4:
        return False
    if any(m["nim"] == nim for m in data):
        return False
    data.append({"nama": nama, "nim": nim, "prodi": prodi, "ipk": ipk})
    return True

def cari(data, nim):
    for m in data:
        if m["nim"] == nim:
            return m
    return None

def hapus(data, nim):
    for i, m in enumerate(data):
        if m["nim"] == nim:
            data.pop(i)
            return True
    return False

def ipk_tertinggi(data):
    if not data:
        return None
    return max(data, key=lambda m: m["ipk"])
```

HASIL PEMERIKSA

```
✅ P1 benar!
```

Latihan P2 — Mini Project 2 — Sistem Nilai & Statistik

Setiap mahasiswa: `{"nama": ..., "nilai": {"MK": angka, ...}}`. Lengkapi:

- `rata_rata(nilai_dict)` → rata-rata nilai; `0` bila dict kosong.
- `statistik(data)` → dict `{"rata_kelas": ..., "max": ..., "min": ...}` dari rata-rata individu; `None` bila data kosong.
- `filter_lulus(data, ambang=70)` → list **nama** yang rata-ratanya ≥ ambang.

SOLUSI

```python
def rata_rata(nilai_dict):
    if not nilai_dict:
        return 0
    return sum(nilai_dict.values()) / len(nilai_dict)

def statistik(data):
    if not data:
        return None
    r = [rata_rata(d["nilai"]) for d in data]
    return {"rata_kelas": sum(r) / len(r), "max": max(r), "min": min(r)}

def filter_lulus(data, ambang=70):
    return [d["nama"] for d in data if rata_rata(d["nilai"]) >= ambang]
```

HASIL PEMERIKSA

```
✅ P2 benar!
```

2

Hari 2 — Functions, Modules & File Handling

Function, lambda, module, error handling, file dan CSV

A. Functions

A. Functions

Ringkas: function membungkus langkah yang bisa dipakai ulang (*DRY*). Pola: `def nama(parameter): ... return hasil`. Parameter bisa punya nilai default (`def f(a, b=10)`), `*args` menampung argumen posisi berlebih (tuple), `**kwargs` menampung argumen bernama berlebih (dict). Tulis **docstring** di baris pertama isi function.

Latihan L1 — luas_segitiga

Buat function `luas_segitiga(alas, tinggi)` yang **mengembalikan** luas (`0.5 × alas × tinggi`).

PENDEKATAN (PSEUDOCODE)

```
FUNCTION luas_segitiga(alas, tinggi)
    RETURN 0.5 × alas × tinggi
```

SOLUSI

```python
def luas_segitiga(alas, tinggi):
    return 0.5 * alas * tinggi
```

HASIL PEMERIKSA

```
✅ L1 benar!
```

Latihan L2 — cek_genap

Buat `cek_genap(n)` yang mengembalikan `True` jika `n` genap, `False` jika ganjil.

SOLUSI

```python
def cek_genap(n):
    return n % 2 == 0
```

HASIL PEMERIKSA

```
✅ L2 benar!
```

Latihan L3 — rata_rata dengan *args

Buat `rata_rata(*angka)` yang mengembalikan rata-rata semua argumen; jika tidak ada argumen kembalikan `0`.

PENDEKATAN (PSEUDOCODE)

```
JIKA tidak ada angka → 0
SELAIN ITU → jumlah(angka) / banyak(angka)
```

SOLUSI

```python
def rata_rata(*angka):
    if not angka:
        return 0
    return sum(angka) / len(angka)
```

HASIL PEMERIKSA

```
✅ L3 benar!
```

Latihan L4 — info_mahasiswa dengan **kwargs

Buat `info_mahasiswa(**data)` yang mengembalikan **list string** `"kunci: nilai"` sesuai urutan argumen.

SOLUSI

```python
def info_mahasiswa(**data):
    return [f"{k}: {v}" for k, v in data.items()]
```

HASIL PEMERIKSA

```
✅ L4 benar!
```

Latihan L5 — konversi_nilai

Buat `konversi_nilai(angka)` : `A` (≥85), `B` (≥70), `C` (≥60), `D` (≥50), selain itu `E`. Nilai di luar 0–100 → kembalikan `"Tidak valid"` (gunakan *early return*).

PENDEKATAN (PSEUDOCODE)

```
JIKA angka di luar 0..100 → "Tidak valid"
cek dari batas tertinggi ke terendah, return pada kecocokan pertama
```

SOLUSI

```python
def konversi_nilai(angka):
    if angka < 0 or angka > 100:
        return "Tidak valid"
    if angka >= 85: return "A"
    if angka >= 70: return "B"
    if angka >= 60: return "C"
    if angka >= 50: return "D"
    return "E"
```

HASIL PEMERIKSA

```
✅ L5 benar!
```

Latihan L6 — hitung_diskon dengan default

Buat `hitung_diskon(total, persen=10)` : jika `total >= 100000` kembalikan **jumlah diskon** (`total × persen / 100`), selain itu `0` .

SOLUSI

```python
def hitung_diskon(total, persen=10):
    if total >= 100000:
        return total * persen / 100
    return 0
```

HASIL PEMERIKSA

```
✅ L6 benar!
```

Latihan L7 — min_max (multiple return)

Buat `min_max(data)` yang mengembalikan **tuple** `(nilai_minimum, nilai_maksimum)`. Setelah itu, lakukan unpacking: `kecil, besar = min_max([3, 1, 4, 2])`.

SOLUSI

```
def min_max(data):
    return min(data), max(data)

kecil, besar = min_max([3, 1, 4, 2])
```

HASIL PEMERIKSA

```
✅ L7 benar!
```

Latihan L8 — Docstring dan hitung_bmi

Buat `hitung_bmi(berat, tinggi)` (`berat / tinggi²`) **dengan docstring** yang tidak kosong. Kembalikan nilai dibulatkan 1 desimal.

PENDEKATAN (PSEUDOCODE)

```
FUNCTION hitung_bmi(berat, tinggi)
    """deskripsi singkat"""
    RETURN round(berat / tinggi ** 2, 1)
```

SOLUSI

```
def hitung_bmi(berat, tinggi):
    """Menghitung BMI = berat (kg) / tinggi (m) kuadrat, dibulatkan 1 desimal."""
    return round(berat / tinggi ** 2, 1)
```

HASIL PEMERIKSA

```
✅ L8 benar!
```

Latihan L9 — Kuadrat 1–20

Dengan list comprehension, isi `kuadrat` dengan kuadrat bilangan 1 sampai 20.

PENDEKATAN (PSEUDOCODE)

```
kuadrat ← [x ** 2 untuk x di range(1, 21)]
```

SOLUSI

```
kuadrat = [x ** 2 for x in range(1, 21)]
```

HASIL PEMERIKSA

```
✅ L9 benar!
```

Latihan L10 — Bonus nilai lulus

Dari `nilai`, ambil hanya yang ≥ 75 lalu kalikan 1.1 dan **bulatkan 1 desimal**. Simpan ke `bonus`.

SOLUSI

```
nilai = [60, 75, 40, 90, 55, 80]
bonus = [round(n * 1.1, 1) for n in nilai if n >= 75]
```

HASIL PEMERIKSA

✅ L10 benar!

Latihan L11 — filter + lambda

Dengan `filter` dan `lambda`, ambil nama yang panjangnya **> 4** dari `nama` (hasil berupa **list**) dan simpan di `panjang`.

SOLUSI

```
nama = ["Ani", "Budi", "Citra", "Eko", "Fajar"]
panjang = list(filter(lambda n: len(n) > 4, nama))
```

HASIL PEMERIKSA

✅ L11 benar!

Latihan L12 — Dictionary comprehension

Buat `paritas`: dictionary `{angka: "genap" / "ganjil"}` untuk angka 1–10.

SOLUSI

```
paritas = {x: ("genap" if x % 2 == 0 else "ganjil") for x in range(1, 11)}
```

HASIL PEMERIKSA

✅ L12 benar!

Latihan L13 — Urutkan dengan lambda

Urutkan `mahasiswa` berdasarkan `ipk` **menurun** dengan `sorted(..., key=lambda ...)`; simpan di `urut`.

SOLUSI

```
mahasiswa = [{"nama": "Ani", "ipk": 3.5}, {"nama": "Budi", "ipk": 3.9}, {"nama": "Citra", "ipk": 3.2}]
urut = sorted(mahasiswa, key=lambda m: m["ipk"], reverse=True)
```

HASIL PEMERIKSA

✅ L13 benar!

Latihan L14 — Module konversi.py

Isi string `isi_modul` dengan kode Python berisi tiga function: `celsius_ke_fahrenheit(c)`, `fahrenheit_ke_celsius(f)`, dan `meter_ke_km(m)`. Sel berikutnya menulisnya ke `konversi.py` lalu mengimpornya.

SOLUSI

```
isi_modul = '''
def celsius_ke_fahrenheit(c):
    return c * 9 / 5 + 32

def fahrenheit_ke_celsius(f):
    return (f - 32) * 5 / 9

def meter_ke_km(m):
    return m / 1000
'''
```

HASIL PEMERIKSA

✅ L14 benar!

Latihan L15 — Module string_utils.py

Sama seperti L14: isi `isi_modul2` dengan function `hitung_vokal(teks)`, `balik_string(teks)`, dan `apakah_palindrome(teks)` (abaikan spasi & huruf besar).

SOLUSI

```
isi_modul2 = '''
def hitung_vokal(teks):
    return sum(1 for c in teks.lower() if c in "aiueo")

def balik_string(teks):
    return teks[::-1]

def apakah_palindrome(teks):
    s = "".join(teks.lower().split())
    return s == s[::-1]
'''
```

HASIL PEMERIKSA

✅ L15 benar!

Latihan L16 — bagi_aman

Buat `bagi_aman(a, b)` yang mengembalikan `a / b`; jika `b == 0` (ZeroDivisionError) atau tipe tidak cocok (TypeError) kembalikan `None`.

PENDEKATAN (PSEUDOCODE)

```
COBA: RETURN a / b
TANGKAP ZeroDivisionError atau TypeError: RETURN None
```

SOLUSI

```
def bagi_aman(a, b):
    try:
        return a / b
    except (ZeroDivisionError, TypeError):
        return None
```

HASIL PEMERIKSA

```
✅ L16 benar!
```

Latihan L17 — input_angka (ulangi sampai valid)

Buat `input_angka(prompt)` yang terus meminta input sampai bisa diubah ke `float`; jika bukan angka cetak `"Masukkan angka valid."` lalu ulangi. Kembalikan `float`.

PENDEKATAN (PSEUDOCODE)

```
ULANGI selamanya:
    COBA: RETURN float(input(prompt))
    TANGKAP ValueError: cetak peringatan
```

SOLUSI

```python
def input_angka(prompt):
    while True:
        try:
            return float(input(prompt))
        except ValueError:
            print("Masukkan angka valid.")
```

HASIL PEMERIKSA

```
Masukkan angka valid.
Masukkan angka valid.
✅ L17 benar!
```

Latihan L18 — Validasi dengan raise

Buat `set_ipk(ipk)` yang mengembalikan `ipk` jika 0–4, selain itu **melempar** `ValueError("IPK harus 0–4")`.

SOLUSI

```python
def set_ipk(ipk):
    if not 0 <= ipk <= 4:
        raise ValueError("IPK harus 0–4")
    return ipk
```

HASIL PEMERIKSA

```
✅ L18 benar!
```

Latihan L19 — Simpan dan muat daftar (teks)

Buat `simpan_daftar(path, daftar)` (tulis satu item per baris) dan `muat_daftar(path)` (kembalikan list string tanpa newline; jika file **tidak ada** kembalikan `[]`).

PENDEKATAN (PSEUDOCODE)

```
simpan: gabungkan item dengan "\n" lalu tulis
muat  : jika file tidak ada → []; selain itu baca dan splitlines()
```

SOLUSI

```
def simpan_daftar(path, daftar):
    Path(path).write_text("\n".join(daftar) + "\n", encoding="utf-8")

def muat_daftar(path):
    p = Path(path)
    if not p.exists():
        return []
    return p.read_text(encoding="utf-8").splitlines()
```

HASIL PEMERIKSA

✅ L19 benar!

Latihan L20 — Simpan dan muat CSV

Buat `simpan_csv(path, rows)` (rows = list of dict; header dari key dict pertama) dan `muat_csv(path)` (kembalikan list of dict; file tidak ada → `[]`).

PENDEKATAN (PSEUDOCODE)

```
simpan: buka file (newline="") → DictWriter(fieldnames) → writeheader → writerows
muat  : file tidak ada → []; selain itu list(DictReader(file))
```

SOLUSI

```
import csv

def simpan_csv(path, rows):
    if not rows:
        return
    with open(path, "w", newline="", encoding="utf-8") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
        w.writeheader()
        w.writerows(rows)

def muat_csv(path):
    p = Path(path)
    if not p.exists():
        return []
    with open(p, "r", encoding="utf-8") as f:
        return list(csv.DictReader(f))
```

HASIL PEMERIKSA

✅ L20 benar!

Latihan P1 — Mini Project 1 — Pencatat Kegiatan Akademik

Lengkapi function (data = list of dict, disimpan ke CSV):

- `tambah_kegiatan(data, tanggal, jenis, deskripsi, status="belum")` → tambah dict `{"tanggal","jenis","deskripsi","status"}`; `jenis` harus salah satu `tugas/kuliah/ujian`, jika tidak kembalikan `False`; sukses → `True`.
- `filter_kegiatan(data, kunci)` → list kegiatan yang `jenis` **atau** `status`-nya sama dengan `kunci`.
- `tandai_selesai(data, indeks)` → ubah `status` jadi `"selesai"` (indeks mulai 1); indeks tidak valid → `False`.

- `simpan_kegiatan(path, data)` dan `muat_kegiatan(path)` memakai `simpan_csv` / `muat_csv`.

SOLUSI

```
JENIS_VALID = ("tugas", "kuliah", "ujian")

def tambah_kegiatan(data, tanggal, jenis, deskripsi, status="belum"):
    if jenis not in JENIS_VALID:
        return False
    data.append({"tanggal": tanggal, "jenis": jenis, "deskripsi": deskripsi, "status": status})
    return True

def filter_kegiatan(data, kunci):
    return [k for k in data if kunci in (k["jenis"], k["status"])]

def tandai_selesai(data, indeks):
    if not 1 <= indeks <= len(data):
        return False
    data[indeks - 1]["status"] = "selesai"
    return True

def simpan_kegiatan(path, data):
    simpan_csv(path, data)

def muat_kegiatan(path):
    return muat_csv(path)
```

HASIL PEMERIKSA

```
✅ P1 benar!
```

Latihan P2 — Mini Project 2 — Perpustakaan Mini

Data buku: list of dict `{"judul","penulis","tahun","status"}` (`status` = `"tersedia"` / `"dipinjam"`). Lengkapi:

- `tambah_buku(buku, judul, penulis, tahun)` → status awal `"tersedia"`.
- `pinjam(buku, judul)` → ubah ke `"dipinjam"`, kembalikan `True`; gagal (tidak ada / sudah dipinjam) → `False`. Pencocokan judul **tidak peka huruf besar/kecil**.
- `kembalikan(buku, judul)` → ubah ke `"tersedia"`; judul tidak ada → `False`.
- `cari(buku, kata)` → list buku yang `judul` **atau** `penulis`-nya mengandung `kata` (tidak peka huruf besar/kecil).

SOLUSI

```python
def tambah_buku(buku, judul, penulis, tahun):
    buku.append({"judul": judul, "penulis": penulis, "tahun": tahun, "status": "tersedia"})

def pinjam(buku, judul):
    for b in buku:
        if b["judul"].lower() == judul.lower() and b["status"] == "tersedia":
            b["status"] = "dipinjam"
            return True
    return False

def kembalikan(buku, judul):
    for b in buku:
        if b["judul"].lower() == judul.lower():
            b["status"] = "tersedia"
            return True
    return False

def cari(buku, kata):
    k = kata.lower()
    return [b for b in buku if k in b["judul"].lower() or k in b["penulis"].lower()]
```

HASIL PEMERIKSA

```
✅ P2 benar!
```

3

Hari 3 — Object-Oriented Programming

Class, inheritance, encapsulation, special methods

A. Class, Object, Attribute, Method

A. Class, Object, Attribute, Method

Ringkas: `class Nama:` mendefinisikan cetakan; `__init__(self, ...)` dipanggil saat object dibuat dan mengisi *instance attribute* (`self.x = ...`). Method adalah function di dalam class dengan `self` sebagai parameter pertama. *Class attribute* dipakai bersama semua object.

Latihan L1 — Class Mahasiswa

Buat class `Mahasiswa` dengan `__init__(self, nama, nim, prodi, ipk=0.0)`, method `info()` yang mengembalikan `"<nama> (<nim>) – <prodi>, IPK <ipk>"`, dan `update_ipk(baru)` yang mengubah `ipk` bila 0–4 lalu mengembalikan `True`; selain itu `False` (ipk tidak berubah).

PENDEKATAN (PSEUDOCODE)

```
CLASS Mahasiswa
    __init__(nama, nim, prodi, ipk=0.0): simpan ke self.*
    info(): f-string sesuai format
    update_ipk(baru): JIKA 0..4 → ubah self.ipk, True; SELAIN → False
```

SOLUSI

```python
class Mahasiswa:
    def __init__(self, nama, nim, prodi, ipk=0.0):
        self.nama = nama
        self.nim = nim
        self.prodi = prodi
        self.ipk = ipk

    def info(self):
        return f"{self.nama} ({self.nim}) – {self.prodi}, IPK {self.ipk}"

    def update_ipk(self, baru):
        if 0 <= baru <= 4:
            self.ipk = baru
            return True
        return False
```

HASIL PEMERIKSA

```
✅ L1 benar!
```

Latihan L2 — Class attribute sebagai penghitung

Buat class `Produk` dengan class attribute `jumlah = 0` yang bertambah setiap object dibuat (di `__init__`), dan instance attribute `nama`.

PENDEKATAN (PSEUDOCODE)

```
jumlah ← 0 di level class
di __init__: simpan nama, lalu Produk.jumlah += 1
```

SOLUSI

```
class Produk:
    jumlah = 0

    def __init__(self, nama):
        self.nama = nama
        Produk.jumlah += 1
```

HASIL PEMERIKSA

✅ L2 benar!

Latihan L3 — RekeningBank

Buat `RekeningBank(pemilik, saldo=0)` dengan method `setor(jumlah)` (hanya jumlah > 0, kembalikan `True/False`) dan `tarik(jumlah)` (hanya `0 < jumlah <= saldo`, kembalikan `True/False`). Atribut `saldo` bisa dibaca langsung.

SOLUSI

```
class RekeningBank:
    def __init__(self, pemilik, saldo=0):
        self.pemilik = pemilik
        self.saldo = saldo

    def setor(self, jumlah):
        if jumlah > 0:
            self.saldo += jumlah
            return True
        return False

    def tarik(self, jumlah):
        if 0 < jumlah <= self.saldo:
            self.saldo -= jumlah
            return True
        return False
```

HASIL PEMERIKSA

✅ L3 benar!

Latihan L4 — Kendaraan, Mobil, Motor

Buat `Kendaraan(merek, tahun)` dengan `info()` → `"<merek> (<tahun>)"`. Turunkan `Mobil(merek, tahun, jumlah_pintu)` dengan `info()` → `"[Mobil] <info induk> — <n> pintu"` dan `Motor(merek, tahun, jenis)` dengan `info()` → `"[Motor] <info induk> — <jenis>"`. Gunakan `super()`.

PENDEKATAN (PSEUDOCODE)

```
Mobil.__init__: super().__init__(merek, tahun); simpan jumlah_pintu
Mobil.info: f"[Mobil] {super().info()} — {pintu} pintu"
Motor serupa dengan 'jenis'
```

SOLUSI

```
class Kendaraan:
    def __init__(self, merek, tahun):
        self.merek = merek
        self.tahun = tahun

    def info(self):
        return f"{self.merek} ({self.tahun})"

class Mobil(Kendaraan):
    def __init__(self, merek, tahun, jumlah_pintu):
        super().__init__(merek, tahun)
        self.jumlah_pintu = jumlah_pintu

    def info(self):
        return f"[Mobil] {super().info()} — {self.jumlah_pintu} pintu"

class Motor(Kendaraan):
    def __init__(self, merek, tahun, jenis):
        super().__init__(merek, tahun)
        self.jenis = jenis

    def info(self):
        return f"[Motor] {super().info()} — {self.jenis}"
```

HASIL PEMERIKSA

> ✅ L4 benar!

Latihan L5 — Polimorfisme: Pegawai dan Manager

Buat `Pegawai(nama, gaji_pokok)` dengan `gaji()` → `gaji_pokok`, dan `Manager(nama, gaji_pokok, tunjangan)` yang **meng-override** `gaji()` → `gaji_pokok + tunjangan`. Lalu isi list `tim` dengan `Pegawai("Rina", 7_000_000)` dan `Manager("Budi", 9_000_000, 3_000_000)`, dan hitung `total_gaji` dengan **satu loop** yang memanggil `gaji()` pada setiap object.

SOLUSI

```
class Pegawai:
    def __init__(self, nama, gaji_pokok):
        self.nama = nama
        self.gaji_pokok = gaji_pokok

    def gaji(self):
        return self.gaji_pokok

class Manager(Pegawai):
    def __init__(self, nama, gaji_pokok, tunjangan):
        super().__init__(nama, gaji_pokok)
        self.tunjangan = tunjangan

    def gaji(self):
        return self.gaji_pokok + self.tunjangan

tim = [Pegawai("Rina", 7_000_000), Manager("Budi", 9_000_000, 3_000_000)]
total_gaji = 0
for p in tim:
    total_gaji += p.gaji()
```

HASIL PEMERIKSA

> ✅ L5 benar!

Latihan L6 — Encapsulation — Rekening aman

Buat `Rekening(pemilik, saldo=0)` yang menyimpan saldo di `self._saldo`, dengan `get_saldo()`, `setor(jumlah)` (hanya > 0) dan `tarik(jumlah)` (hanya `0 < jumlah <= saldo`). Kedua method mengembalikan `True/False`. **Jangan** akses `_saldo` dari luar class.

SOLUSI

```
class Rekening:
    def __init__(self, pemilik, saldo=0):
        self.pemilik = pemilik
        self._saldo = saldo

    def get_saldo(self):
        return self._saldo

    def setor(self, jumlah):
        if jumlah > 0:
            self._saldo += jumlah
            return True
        return False

    def tarik(self, jumlah):
        if 0 < jumlah <= self._saldo:
            self._saldo -= jumlah
            return True
        return False
```

HASIL PEMERIKSA

> ✅ L6 benar!

Latihan L7 — Special methods pada Buku

Buat `Buku(judul, penulis, halaman)` dengan: `__str__` → `"'<judul>' oleh <penulis>"`; `__repr__` → `"Buku('<judul>', '<penulis>', <halaman>)"`; `__len__` → `halaman`; `__eq__` → sama bila `judul` **dan** `penulis` sama.

SOLUSI

```
class Buku:
    def __init__(self, judul, penulis, halaman):
        self.judul = judul
        self.penulis = penulis
        self.halaman = halaman

    def __str__(self):
        return f"'{self.judul}' oleh {self.penulis}"

    def __repr__(self):
        return f"Buku('{self.judul}', '{self.penulis}', {self.halaman})"

    def __len__(self):
        return self.halaman

    def __eq__(self, other):
        return self.judul == other.judul and self.penulis == other.penulis
```

HASIL PEMERIKSA

> ✅ L7 benar!

Latihan L8 — Titik dan jarak

Buat `Titik(x, y)` dengan `jarak_ke(lain)` (jarak Euclid; boleh pakai `math.hypot`) dan `__str__` → `"(x, y)"`.

SOLUSI

```
import math

class Titik:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def jarak_ke(self, lain):
        return math.hypot(self.x - lain.x, self.y - lain.y)

    def __str__(self):
        return f"({self.x}, {self.y})"
```

HASIL PEMERIKSA

> ✅ L8 benar!

Latihan L9 — Context manager FileCatatan

Buat class `FileCatatan(path, mode="w")` sebagai context manager: `__enter__` membuka file (encoding utf-8) dan mengembalikannya, `__exit__` menutup file **dan tidak menelan exception** (kembalikan `False`).

SOLUSI

```
class FileCatatan:
    def __init__(self, path, mode="w"):
        self.path = path
        self.mode = mode
        self.file = None

    def __enter__(self):
        self.file = open(self.path, self.mode, encoding="utf-8")
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        return False
```

HASIL PEMERIKSA

```
✅ L9 benar!
```

Latihan L10 — Perbaiki mutable default

Perbaiki class `Kelas` agar setiap object punya list `mahasiswa` sendiri: pakai default `None` lalu buat list baru di `__init__`.

PENDEKATAN (PSEUDOCODE)

```
def __init__(self, nama, mahasiswa=None):
    self.mahasiswa ← mahasiswa JIKA bukan None, SELAIN ITU list kosong baru
```

SOLUSI

```
class Kelas:
    def __init__(self, nama, mahasiswa=None):
        self.nama = nama
        self.mahasiswa = mahasiswa if mahasiswa is not None else []
```

HASIL PEMERIKSA

```
✅ L10 benar!
```

Latihan P1 — Mini Project 1 — Manajemen Tugas Kuliah

Buat dua class:

- `Tugas(matkul, deskripsi, deadline)` dengan `selesai = False`, `tandai_selesai()`, dan `__str__` → `"[✓] matkul: deskripsi (deadline)"` bila selesai, `"[○] ..."` bila belum.
- `ManajerTugas` dengan `daftar` (list), `tambah(tugas)`, `tampilkan()` (kembalikan **list string** bernomor mulai 1: `"1. [○] ..."`), `tandai(index)` (index mulai 1; kembalikan `True` / `False` bila index tidak valid), dan `belum_selesai()` (list `Tugas` yang belum selesai).

SOLUSI

```
class Tugas:
    def __init__(self, matkul, deskripsi, deadline):
        self.matkul = matkul
        self.deskripsi = deskripsi
        self.deadline = deadline
        self.selesai = False

    def tandai_selesai(self):
        self.selesai = True

    def __str__(self):
        st = "✓" if self.selesai else "○"
        return f"[{st}] {self.matkul}: {self.deskripsi} ({self.deadline})"

class ManajerTugas:
    def __init__(self):
        self.daftar = []

    def tambah(self, tugas):
        self.daftar.append(tugas)

    def tampilkan(self):
        return [f"{i}. {t}" for i, t in enumerate(self.daftar, 1)]

    def tandai(self, index):
        if 1 <= index <= len(self.daftar):
            self.daftar[index - 1].tandai_selesai()
            return True
        return False

    def belum_selesai(self):
        return [t for t in self.daftar if not t.selesai]
```

HASIL PEMERIKSA

```
✅ P1 benar!
```

Latihan P2 — Mini Project 2 — Perpustakaan dengan Inheritance

Buat hierarki:

- `ItemPerpustakaan(judul)` dengan `tersedia = True`, `pinjam()` (jika tersedia → set `False`, kembalikan `True`; selain itu `False`), `kembalikan()`.
- `Buku(judul, penulis, halaman)` → `__str__`: `"Buku: <judul> – <penulis> [tersedia/dipinjam]"`.
- `Jurnal(judul, volume, tahun)` → `__str__`: `"Jurnal: <judul> Vol.<volume> (<tahun>) [tersedia/dipinjam]"`.
- `Perpustakaan` dengan `koleksi`, `tambah(item)`, `cari(keyword)` (judul mengandung kata, tak peka huruf besar/kecil), `pinjam(judul)` (kembalikan `True/False`, judul tak peka huruf besar/kecil), dan `daftar()` → list `str(item)` seluruh koleksi (**polimorfisme**: satu loop untuk semua jenis).

SOLUSI

```
class ItemPerpustakaan:
    def __init__(self, judul):
        self.judul = judul
        self.tersedia = True
```

```
    def pinjam(self):
        if self.tersedia:
            self.tersedia = False
            return True
        return False

    def kembalikan(self):
        self.tersedia = True

    def _status(self):
        return "tersedia" if self.tersedia else "dipinjam"

class Buku(ItemPerpustakaan):
    def __init__(self, judul, penulis, halaman):
        super().__init__(judul)
        self.penulis = penulis
        self.halaman = halaman

    def __str__(self):
        return f"Buku: {self.judul} — {self.penulis} [{self._status()}]"

class Jurnal(ItemPerpustakaan):
    def __init__(self, judul, volume, tahun):
        super().__init__(judul)
        self.volume = volume
        self.tahun = tahun

    def __str__(self):
        return f"Jurnal: {self.judul} Vol.{self.volume} ({self.tahun}) [{self._status()}]"

class Perpustakaan:
    def __init__(self):
        self.koleksi = []

    def tambah(self, item):
        self.koleksi.append(item)

    def cari(self, keyword):
        k = keyword.lower()
        return [i for i in self.koleksi if k in i.judul.lower()]

    def pinjam(self, judul):
        for i in self.koleksi:
            if i.judul.lower() == judul.lower():
                return i.pinjam()
        return False

    def daftar(self):
        return [str(i) for i in self.koleksi]
```

HASIL PEMERIKSA

```
✅ P2 benar!
```

4

Hari 4 — NumPy & Pandas

Array, DataFrame, cleaning, groupby, merge, pivot

A. NumPy

A. NumPy

Ringkas: `np.array` menyimpan data numerik seragam sehingga operasi vektor jauh lebih cepat dan ringkas daripada loop. `arr[baris, kolom]` untuk indexing, `arr[a:b, c:d]` untuk slicing, `arr[arr > 60]` untuk *boolean masking*. Operasi antar array berbeda bentuk memakai *broadcasting*. `axis=0` = turun (per kolom), `axis=1` = ke samping (per baris).

Latihan L1 — Membuat dan membentuk ulang array

Buat `m` = array `0..11` yang dibentuk menjadi matriks **3 baris × 4 kolom** (`np.arange` + `reshape`). Lalu hitung `jumlah_kolom` (jumlah tiap kolom) dan `jumlah_baris` (jumlah tiap baris).

PENDEKATAN (PSEUDOCODE)

```
m ← arange(12) lalu reshape(3, 4)
jumlah per kolom → sum(axis=0)   jumlah per baris → sum(axis=1)
```

SOLUSI

```python
m = np.arange(12).reshape(3, 4)
jumlah_kolom = m.sum(axis=0)
jumlah_baris = m.sum(axis=1)
```

HASIL PEMERIKSA

```
✅ L1 benar!
```

Latihan L2 — Indexing dan slicing

Dari `arr`, isi: `elemen` (baris 0, kolom 2), `submat` (baris 0–1, kolom 1–2), dan `kolom_terakhir` (seluruh baris pada kolom terakhir).

SOLUSI

```python
arr = np.array([[10, 20, 30, 40], [50, 60, 70, 80], [90, 100, 110, 120]])
elemen = arr[0, 2]
submat = arr[0:2, 1:3]
kolom_terakhir = arr[:, -1]
```

HASIL PEMERIKSA

```
✅ L2 benar!
```

Latihan L3 — Boolean masking

Dari `arr` (sama seperti L2), isi `besar` (semua elemen > 60) dan `kelipatan_20` (elemen yang habis dibagi 20).

PENDEKATAN (PSEUDOCODE)

```
besar ← arr[arr > 60]
kelipatan_20 ← arr[arr % 20 == 0]
```

SOLUSI

```
arr = np.array([[10, 20, 30, 40], [50, 60, 70, 80], [90, 100, 110, 120]])
besar = arr[arr > 60]
kelipatan_20 = arr[arr % 20 == 0]
```

HASIL PEMERIKSA

✅ L3 benar!

Latihan L4 — np.where

Buat `hasil` : salinan `arr` dengan elemen ≤ 70 diganti `-1` dan elemen lain tetap (`np.where`).

SOLUSI

```
arr = np.array([[10, 20, 30, 40], [50, 60, 70, 80], [90, 100, 110, 120]])
hasil = np.where(arr > 70, arr, -1)
```

HASIL PEMERIKSA

✅ L4 benar!

Latihan L5 — Broadcasting: nilai akhir berbobot

Setiap baris `nilai` = `[UTS, UAS, Tugas]` satu mahasiswa. Dengan bobot `[0.3, 0.4, 0.3]` , hitung `nilai_akhir` (array 3 elemen) **tanpa loop** (perkalian broadcasting lalu `sum(axis=1)`).

PENDEKATAN (PSEUDOCODE)

```
nilai * bobot  → setiap baris dikali bobot (broadcasting)
.sum(axis=1)   → jumlahkan per baris
```

SOLUSI

```
nilai = np.array([[80, 90, 70], [60, 75, 85], [90, 95, 88]])
bobot = np.array([0.3, 0.4, 0.3])
nilai_akhir = (nilai * bobot).sum(axis=1)
```

HASIL PEMERIKSA

✅ L5 benar!

Latihan L6 — Statistik bagian tengah matriks

`a` adalah matriks 6×6 berisi 1..36. Ambil bagian tengah 4×4 (baris & kolom indeks 1–4) ke `tengah` , lalu hitung `rata` , `simpangan` (`std`).

SOLUSI

```
a = np.arange(1, 37).reshape(6, 6)
tengah = a[1:5, 1:5]
rata = tengah.mean()
simpangan = tengah.std()
```

HASIL PEMERIKSA

✅ L6 benar!

Latihan L7 — Normalisasi min-max

Ubah `x` menjadi skala 0–1: `(x - min) / (max - min)`. Simpan di `x_norm`.

SOLUSI

```
x = np.array([2.0, 4.0, 6.0, 8.0])
x_norm = (x - x.min()) / (x.max() - x.min())
```

HASIL PEMERIKSA

```
✅ L7 benar!
```

Latihan L8 — Korelasi dua variabel

Dengan `a` dan `b` (acak ber-seed), hitung koefisien korelasi Pearson ke `korelasi` memakai `np.corrcoef`.

PENDEKATAN (PSEUDOCODE)

```
np.corrcoef(a, b) menghasilkan matriks 2×2; ambil elemen [0, 1]
```

SOLUSI

```
np.random.seed(42)
a = np.random.randn(100)
b = np.random.randn(100)
korelasi = np.corrcoef(a, b)[0, 1]
```

HASIL PEMERIKSA

```
✅ L8 benar!
```

Latihan L9 — Isi diagonal

Mulai dari matriks identitas 5×5 `m = np.eye(5)`, isi diagonalnya dengan `[10, 20, 30, 40, 50]` (`np.fill_diagonal`).

SOLUSI

```
m = np.eye(5)
np.fill_diagonal(m, [10, 20, 30, 40, 50])
```

HASIL PEMERIKSA

```
✅ L9 benar!
```

Latihan L10 — Series

Buat `s` = Series `[85, 90, 78, 92]` berindeks `["Ani","Budi","Citra","Dedi"]`. Isi `tinggi` dengan elemen Series yang ≥ 90 dan `rata` dengan rata-rata Series.

SOLUSI

```
s = pd.Series([85, 90, 78, 92], index=["Ani", "Budi", "Citra", "Dedi"])
tinggi = s[s >= 90]
rata = s.mean()
```

HASIL PEMERIKSA

```
✅ L10 benar!
```

Latihan L11 — Membuat dan memeriksa DataFrame

Buat `df` dari kamus di bawah, lalu isi `n_baris`, `n_kolom`, dan `nama_kolom` (list), serta `ipk_rata` (rata-rata kolom `ipk`).

SOLUSI

```
data = {
    "nama": ["Ani", "Budi", "Citra", "Dedi", "Eka", "Fajar"],
    "prodi": ["IF", "SI", "IF", "SI", "IF", "SI"],
    "ipk": [3.75, 3.20, 3.90, 2.85, 3.55, 3.10],
}
df = pd.DataFrame(data)
n_baris, n_kolom = df.shape
nama_kolom = df.columns.tolist()
ipk_rata = df["ipk"].mean()
```

HASIL PEMERIKSA

```
✅ L11 benar!
```

Latihan L12 — Seleksi dan filter

Dengan `df` dari L11: `cumlaude` = list **nama** mahasiswa ber-IPK ≥ 3.5; `if_tinggi` = DataFrame mahasiswa prodi IF **dan** IPK ≥ 3.5.

PENDEKATAN (PSEUDOCODE)

```
gabungkan kondisi dengan & dan beri kurung pada tiap kondisi
```

SOLUSI

```
df = pd.DataFrame({
    "nama": ["Ani", "Budi", "Citra", "Dedi", "Eka", "Fajar"],
    "prodi": ["IF", "SI", "IF", "SI", "IF", "SI"],
    "ipk": [3.75, 3.20, 3.90, 2.85, 3.55, 3.10],
})
cumlaude = df.loc[df["ipk"] >= 3.5, "nama"].tolist()
if_tinggi = df[(df["prodi"] == "IF") & (df["ipk"] >= 3.5)]
```

HASIL PEMERIKSA

```
✅ L12 benar!
```

Latihan L13 — Kolom baru: status dan grade

Tambahkan kolom `status` (`"Cumlaude"` jika ipk ≥ 3.5, selain itu `"Reguler"`, pakai `np.where`) dan kolom `grade` memakai `.apply` dengan function `tentukan_grade(ipk)`: `A` (≥3.5), `B` (≥3.0), `C` (≥2.5), selain itu `D`.

SOLUSI

```
df = pd.DataFrame({"nama": ["Ani", "Budi", "Dedi"], "ipk": [3.75, 3.20, 2.85]})

def tentukan_grade(ipk):
    if ipk >= 3.5:
        return "A"
    if ipk >= 3.0:
        return "B"
    if ipk >= 2.5:
        return "C"
    return "D"

df["status"] = np.where(df["ipk"] >= 3.5, "Cumlaude", "Reguler")
df["grade"] = df["ipk"].apply(tentukan_grade)
```

HASIL PEMERIKSA

> ✅ L13 benar!

Latihan L14 — Sorting

Urutkan `df` berdasarkan `ipk` menurun ke `urut`, dan ambil 2 baris dengan IPK tertinggi ke `top2` (`nlargest`).

SOLUSI

```
df = pd.DataFrame({"nama": ["Ani", "Budi", "Citra", "Dedi"], "ipk": [3.5, 3.9, 3.2, 3.7]})
urut = df.sort_values("ipk", ascending=False)
top2 = df.nlargest(2, "ipk")
```

HASIL PEMERIKSA

> ✅ L14 benar!

Latihan L15 — Missing values dan duplikat

Dari `kotor`: isi `ipk` yang kosong dengan **median**, isi `prodi` yang kosong dengan `"Unknown"`, lalu hapus baris duplikat penuh. Simpan hasil di `bersih`.

PENDEKATAN (PSEUDOCODE)

```
salin df → isi ipk dengan median → isi prodi dengan "Unknown" → drop_duplicates()
```

SOLUSI

```
kotor = pd.DataFrame({
    "nama": ["Ani", "Budi", "Budi", "Dedi"],
    "ipk": [3.7, np.nan, np.nan, 2.8],
    "prodi": ["IF", "SI", "SI", None],
})
bersih = kotor.copy()
bersih["ipk"] = bersih["ipk"].fillna(bersih["ipk"].median())
bersih["prodi"] = bersih["prodi"].fillna("Unknown")
bersih = bersih.drop_duplicates()
```

HASIL PEMERIKSA

> ✅ L15 benar!

Latihan L16 — Konversi tipe data

Kolom `umur` berisi teks, sebagian tidak valid. Ubah ke angka dengan `pd.to_numeric(..., errors="coerce")` dan simpan di kolom `umur_num`; lalu `n_tidak_valid` = jumlah NaN pada kolom itu.

SOLUSI

```
df = pd.DataFrame({"umur": ["20", "21", "n/a", "19"]})
df["umur_num"] = pd.to_numeric(df["umur"], errors="coerce")
n_tidak_valid = int(df["umur_num"].isna().sum())
```

HASIL PEMERIKSA

> ✅ L16 benar!

Latihan L17 — GroupBy

Dari `df`: `rata_prodi` = rata-rata `ipk` per `prodi` (Series), dan `ringkas` = DataFrame hasil `agg` per prodi dengan kolom `rata` (mean ipk), `n` (jumlah mahasiswa).

PENDEKATAN (PSEUDOCODE)

```
df.groupby("prodi").agg(rata=("ipk", "mean"), n=("nama", "count"))
```

SOLUSI

```
df = pd.DataFrame({
    "nama": ["Ani", "Budi", "Citra", "Dedi", "Eka"],
    "prodi": ["IF", "SI", "IF", "SI", "IF"],
    "ipk": [3.75, 3.20, 3.90, 2.85, 3.55],
})
rata_prodi = df.groupby("prodi")["ipk"].mean()
ringkas = df.groupby("prodi").agg(rata=("ipk", "mean"), n=("nama", "count"))
```

HASIL PEMERIKSA

> ✅ L17 benar!

Latihan L18 — Merge

Gabungkan `mahasiswa` dan `nilai` pada `nim`: `inner` (hanya yang ada di keduanya) dan `kiri` (semua mahasiswa; `skor` kosong bila tak ada nilai).

SOLUSI

```
mahasiswa = pd.DataFrame({"nim": ["2201", "2202", "2203"], "nama": ["Ani", "Budi", "Citra"]})
nilai = pd.DataFrame({"nim": ["2201", "2202", "2204"], "skor": [85, 90, 78]})
inner = pd.merge(mahasiswa, nilai, on="nim", how="inner")
kiri = pd.merge(mahasiswa, nilai, on="nim", how="left")
```

HASIL PEMERIKSA

> ✅ L18 benar!

Latihan L19 — Pivot table dan CSV

Buat `pivot`: rata-rata `nilai` dengan `nama` sebagai baris dan `matkul` sebagai kolom. Lalu simpan `pivot` ke `latihan_hari4/pivot.csv` dan baca kembali ke `kembali` (`index_col=0`).

SOLUSI

```
data = pd.DataFrame({
    "nama": ["Ani", "Ani", "Budi", "Budi"],
    "matkul": ["Python", "BD", "Python", "BD"],
    "nilai": [85, 90, 78, 88],
})
pivot = data.pivot_table(values="nilai", index="nama", columns="matkul", aggfunc="mean")
pivot.to_csv(WORK / "pivot.csv")
kembali = pd.read_csv(WORK / "pivot.csv", index_col=0)
```

HASIL PEMERIKSA

✅ L19 benar!

Latihan P1 — Project 1 — Kalkulator Nilai Akhir (Dasar)

Buat `hitung_nilai(df)` yang menerima DataFrame berkolom `nilai_uts`, `nilai_uas`, `nilai_tugas` dan mengembalikan **salinan** dengan kolom baru `nilai_akhir` (30% UTS + 40% UAS + 30% Tugas) dan `grade` (`A` ≥85, `B` ≥70, `C` ≥55, `D` ≥40, selain itu `E`). Buat juga `top_n(df, n)` → n baris dengan `nilai_akhir` tertinggi.

SOLUSI

```
def hitung_nilai(df):
    hasil = df.copy()
    hasil["nilai_akhir"] = hasil["nilai_uts"] * 0.3 + hasil["nilai_uas"] * 0.4 +
hasil["nilai_tugas"] * 0.3

    def grade(n):
        if n >= 85: return "A"
        if n >= 70: return "B"
        if n >= 55: return "C"
        if n >= 40: return "D"
        return "E"

    hasil["grade"] = hasil["nilai_akhir"].apply(grade)
    return hasil

def top_n(df, n):
    return df.nlargest(n, "nilai_akhir")
```

HASIL PEMERIKSA

✅ P1 benar!

Latihan P2 — Project 2 — Cleaning Dataset Kotor

Buat `bersihkan(df)` untuk DataFrame berkolom `nama`, `ipk`, `prodi`, `umur`: (1) `strip` spasi di `nama`, (2) `nama` kosong → `"Unknown"`, (3) `ipk` kosong → median, (4) `prodi` kosong → `"Unknown"`, (5) `umur` → angka (`to_numeric`, gagal → NaN), (6) buang duplikat penuh **setelah** langkah 1–5. Kembalikan salinan; reset index.

PENDEKATAN (PSEUDOCODE)

```
urutan penting: strip dulu → baru drop_duplicates (agar "Ani" == " Ani ")
```

SOLUSI

```python
def bersihkan(df):
    d = df.copy()
    d["nama"] = d["nama"].fillna("Unknown").str.strip()
    d["ipk"] = d["ipk"].fillna(d["ipk"].median())
    d["prodi"] = d["prodi"].fillna("Unknown")
    d["umur"] = pd.to_numeric(d["umur"], errors="coerce")
    return d.drop_duplicates().reset_index(drop=True)
```

HASIL PEMERIKSA

✅ P2 benar!

Latihan P3 — Project 3 — Merge Multi-Tabel (Menengah)

Buat `gabung_dan_ranking(mahasiswa, nilai, kehadiran)` yang meng-merge ketiga tabel pada `nim`, menambah kolom `rank` (peringkat `nilai_akhir`, 1 = tertinggi, bertipe int), dan mengembalikan DataFrame **terurut menurut rank**. Buat juga `korelasi_hadir_nilai(df)` → korelasi `persen_hadir` dan `nilai_akhir`.

SOLUSI

```python
def gabung_dan_ranking(mahasiswa, nilai, kehadiran):
    df = mahasiswa.merge(nilai, on="nim").merge(kehadiran, on="nim")
    df["rank"] = df["nilai_akhir"].rank(ascending=False).astype(int)
    return df.sort_values("rank").reset_index(drop=True)

def korelasi_hadir_nilai(df):
    return df["persen_hadir"].corr(df["nilai_akhir"])
```

HASIL PEMERIKSA

✅ P3 benar!

Latihan P4 — Project 4 — Pivot dan Reporting (Menengah–Tinggi)

Buat `laporan(data)` untuk DataFrame `nama`, `matkul`, `nilai`: pivot rata-rata `nilai` (baris `nama`, kolom `matkul`), tambah kolom `rata_rata` (rata-rata semua matkul per baris) dan `status` (`"Lulus"` jika `rata_rata >= 75`, selain itu `"Tidak Lulus"`). Kembalikan DataFrame pivot tersebut.

SOLUSI

```python
def laporan(data):
    pivot = data.pivot_table(values="nilai", index="nama", columns="matkul", aggfunc="mean")
    pivot["rata_rata"] = pivot.mean(axis=1)
    pivot["status"] = np.where(pivot["rata_rata"] >= 75, "Lulus", "Tidak Lulus")
    return pivot
```

HASIL PEMERIKSA

✅ P4 benar!

Latihan P5 — Project 5 — Ringkasan EDA (Tinggi)

Buat `ringkasan_eda(df)` yang mengembalikan dict dengan kunci: `baris`, `kolom`, `missing` (dict kolom → jumlah NaN, **hanya** kolom yang punya missing), `duplikat` (jumlah baris duplikat), `numerik` (list nama kolom numerik), `kategorik` (list nama kolom non-numerik). Fungsi ini bisa dipakai pada dataset apa pun (mis.

Titanic) — di sini diuji dengan data kecil.

SOLUSI

```python
def ringkasan_eda(df):
    miss = df.isna().sum()
    return {
        "baris": len(df),
        "kolom": df.shape[1],
        "missing": {k: int(v) for k, v in miss.items() if v > 0},
        "duplikat": int(df.duplicated().sum()),
        "numerik": df.select_dtypes(include="number").columns.tolist(),
        "kategorik": df.select_dtypes(exclude="number").columns.tolist(),
    }
```

HASIL PEMERIKSA

```
✅ P5 benar!
```

5

Hari 5 — Visualisasi, API & Scraping

Matplotlib, Seaborn, API/JSON, scraping, arXiv

A. Matplotlib

A. Matplotlib

Ringkas: `fig, ax = plt.subplots()` membuat kanvas; `ax.plot/bar/hist/scatter/pie` menggambar; `ax.set_title/xlabel/ylabel` memberi label; `ax.legend()` memunculkan legenda; `fig.tight_layout()` merapikan. Setiap grafik wajib punya **judul** dan **label sumbu**.

Latihan L1 — Line plot dua seri

Buat function `plot_garis(x, seri, judul)` dengan `seri` berupa dict `{nama: list_y}`. Gambar satu garis per seri (beri `label`), pasang `judul`, label sumbu x dan y (tidak kosong), dan legenda. Kembalikan `fig`.

PENDEKATAN (PSEUDOCODE)

```
fig, ax ← plt.subplots()
UNTUK setiap (nama, y) di seri: ax.plot(x, y, label=nama)
set_title, set_xlabel, set_ylabel, legend → RETURN fig
```

SOLUSI

```python
def plot_garis(x, seri, judul):
    fig, ax = plt.subplots(figsize=(7, 4))
    for nama, y in seri.items():
        ax.plot(x, y, marker="o", label=nama)
    ax.set_title(judul)
    ax.set_xlabel("Bulan")
    ax.set_ylabel("Nilai")
    ax.legend()
    fig.tight_layout()
    return fig
```

HASIL PEMERIKSA

```
✅ L1 benar!
```

Latihan L2 — Bar chart dengan label nilai

Buat `plot_bar(kategori, nilai, judul)`: batang vertikal, tulis angka di atas setiap batang (`ax.text`), beri judul dan label sumbu. Kembalikan `fig`.

PENDEKATAN (PSEUDOCODE)

```
batang ← ax.bar(kategori, nilai)
UNTUK (b, v): ax.text(x_tengah_batang, tinggi + sedikit, str(v))
x_tengah = b.get_x() + b.get_width() / 2
```

SOLUSI

```python
def plot_bar(kategori, nilai, judul):
    fig, ax = plt.subplots(figsize=(7, 4))
    batang = ax.bar(kategori, nilai, color="steelblue", edgecolor="black")
    for b, v in zip(batang, nilai):
        ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 1, str(v), ha="center")
    ax.set_title(judul)
    ax.set_xlabel("Program studi")
    ax.set_ylabel("Jumlah mahasiswa")
    fig.tight_layout()
    return fig
```

HASIL PEMERIKSA

```
✅ L2 benar!
```

Latihan L3 — Histogram dengan garis rata-rata

Buat `plot_hist(data, bins, judul)`: histogram dengan `bins` kotak, garis vertikal merah putus-putus di rata-rata (`ax.axvline`, beri `label` berisi nilai rata-rata), judul, dan label sumbu. Kembalikan `fig`.

SOLUSI

```python
def plot_hist(data, bins, judul):
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.hist(data, bins=bins, color="steelblue", edgecolor="white")
    rata = float(np.mean(data))
    ax.axvline(rata, color="red", linestyle="--", label=f"Rata-rata = {rata:.1f}")
    ax.set_title(judul)
    ax.set_xlabel("Nilai")
    ax.set_ylabel("Frekuensi")
    ax.legend()
    fig.tight_layout()
    return fig
```

HASIL PEMERIKSA

```
✅ L3 benar!
```

Latihan L4 — Dashboard 2×2 dengan subplot

Buat `dashboard()` yang mengembalikan figure 2×2 (`plt.subplots(2, 2)`) berisi: (0,0) garis `[1,4,9,16]`, (0,1) bar `A,B,C → 10,20,15`, (1,0) histogram 200 data acak ber-seed, (1,1) scatter 40 titik acak ber-seed. **Setiap panel punya judul** dan figure punya `suptitle`.

SOLUSI

```
def dashboard():
    rng = np.random.default_rng(7)
    fig, axes = plt.subplots(2, 2, figsize=(9, 7))
    axes[0, 0].plot([1, 2, 3, 4], [1, 4, 9, 16], "b-o"); axes[0, 0].set_title("Garis")
    axes[0, 1].bar(["A", "B", "C"], [10, 20, 15], color="orange"); axes[0, 1].set_title("Bar")
    axes[1, 0].hist(rng.normal(size=200), bins=20, color="green", alpha=0.7); axes[1,
0].set_title("Histogram")
    axes[1, 1].scatter(rng.random(40), rng.random(40), c="red", alpha=0.6); axes[1,
1].set_title("Scatter")
    fig.suptitle("Dashboard 2x2")
    fig.tight_layout()
    return fig
```

HASIL PEMERIKSA

```
✅ L4 benar!
```

Latihan L5 — Simpan grafik ke file

Buat `simpan_grafik(fig, nama_file)` yang menyimpan `fig` ke `latihan_hari5/<nama_file>` dengan `dpi=120` dan `bbox_inches="tight"`, lalu mengembalikan objek `Path` file tersebut.

SOLUSI

```
def simpan_grafik(fig, nama_file):
    path = WORK / nama_file
    fig.savefig(path, dpi=120, bbox_inches="tight")
    return path
```

HASIL PEMERIKSA

```
✅ L5 benar!
```

Latihan L6 — Boxplot per hari

Buat `plot_box(df)` yang menggambar boxplot `total_bill` per `day` (urutan `Thur, Fri, Sat, Sun`) dengan judul `"Total Bill per Hari"`. Kembalikan `ax`.

PENDEKATAN (PSEUDOCODE)

```
sns.boxplot(data=df, x="day", y="total_bill", order=[...])
lalu ax.set_title(...) dan RETURN ax
```

SOLUSI

```
def plot_box(df):
    ax = sns.boxplot(data=df, x="day", y="total_bill", order=["Thur", "Fri", "Sat", "Sun"])
    ax.set_title("Total Bill per Hari")
    return ax
```

HASIL PEMERIKSA

```
✅ L6 benar!
```

Latihan L7 — Heatmap korelasi

Buat `plot_korelasi(df)` yang menggambar heatmap korelasi kolom **numerik** `df` dengan angka (`annot=True`, `fmt=".2f"`, `cmap="coolwarm"`, `center=0`) dan judul `"Korelasi"`. Kembalikan `ax`.

PENDEKATAN (PSEUDOCODE)

```
korelasi ← df.select_dtypes(include="number").corr()
sns.heatmap(korelasi, annot=True, fmt=".2f", cmap="coolwarm", center=0)
```

SOLUSI

```python
def plot_korelasi(df):
    korelasi = df.select_dtypes(include="number").corr()
    ax = sns.heatmap(korelasi, annot=True, fmt=".2f", cmap="coolwarm", center=0)
    ax.set_title("Korelasi")
    return ax
```

HASIL PEMERIKSA

```
✅ L7 benar!
```

Latihan L8 — Countplot dan barplot

Buat `plot_hitung(df)` yang mengembalikan `ax` berisi **countplot** jumlah transaksi per `day` dengan `hue="sex"` (urutan hari `Thur, Fri, Sat, Sun`), judul `"Jumlah Transaksi per Hari"`.

SOLUSI

```python
def plot_hitung(df):
    ax = sns.countplot(data=df, x="day", hue="sex", order=["Thur", "Fri", "Sat", "Sun"])
    ax.set_title("Jumlah Transaksi per Hari")
    return ax
```

HASIL PEMERIKSA

```
✅ L8 benar!
```

Latihan L9 — ambil_json yang tahan error

Buat `ambil_json(url, params=None, timeout=10)`: lakukan GET, `raise_for_status()`, lalu kembalikan `resp.json()`. Jika terjadi `requests.exceptions.RequestException` **atau** respons bukan JSON (`ValueError`), cetak `"Gagal: ..."` dan kembalikan `None`. Pemeriksa menjalankan server HTTP lokal sungguhan.

PENDEKATAN (PSEUDOCODE)

```
COBA: resp ← requests.get(url, params, timeout); resp.raise_for_status(); RETURN resp.json()
TANGKAP RequestException atau ValueError: cetak "Gagal", RETURN None
```

SOLUSI

```
import requests

def ambil_json(url, params=None, timeout=10):
    try:
        resp = requests.get(url, params=params, timeout=timeout)
        resp.raise_for_status()
        return resp.json()
    except (requests.exceptions.RequestException, ValueError) as e:
        print("Gagal:", type(e).__name__)
        return None
```

HASIL PEMERIKSA

```
Gagal: HTTPError
Gagal: JSONDecodeError
Gagal: ReadTimeout
Gagal: ConnectionError
✅ L9 benar!
```

Latihan L10 — json_normalize dan agregasi

Dari `users` (JSON bertingkat), buat `df` dengan `pd.json_normalize`, lalu isi `jumlah_per_kota` (Series jumlah user per `address.city`, urut nama kota menaik).

SOLUSI

```
users = [
    {"id": 1, "name": "A", "address": {"city": "Bandung"}},
    {"id": 2, "name": "B", "address": {"city": "Jakarta"}},
    {"id": 3, "name": "C", "address": {"city": "Bandung"}},
    {"id": 4, "name": "D", "address": {"city": "Surabaya"}},
]
df = pd.json_normalize(users)
jumlah_per_kota = df["address.city"].value_counts().sort_index()
```

HASIL PEMERIKSA

```
✅ L10 benar!
```

Latihan L11 — Mengolah respons cuaca

`respons` meniru JSON Open-Meteo (data **contoh**). Buat `df_cuaca` dengan kolom `tanggal`, `tmax`, `tmin` dari `respons["daily"]`, lalu `hari_terpanas` = tanggal dengan `tmax` tertinggi dan `rentang_rata` = rata-rata (`tmax - tmin`).

SOLUSI

```
respons = {"daily": {
    "time": ["2026-10-01", "2026-10-02", "2026-10-03"],
    "temperature_2m_max": [32.1, 33.4, 31.0],
    "temperature_2m_min": [24.5, 25.0, 24.1],
}}
d = respons["daily"]
df_cuaca = pd.DataFrame({"tanggal": d["time"], "tmax": d["temperature_2m_max"], "tmin":
d["temperature_2m_min"]})
hari_terpanas = df_cuaca.loc[df_cuaca["tmax"].idxmax(), "tanggal"]
rentang_rata = (df_cuaca["tmax"] - df_cuaca["tmin"]).mean()
```

HASIL PEMERIKSA

✅ L11 benar!

Latihan L12 — Scraping HTML kutipan

Dari string `html`, buat `ambil_kutipan(html)` → list dict `{"penulis": ..., "teks": ...}` untuk setiap `<div class="quote">`. Jika elemen `span.text` atau `small.author` tidak ada pada suatu kutipan, **lewati** kutipan itu (jangan crash).

PENDEKATAN (PSEUDOCODE)

```
untuk setiap div.quote:
    teks ← cari span.text ; penulis ← cari small.author
    JIKA keduanya ada → tambahkan dict ; SELAIN ITU lewati
```

SOLUSI

```
from bs4 import BeautifulSoup

html = '''
<div class="quote"><span class="text">Imajinasi lebih penting dari pengetahuan.</span><small
class="author">Einstein</small></div>
<div class="quote"><span class="text">Tanpa penulis.</span></div>
<div class="quote"><span class="text">Jadilah dirimu sendiri.</span><small
class="author">Wilde</small></div>
'''

def ambil_kutipan(html):
    soup = BeautifulSoup(html, "html.parser")
    hasil = []
    for q in soup.find_all("div", class_="quote"):
        teks = q.find("span", class_="text")
        penulis = q.find("small", class_="author")
        if teks and penulis:
            hasil.append({"penulis": penulis.get_text(), "teks": teks.get_text()})
    return hasil
```

HASIL PEMERIKSA

✅ L12 benar!

Latihan L13 — Mengurai feed arXiv

`xml` adalah feed Atom contoh (data **fiktif**). Buat `baca_feed(xml)` → DataFrame kolom `judul`, `penulis` (gabungan nama dipisah `", "`), `tahun` (int dari `published`). Buat juga `kata_sering(df, n)` → list `(kata, jumlah)` n kata judul teratas (huruf kecil, ≥ 3 huruf, tanpa stopword `{"the","for","and","of","in","with"}`).

SOLUSI

```
import feedparser
from collections import Counter
import re

xml = '''<?xml version="1.0"?>
<feed xmlns="http://www.w3.org/2005/Atom">
 <entry><title>Deep Learning for Education</title><published>2024-03-01T00:00:00Z</published>
   <author><name>A. Contoh</name></author><author><name>B. Sampel</name></author></entry>
 <entry><title>Learning Analytics with Deep Models</title><published>2025-06-
10T00:00:00Z</published>
   <author><name>C. Fiktif</name></author></entry>
 <entry><title>Deep Learning in Practice</title><published>2025-09-02T00:00:00Z</published>
   <author><name>A. Contoh</name></author></entry>
</feed>'''

def baca_feed(xml):
    feed = feedparser.parse(xml)
    baris = []
    for e in feed.entries:
        baris.append({
            "judul": " ".join(e.title.split()),
            "penulis": ", ".join(a.name for a in e.authors),
            "tahun": int(e.published[:4]),
        })
    return pd.DataFrame(baris)

def kata_sering(df, n):
    stop = {"the", "for", "and", "of", "in", "with"}
    hitung = Counter()
    for judul in df["judul"]:
        for w in re.findall(r"[a-zA-Z]{3,}", judul.lower()):
            if w not in stop:
                hitung[w] += 1
    return hitung.most_common(n)
```

HASIL PEMERIKSA

✅ L13 benar!

Latihan P1 — Project 1 — Dashboard Visualisasi Nilai (Dasar)

Buat `dashboard_nilai(df)` untuk DataFrame `nama, prodi, nilai` yang mengembalikan figure 2×2: (0,0) histogram `nilai`, (0,1) bar rata-rata `nilai` per `prodi`, (1,0) pie proporsi jumlah mahasiswa per prodi, (1,1) boxplot `nilai` per `prodi`. Setiap panel punya judul tidak kosong; figure punya `suptitle`.

SOLUSI

```
def dashboard_nilai(df):
    fig, axes = plt.subplots(2, 2, figsize=(10, 8))
    axes[0, 0].hist(df["nilai"], bins=8, color="steelblue", edgecolor="white")
    axes[0, 0].set_title("Distribusi Nilai")
    rata = df.groupby("prodi")["nilai"].mean()
    axes[0, 1].bar(rata.index, rata.values, color="coral")
    axes[0, 1].set_title("Rata-rata Nilai per Prodi")
    jumlah = df["prodi"].value_counts()
    axes[1, 0].pie(jumlah.values, labels=jumlah.index, autopct="%1.0f%%")
    axes[1, 0].set_title("Proporsi Mahasiswa per Prodi")
    sns.boxplot(data=df, x="prodi", y="nilai", ax=axes[1, 1])
    axes[1, 1].set_title("Sebaran Nilai per Prodi")
    fig.suptitle("Dashboard Nilai Mahasiswa")
    fig.tight_layout()
    return fig
```

HASIL PEMERIKSA

```
✅ P1 benar!
```

Latihan P2 — Project 2 — API + Visualisasi (Menengah)

Data `posts` (list of dict, meniru JSONPlaceholder; **contoh**). Buat `jumlah_post_per_user(posts)` → Series (indeks `userId`, urut menaik) jumlah post per user, dan `plot_jumlah_post(seri)` → `ax` bar chart dengan judul `"Jumlah Post per User"` dan label sumbu.

SOLUSI

```
posts = [{"userId": u, "id": i, "title": f"judul {i}"} for i, u in enumerate([1, 1, 1, 2, 2, 3, 3, 3, 3, 4], 1)]

def jumlah_post_per_user(posts):
    return pd.DataFrame(posts).groupby("userId").size().sort_index()

def plot_jumlah_post(seri):
    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(seri.index.astype(str), seri.values, color="seagreen")
    ax.set_title("Jumlah Post per User")
    ax.set_xlabel("userId")
    ax.set_ylabel("Jumlah post")
    return ax
```

HASIL PEMERIKSA

```
✅ P2 benar!
```

Latihan P3 — Project 3 — Pipeline Paper arXiv (Menengah–Tinggi)

Gunakan `baca_feed` dan `kata_sering` dari L13. Buat `analisis_arxiv(xml)` yang mengembalikan dict: `per_tahun` (Series jumlah paper per tahun, urut tahun), `kata` (`kata_sering(df, 3)`), dan `penulis_teratas` (nama penulis dengan paper terbanyak, string; pecah kolom `penulis` pada `", "`). Gunakan feed `xml` dari L13.

SOLUSI

```
def analisis_arxiv(xml):
    df = baca_feed(xml)
    per_tahun = df.groupby("tahun").size().sort_index()
    semua = Counter(a for s in df["penulis"] for a in s.split(", "))
    return {"per_tahun": per_tahun, "kata": kata_sering(df, 3), "penulis_teratas":
semua.most_common(1)[0][0]}
```

HASIL PEMERIKSA

✅ P3 benar!

Latihan P4 — Project 4 — EDA Lengkap + Visual (Tinggi)

Buat `figur_eda(df)` untuk `tips_mini` (atau data serupa) yang mengembalikan figure 2×2: (0,0) histogram `total_bill`, (0,1) bar rata-rata `tip` per `day`, (1,0) scatter `total_bill` vs `tip` (warna per `sex`), (1,1) heatmap korelasi numerik. Semua panel berjudul; figure punya `suptitle`. Tambahkan juga `insight(df)` → list **minimal 3 string** kalimat insight yang memuat angka dari data (mis. rata-rata tip, hari dengan tip terbesar).

SOLUSI

```
def figur_eda(df):
    fig, axes = plt.subplots(2, 2, figsize=(11, 8))
    axes[0, 0].hist(df["total_bill"], bins=15, color="steelblue", edgecolor="white")
    axes[0, 0].set_title("Distribusi Total Bill")
    rata = df.groupby("day")["tip"].mean().reindex(["Thur", "Fri", "Sat", "Sun"])
    axes[0, 1].bar(rata.index, rata.values, color="coral")
    axes[0, 1].set_title("Rata-rata Tip per Hari")
    sns.scatterplot(data=df, x="total_bill", y="tip", hue="sex", ax=axes[1, 0])
    axes[1, 0].set_title("Tip vs Total Bill")
    sns.heatmap(df.select_dtypes(include="number").corr(), annot=True, fmt=".2f", cmap="coolwarm",
center=0, ax=axes[1, 1])
    axes[1, 1].set_title("Korelasi")
    fig.suptitle("EDA Tips (data contoh)")
    fig.tight_layout()
    return fig

def insight(df):
    rata_tip = df["tip"].mean()
    hari = df.groupby("day")["tip"].mean().idxmax()
    korelasi = df["total_bill"].corr(df["tip"])
    return [
        f"Rata-rata tip adalah {rata_tip:.2f}.",
        f"Hari dengan rata-rata tip tertinggi: {hari}.",
        f"Korelasi total_bill dan tip = {korelasi:.2f}.",
    ]
```

HASIL PEMERIKSA

✅ P4 benar!

6

Hari 6 — Capstone Project

Perencanaan, pipeline, README, dan pemeriksaan deliverable

A. Memilih Jalur dan Topik (Lembar Perencanaan)

B. Pertanyaan Analisis dan Struktur Folder

A. Memilih Jalur dan Topik (Lembar Perencanaan)

Pilih **satu** jalur. Isi tabel di sel Markdown berikutnya (klik dua kali untuk mengedit).

B. Pertanyaan Analisis dan Struktur Folder

Pertanyaan analisis ditulis lebih dulu — bukan setelah plot jadi. Pertanyaan yang baik dapat dijawab dengan data, spesifik, dan diakhiri tanda tanya.

Latihan L1 — Menulis pertanyaan analisis

Isi list `pertanyaan` dengan **2–4 pertanyaan** untuk topik capstone Anda (atau contoh: data nilai mahasiswa). Setiap pertanyaan berupa string yang diakhiri `?` dan minimal 20 karakter.

SOLUSI

```
pertanyaan = [
    "Apakah jam belajar berhubungan dengan nilai rata-rata mahasiswa?",
    "Program studi mana yang memiliki rata-rata nilai tertinggi?",
    "Apakah ada perbedaan kelulusan antara mahasiswa laki-laki dan perempuan?",
]
```

HASIL PEMERIKSA

```
✅ L1 benar!
```

Latihan L2 — Membuat struktur folder proyek

Buat `buat_struktur(root)` yang membuat folder `data/raw`, `data/processed`, `src`, `figures`, `output` di bawah `root` (`pathlib`, `mkdir(parents=True, exist_ok=True)`) dan file kosong `requirements.txt` berisi baris `pandas`, `matplotlib`, `seaborn`. Kembalikan `Path(root)`.

PENDEKATAN (PSEUDOCODE)

```
UNTUK setiap sub di [data/raw, data/processed, src, figures, output]:
    (root / sub).mkdir(parents=True, exist_ok=True)
tulis requirements.txt; RETURN root
```

SOLUSI

```
def buat_struktur(root):
    root = Path(root)
    for sub in ("data/raw", "data/processed", "src", "figures", "output"):
        (root / sub).mkdir(parents=True, exist_ok=True)
    (root / "requirements.txt").write_text("pandas\nmatplotlib\nseaborn\n", encoding="utf-8")
    return root
```

HASIL PEMERIKSA

```
✅ L2 benar!
```

Latihan L3 — clean(df)

Buat `clean(df)` yang mengembalikan **salinan** bersih: (1) `strip` spasi pada `nama`; (2) isi `nilai_uts` kosong dengan **median** kolom itu; (3) buang baris duplikat (lakukan **setelah** langkah 1–2); (4) tambah `rata_nilai` = rata-rata `nilai_uts` dan `nilai_uas`; (5) tambah `lulus` (`True` bila `rata_nilai >= 70`); (6) reset index.

PENDEKATAN (PSEUDOCODE)

```
d ← salinan df
nama → strip ; nilai_uts kosong → median
drop_duplicates() (SETELAH strip, agar ' Mhs07 ' == 'Mhs07')
rata_nilai ← (uts + uas) / 2 ; lulus ← rata_nilai >= 70 ; reset_index(drop=True)
```

SOLUSI

```python
def clean(df):
    d = df.copy()
    d["nama"] = d["nama"].str.strip()
    d["nilai_uts"] = d["nilai_uts"].fillna(d["nilai_uts"].median())
    d = d.drop_duplicates()
    d["rata_nilai"] = (d["nilai_uts"] + d["nilai_uas"]) / 2
    d["lulus"] = d["rata_nilai"] >= 70
    return d.reset_index(drop=True)
```

HASIL PEMERIKSA

```
✅ L3 benar!
```

Latihan L4 — analyze(df)

Buat `analyze(df)` (df hasil `clean`) yang mengembalikan dict berisi: `per_prodi` (Series rata-rata `rata_nilai` per `prodi`), `per_gender` (Series rata-rata `rata_nilai` per `gender`), `lulus_per_prodi` (Series **proporsi** lulus per prodi), dan `korelasi_jam_nilai` (korelasi `jam_belajar` dengan `rata_nilai`, float).

PENDEKATAN (PSEUDOCODE)

```
per_prodi ← groupby("prodi")["rata_nilai"].mean()
lulus_per_prodi ← groupby("prodi")["lulus"].mean()  (rata-rata boolean = proporsi)
korelasi ← df["jam_belajar"].corr(df["rata_nilai"])
```

SOLUSI

```python
def analyze(df):
    hasil = {}
    hasil["per_prodi"] = df.groupby("prodi")["rata_nilai"].mean()
    hasil["per_gender"] = df.groupby("gender")["rata_nilai"].mean()
    hasil["lulus_per_prodi"] = df.groupby("prodi")["lulus"].mean()
    hasil["korelasi_jam_nilai"] = float(df["jam_belajar"].corr(df["rata_nilai"]))
    return hasil
```

HASIL PEMERIKSA

```
✅ L4 benar!
```

Latihan L5 — visualize(df, hasil, fig_dir)

Buat `visualize(df, hasil, fig_dir)` yang menyimpan **minimal 3** grafik PNG (judul + label sumbu pada tiap grafik) ke folder `fig_dir`: `01_hist_nilai.png` (histogram `rata_nilai`), `02_prodi.png` (bar `hasil["per_prodi"]`), dan `03_jam_vs_nilai.png` (scatter `jam_belajar` vs `rata_nilai`). Buat folder bila belum ada, tutup figure setelah disimpan (`plt.close`), dan kembalikan list nama file yang dibuat.

PENDEKATAN (PSEUDOCODE)

```
fig_dir.mkdir(...)
UNTUK setiap grafik: fig, ax ← subplots; gambar; set_title/xlabel/ylabel; savefig(fig_dir / nama);
plt.close(fig)
```

SOLUSI

```python
def visualize(df, hasil, fig_dir):
    fig_dir = Path(fig_dir)
    fig_dir.mkdir(parents=True, exist_ok=True)
    dibuat = []

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.hist(df["rata_nilai"], bins=10, color="steelblue", edgecolor="white")
    ax.set_title("Distribusi Rata-rata Nilai"); ax.set_xlabel("Rata-rata nilai");
ax.set_ylabel("Jumlah mahasiswa")
    fig.tight_layout(); fig.savefig(fig_dir / "01_hist_nilai.png", dpi=120); plt.close(fig)
    dibuat.append("01_hist_nilai.png")

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.bar(hasil["per_prodi"].index, hasil["per_prodi"].values, color="coral")
    ax.set_title("Rata-rata Nilai per Prodi"); ax.set_xlabel("Program studi");
ax.set_ylabel("Rata-rata nilai")
    fig.tight_layout(); fig.savefig(fig_dir / "02_prodi.png", dpi=120); plt.close(fig)
    dibuat.append("02_prodi.png")

    fig, ax = plt.subplots(figsize=(6, 4))
    ax.scatter(df["jam_belajar"], df["rata_nilai"], alpha=0.7)
    ax.set_title("Jam Belajar vs Rata-rata Nilai"); ax.set_xlabel("Jam belajar per minggu");
ax.set_ylabel("Rata-rata nilai")
    fig.tight_layout(); fig.savefig(fig_dir / "03_jam_vs_nilai.png", dpi=120); plt.close(fig)
    dibuat.append("03_jam_vs_nilai.png")
    return dibuat
```

HASIL PEMERIKSA

```
✅ L5 benar!
```

Latihan L6 — tulis_insight(hasil, path)

Buat `tulis_insight(hasil, path)` yang menulis file teks berisi **minimal 5 butir bernomor** (`1. ...`, `2. ...`) — setiap butir memuat **angka** dari `hasil` — diikuti baris `Batasan:` dan **minimal 2 butir batasan** (diawali `- `). Buat folder induk `path` bila belum ada.

PENDEKATAN (PSEUDOCODE)

```
ambil angka dari hasil (idxmax(), max(), korelasi, ...) dan masukkan ke f-string
tulis: 5+ butir bernomor, baris kosong, "Batasan:", 2+ butir "- ..."
```

SOLUSI

```
def tulis_insight(hasil, path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    pp, pg, lp = hasil["per_prodi"], hasil["per_gender"], hasil["lulus_per_prodi"]
    baris = [
        f"1. Prodi dengan rata-rata nilai tertinggi: {pp.idxmax()} ({pp.max():.1f}).",
        f"2. Prodi dengan rata-rata nilai terendah: {pp.idxmin()} ({pp.min():.1f}).",
        f"3. Korelasi jam belajar dan nilai = {hasil['korelasi_jam_nilai']:.2f} (positif kuat).",
        f"4. Selisih rata-rata nilai antar gender hanya {abs(pg.max() - pg.min()):.1f} poin.",
        f"5. Proporsi lulus tertinggi di {lp.idxmax()} ({lp.max():.0%}) dan terendah di
{lp.idxmin()} ({lp.min():.0%}).",
        "",
        "Batasan:",
        "- Data latihan bersifat fiktif dan berukuran kecil (38 mahasiswa).",
        "- Korelasi tidak membuktikan sebab-akibat; faktor lain tidak diamati.",
    ]
    path.write_text("\n".join(baris) + "\n", encoding="utf-8")
```

HASIL PEMERIKSA

> ✅ L6 benar!

Latihan L7 — README dan main() — menyatukan pipeline

Buat `tulis_readme(root, judul, pertanyaan)` yang menulis `README.md` dengan judul `# <judul>` dan **10 bagian bernomor** `## 1. ...` sampai `## 10. ...` (bagian 2 memuat daftar `pertanyaan`). Lalu buat `main(root)` yang: `buat_struktur(root)` → `clean(data_latihan)` → simpan ke `data/processed/data_bersih.csv` → `analyze` → `visualize` ke `figures/` → `tulis_insight` ke `output/insight.txt` → `tulis_readme`. Kembalikan hasil `cek_deliverable(root)` (harus `[]`).

PENDEKATAN (PSEUDOCODE)

```
tulis_readme: judul + 10 bagian "## n. nama"; bagian 2 → daftar pertanyaan
main: buat_struktur → clean → to_csv → analyze → visualize → tulis_insight → tulis_readme → RETURN
cek_deliverable(root)
```

SOLUSI

```
def tulis_readme(root, judul, pertanyaan):
    bagian = ["Latar belakang & motivasi", "Pertanyaan analisis", "Data (sumber, ukuran, cara
dapat, etika)",
              "Metode (cleaning, analisis, tools)", "Hasil utama (temuan + rujukan figure)",
              "Visualisasi (daftar figure + arti singkat)", "Batasan", "Kesimpulan & kerja
lanjutan",
              "Cara menjalankan", "Struktur folder"]
    baris = [f"# {judul}", ""]
    for i, nama in enumerate(bagian, 1):
        baris += [f"## {i}. {nama}", ""]
        if i == 2:
            baris += [f"{n}. {q}" for n, q in enumerate(pertanyaan, 1)] + [""]
        else:
            baris += ["(isi dengan kata Anda sendiri)", ""]
    (Path(root) / "README.md").write_text("\n".join(baris), encoding="utf-8")

def main(root):
    root = buat_struktur(root)
    df = clean(data_latihan)
    df.to_csv(root / "data" / "processed" / "data_bersih.csv", index=False)
    hasil = analyze(df)
    visualize(df, hasil, root / "figures")
    tulis_insight(hasil, root / "output" / "insight.txt")
    tulis_readme(root, "Capstone Contoh — Nilai Mahasiswa", pertanyaan)
    return cek_deliverable(root)
```

HASIL PEMERIKSA

> ✅ L7 benar!

Latihan L8 — Jalur B — ambil data dengan cache dan fallback

Buat `ambil_dengan_cache(url, cache_path, timeout=5)`: GET `url` → simpan JSON mentah ke `cache_path` (agar tidak perlu mengulang) → kembalikan `(data, "api")`. Jika request gagal (`RequestException` / `ValueError`) dan cache **ada**, kembalikan `(data_dari_cache, "cache")`; jika cache juga tidak ada kembalikan `(None, "gagal")`.

PENDEKATAN (PSEUDOCODE)

```
COBA: GET → raise_for_status → data ← json ; simpan ke cache ; RETURN (data, "api")
TANGKAP RequestException/ValueError:
    JIKA cache ada → RETURN (isi cache, "cache")
    SELAIN ITU → RETURN (None, "gagal")
```

SOLUSI

```python
import json
import requests

def ambil_dengan_cache(url, cache_path, timeout=5):
    cache_path = Path(cache_path)
    try:
        resp = requests.get(url, timeout=timeout)
        resp.raise_for_status()
        data = resp.json()
        cache_path.parent.mkdir(parents=True, exist_ok=True)
        cache_path.write_text(json.dumps(data), encoding="utf-8")
        return data, "api"
    except (requests.exceptions.RequestException, ValueError):
        if cache_path.exists():
            return json.loads(cache_path.read_text(encoding="utf-8")), "cache"
        return None, "gagal"
```

HASIL PEMERIKSA

```
✅ L8 benar!
```

Latihan L9 — Jalur C — parameter paginasi arXiv

Buat `params_arxiv(query, halaman, per_halaman=10)` yang mengembalikan dict parameter API arXiv: `search_query`, `start` (indeks mulai: `(halaman-1) × per_halaman`; halaman dimulai dari 1), `max_results`, `sortBy="submittedDate"`, `sortOrder="descending"`. Lemparkan `ValueError` bila `halaman < 1`. Lalu `total_halaman(n_paper, per_halaman=10)` → jumlah halaman yang dibutuhkan (pembulatan ke atas).

PENDEKATAN (PSEUDOCODE)

```
start ← (halaman - 1) × per_halaman
total_halaman ← pembagian dibulatkan ke atas (-(-n // per))
```

SOLUSI

```python
def params_arxiv(query, halaman, per_halaman=10):
    if halaman < 1:
        raise ValueError("halaman dimulai dari 1")
    return {
        "search_query": query,
        "start": (halaman - 1) * per_halaman,
        "max_results": per_halaman,
        "sortBy": "submittedDate",
        "sortOrder": "descending",
    }

def total_halaman(n_paper, per_halaman=10):
    return -(-n_paper // per_halaman)
```

HASIL PEMERIKSA

```
✅ L9 benar!
```



Lampiran — Contoh Capstone Lengkap

Solusi utuh untuk Modul 6: empat jalur capstone beserta dokumentasi, grafik hasil, dan kode sumbernya.

Jalur A — EDA Dataset Publik (Titanic)

Jalur B — API PyPI & npm → Analisis → Visualisasi

Jalur C — Pipeline Paper arXiv

Jalur D — Aplikasi Analisis Nilai Berbasis OOP

Jalur A — EDA Dataset Publik (Titanic)

> Contoh pengerjaan lengkap untuk **Modul 6 (versi 2), Jalur A**. Data nyata (seaborn-data); semua angka di bawah dihitung oleh `src/main.py`.

1. Latar belakang & motivasi

Tenggelamnya Titanic (1912) adalah dataset klasik untuk latihan EDA: ada missing value, variabel kategorik dan numerik, serta pertanyaan yang mudah dipahami orang awam — siapa yang lebih mungkin selamat, dan mengapa pola itu muncul.

2. Pertanyaan analisis

1. Faktor apa (jenis kelamin, kelas tiket, usia, ukuran keluarga, harga tiket) yang paling berkaitan dengan keselamatan?
2. Apakah kelas tiket dan jenis kelamin saling memperkuat atau berdiri sendiri?
3. Apakah anak-anak dan keluarga kecil punya peluang selamat yang berbeda dari penumpang lain?

3. Data (sumber, ukuran, cara dapat, etika)

- **Sumber:** `titanic.csv` dari repositori seaborn-data; salinan lokal di `data/raw/titanic.csv` (diunduh otomatis jika belum ada).
- **Ukuran:** 891 baris × 15 kolom. Kosong: `age` 177 (19,9%), `deck` 688 (77,2%), `embarked` / `embark_town` masing-masing 2; 107 baris identik penuh.
- **Etika:** data historis publik tanpa informasi pribadi sensitif; dipakai untuk edukasi.

4. Metode (cleaning, analisis, tools)

- **Cleaning** (terdokumentasi pada output `run_log.txt` dan `data/processed/data_bersih.csv`): `age` diisi median per (kelas, jenis kelamin) + penanda `age_imputed`; `embarked` diisi modus; `deck` kosong → kategori "Unknown"; kolom redundan `alive` dan `class` dihapus. **107 baris identik tidak dihapus** karena tidak ada ID penumpang.
- **Fitur baru:** `family_size`, `age_group`, `fare_quartile`, `status`.
- **Analisis:** 6 agregasi `groupby` (jenis kelamin, kelas, kelompok usia, ukuran keluarga, pelabuhan, fare per kelas) + `pivot_table` jenis kelamin × kelas + matriks korelasi + uji chi-square dan Mann-Whitney.
- **Tools:** pandas, numpy, matplotlib, seaborn, scipy, openpyxl.

5. Hasil utama (temuan + rujukan figure)

1. Jenis kelamin adalah pembeda terkuat. 74.2% penumpang perempuan selamat, dibanding 18.9% laki-laki (selisih 55 poin persentase; chi-square p = 1.2e-58).
2. Kelas tiket memengaruhi peluang selamat. Kelas 1: 63.0%, kelas 2: 47.3%, kelas 3: 24.2% (chi-square p = 4.5e-23). Korelasi pclass dengan survived = -0.34.
3. Efek kelas dan jenis kelamin saling menguatkan. Perempuan kelas 1 selamat 96.8%, sedangkan laki-laki kelas 3 hanya 13.5%. Namun perempuan kelas 3 (50.0%) masih lebih baik daripada laki-laki kelas 1 (36.9%).

4. Anak-anak diprioritaskan. Kelompok usia 0-12 tahun selamat 58.0% (n=69), lebih tinggi daripada dewasa muda 19-35 tahun (35.8%) dan lansia 60+ (22.7%).
5. Ukuran keluarga berhubungan non-linier dengan keselamatan. Penumpang sendirian selamat 30.4%, keluarga 2-4 orang 57.9%, dan keluarga >= 5 orang hanya 16.1% (n=62).
6. Harga tiket berkorelasi positif dengan keselamatan. Median fare kelas 1 = 60.3 vs kelas 3 = 8.1; penumpang yang selamat membayar lebih mahal (Mann-Whitney p = 4.6e-22; korelasi fare-survived = 0.26).

Rujukan figure: butir 1–3 → `figures/01_keselamatan_kelas_sex.png`; butir 4 → `figures/02_distribusi_usia.png` dan `06_usia_dan_sex.png`; butir 5 → `figures/05_ukuran_keluarga.png`; butir 6 → `figures/03_harga_tiket_per_kelas.png` dan `04_korelasi.png`.

Ringkasan tabel: `output/ringkasan.csv`, `output/pivot_sex_kelas.csv`, laporan Excel multi-sheet `output/laporan_titanic.xlsx`, dan teks lengkap di `output/insight.txt`.

6. Visualisasi (daftar figure + arti singkat)

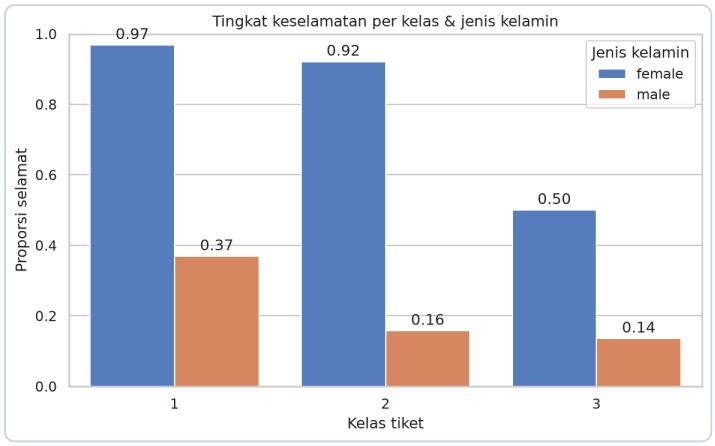
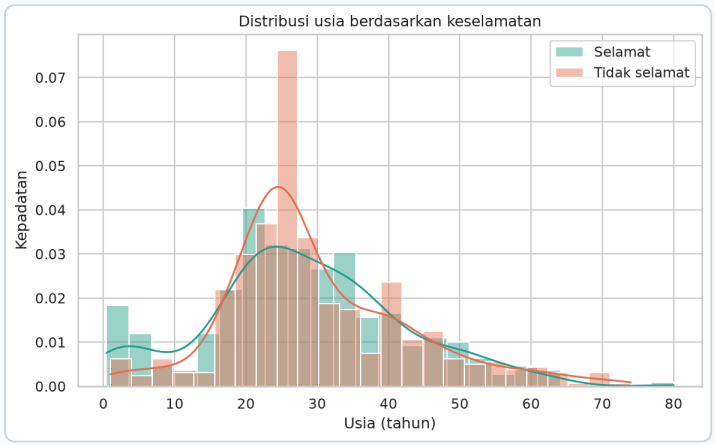
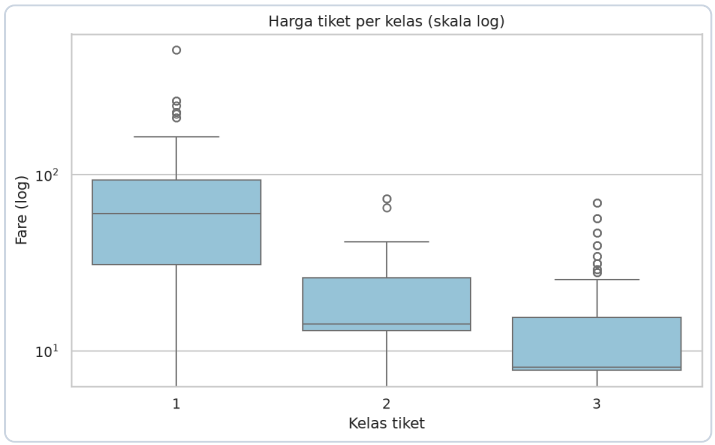
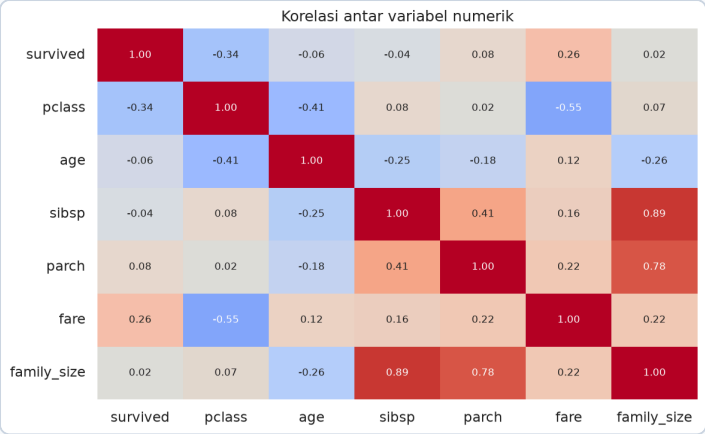
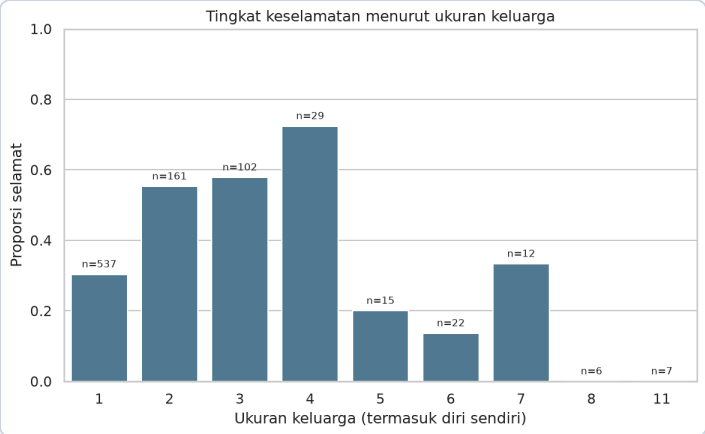
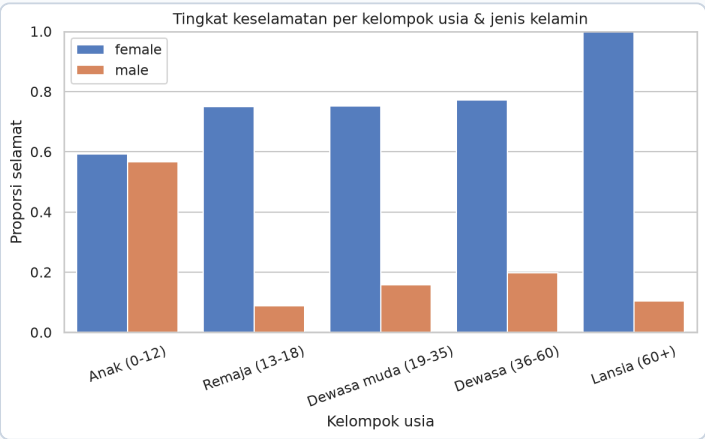
Figure	Arti singkat
	Perempuan selamat jauh lebih sering di semua kelas; selisih terbesar pada kelas 2.
	Anak kecil lebih banyak yang selamat; puncak usia 20–30 tahun didominasi yang tidak selamat.
	Harga tiket turun tajam dari kelas 1 ke 3 (skala log), dengan banyak outlier.

Figure	Arti singkat
	`pclass` (-0,34) dan `fare` (+0,26) adalah korelasi numerik terkuat dengan `survived` .
	Keluarga 2–4 orang lebih sering selamat; penumpang sendirian dan keluarga besar lebih rendah.
	Perempuan > laki-laki di semua kelompok usia, kecuali anak-anak (hampir setara).

Dashboard 2×3: `figures/00_dashboard.png` (juga PDF).

7. Batasan

- Data observasional dari satu peristiwa: temuan menunjukkan asosiasi, bukan sebab-akibat.
- 177 nilai usia (19,9%) diimputasi dengan median per kelas & jenis kelamin sehingga sebaran usia sedikit lebih sempit dari aslinya.
- Kolom deck 77% kosong sehingga tidak dipakai sebagai faktor analisis.

- 107 baris identik dipertahankan karena tidak ada ID penumpang; jika sebenarnya duplikat, proporsi bisa bergeser tipis.
- Kelompok kecil (lansia n=22, keluarga >= 5 orang n=62) membuat persentasenya tidak stabil.
- Harga tiket (fare) berkorelasi kuat dengan kelas tiket, sehingga efeknya tidak independen.

8. Kesimpulan & kerja lanjutan

Jenis kelamin dan kelas tiket adalah dua faktor terkuat dan saling memperkuat; anak-anak juga diprioritaskan. Ukuran keluarga dan harga tiket memberi gambaran tambahan, tetapi harga tiket sendiri berkorelasi kuat dengan kelas (pclass–fare = -0,55) sehingga tidak sepenuhnya independen. Semua temuan adalah asosiasi, bukan sebab-akibat.

Kerja lanjutan: model sederhana (logistic regression) untuk memperkirakan peluang selamat; imputasi usia yang lebih baik; analisis interaksi usia × kelas.

9. Cara menjalankan

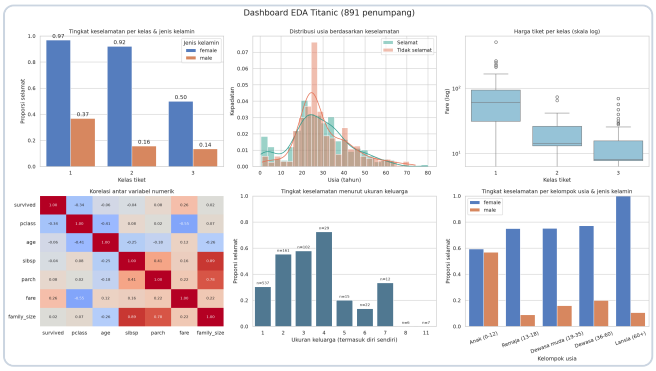
```
pip install -r requirements.txt
python src/main.py
```

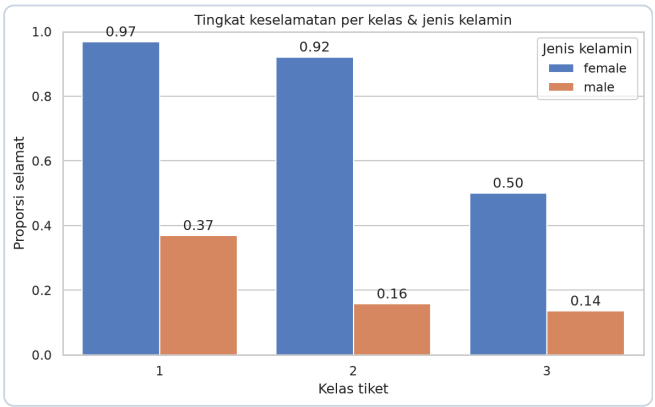
Log eksekusi: `output/run_log.txt`. Program dapat dijalankan dari folder mana pun.

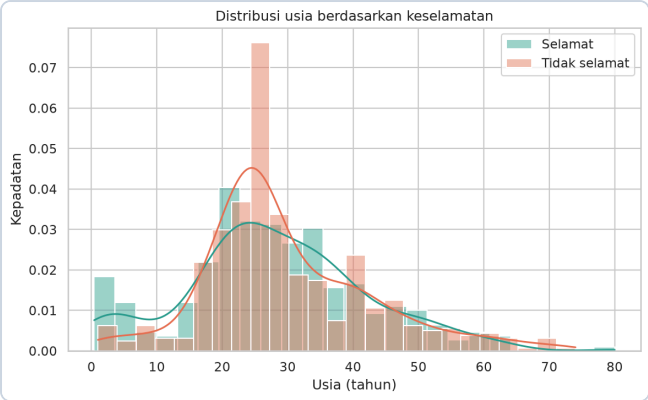
10. Struktur folder

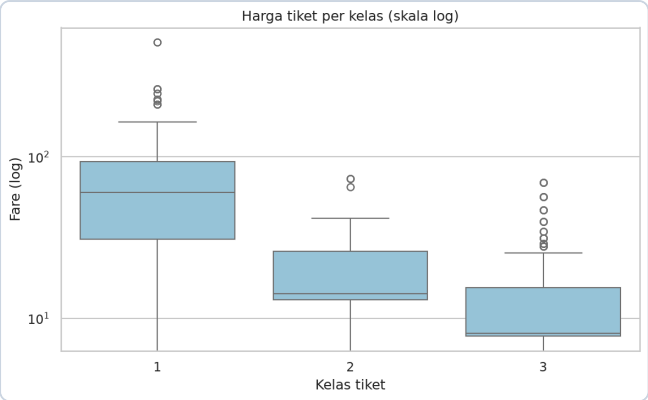
```
jalur_a_eda_titanic/
├── README.md
├── requirements.txt
├── data/
│   ├── raw/titanic.csv
│   └── processed/data_bersih.csv
├── src/main.py
├── figures/                # 6 PNG + dashboard (PNG/PDF)
└── output/
    ├── ringkasan.csv  pivot_sex_kelas.csv  laporan_titanic.xlsx
    ├── insight.txt
    └── run_log.txt
```

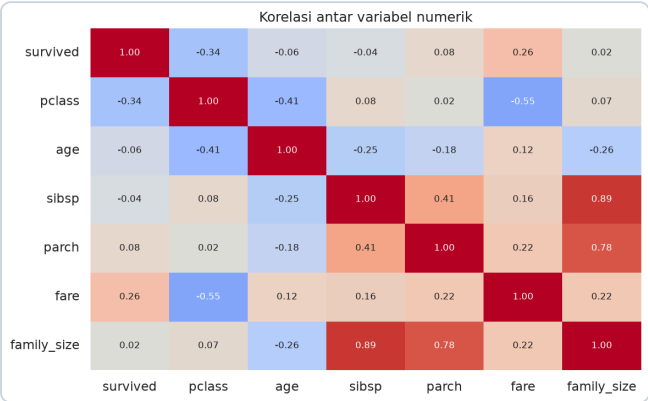
Grafik hasil

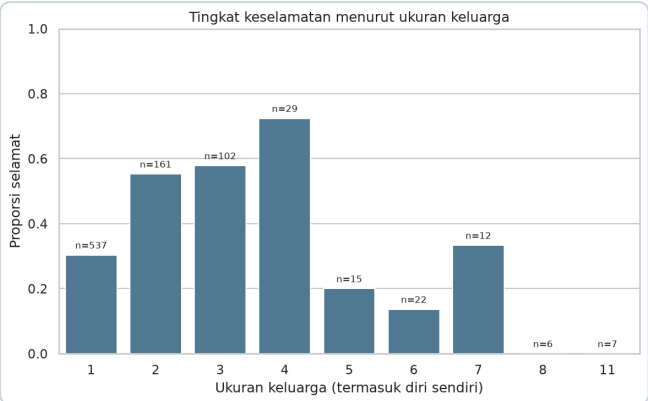


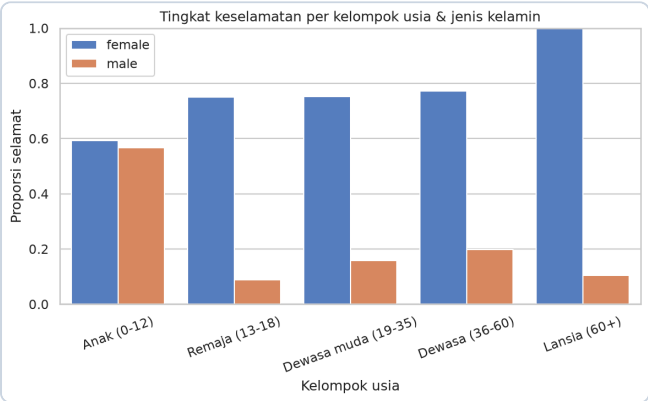












Kode sumber

`jalur_a_eda_titanic/src/main.py`

```
"""Capstone Modul 6 (v2) — Jalur A: EDA dataset publik Titanic.

Alur: load + inspeksi -> cleaning -> feature engineering -> analisis
(groupby / pivot / korelasi / uji statistik) -> visualisasi -> insight -> export.

Jalankan dari folder mana pun:
    python src/main.py
"""
from pathlib import Path

import matplotlib

matplotlib.use("Agg")  # aman dijalankan tanpa layar (server/CI)
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats
```

```
BASE_DIR = Path(__file__).resolve().parent.parent
RAW_CSV = BASE_DIR / "data" / "raw" / "titanic.csv"
SUMBER_URL = (
    "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv"
)
CLEAN_CSV = BASE_DIR / "data" / "processed" / "data_bersih.csv"
FIG_DIR = BASE_DIR / "figures"
REPORT_DIR = BASE_DIR / "output"

PERTANYAAN = [
    "1. Faktor apa (jenis kelamin, kelas tiket, usia, ukuran keluarga, harga tiket) yang paling berkaitan dengan
keselamatan?",
    "2. Apakah kelas tiket dan jenis kelamin saling memperkuat atau berdiri sendiri?",
    "3. Apakah anak-anak dan keluarga kecil punya peluang selamat yang berbeda dari penumpang lain?",
]
BATASAN = [
    "Data observasional dari satu peristiwa: temuan menunjukkan asosiasi, bukan sebab-akibat.",
    "177 nilai usia (19,9%) diimputasi dengan median per kelas & jenis kelamin sehingga sebaran usia sedikit lebih
sempit dari aslinya.",
    "Kolom deck 77% kosong sehingga tidak dipakai sebagai faktor analisis.",
    "107 baris identik dipertahankan karena tidak ada ID penumpang; jika sebenarnya duplikat, proporsi bisa
bergeser tipis.",
    "Kelompok kecil (lansia n=22, keluarga >= 5 orang n=62) membuat persentasenya tidak stabil.",
    "Harga tiket (fare) berkorelasi kuat dengan kelas tiket, sehingga efeknya tidak independen.",
]

sns.set_theme(style="whitegrid", palette="muted")
WARNA = {"Selamat": "#2a9d8f", "Tidak selamat": "#e76f51"}


# ---------------------------------------------------------------------------
# 1. Load + inspeksi
# ---------------------------------------------------------------------------
def load_data(path=RAW_CSV, url=SUMBER_URL):
    """Baca CSV lokal; jika belum ada, unduh dari sumber lalu simpan."""
    try:
        return pd.read_csv(path)
    except FileNotFoundError:
        print(f"{path.name} belum ada, mengunduh dari sumber ...")
        df = pd.read_csv(url)
        path.parent.mkdir(parents=True, exist_ok=True)
        df.to_csv(path, index=False)
        return df


def inspeksi(df):
    """Cetak ringkasan awal: shape, dtypes, missing, duplikat."""
    print("=" * 60)
    print("1. INSPEKSI AWAL")
    print("=" * 60)
    print("Shape:", df.shape)
    print("\nTipe data:")
    print(df.dtypes.to_string())
    miss = df.isna().sum()
    miss = miss[miss > 0]
    print("\nMissing values (jumlah, persen):")
    for kolom, n in miss.items():
        print(f"  {kolom:<12} {n:>4}  ({n / len(df):.1%})")
    print("\nBaris duplikat penuh:", int(df.duplicated().sum()))
    print("\nStatistik numerik:")
    print(df.describe().round(2).to_string())


# ---------------------------------------------------------------------------
# 2. Cleaning + feature engineering
# ---------------------------------------------------------------------------
def cleaning(df):
    """Bersihkan data dan kembalikan (df_bersih, catatan_cleaning)."""
    df = df.copy()
    catatan = []

    n_dup = int(df.duplicated().sum())
    catatan.append(
        f"{n_dup} baris identik terdeteksi tetapi TIDAK dihapus: dataset tidak "
        "punya ID penumpang, sehingga baris identik bisa jadi orang berbeda "
```

```
        "(mis. anggota keluarga dengan tiket sama)."
    )

    # age: isi dengan median per kombinasi kelas & jenis kelamin
    n_age = int(df["age"].isna().sum())
    df["age_imputed"] = df["age"].isna()
    df["age"] = df["age"].fillna(
        df.groupby(["pclass", "sex"])["age"].transform("median")
    )
    catatan.append(
        f"{n_age} nilai age kosong diisi median per (pclass, sex); "
        "kolom age_imputed menandai baris tersebut."
    )

    # embarked / embark_town: hanya 2 kosong -> modus
    for kolom in ("embarked", "embark_town"):
        df[kolom] = df[kolom].fillna(df[kolom].mode()[0])
    catatan.append("2 nilai embarked/embark_town kosong diisi modus (Southampton/S).")

    # deck: terlalu banyak kosong -> kategori 'Unknown'
    n_deck = int(df["deck"].isna().sum())
    df["deck"] = df["deck"].fillna("Unknown")
    catatan.append(
        f"{n_deck} nilai deck kosong ({n_deck / len(df):.0%}) diberi kategori "
        "'Unknown' (tidak diimputasi karena terlalu banyak)."
    )

    # kolom redundan
    df = df.drop(columns=["alive", "class"])
    catatan.append("Kolom alive & class dihapus (duplikasi dari survived & pclass).")

    # tipe data
    for kolom in ("sex", "embarked", "embark_town", "who", "deck"):
        df[kolom] = df[kolom].astype("category")
    catatan.append("Kolom teks dikonversi ke tipe category.")

    # feature engineering
    df["family_size"] = df["sibsp"] + df["parch"] + 1
    df["age_group"] = pd.cut(
        df["age"],
        bins=[0, 12, 18, 35, 60, 100],
        labels=["Anak (0-12)", "Remaja (13-18)", "Dewasa muda (19-35)",
                "Dewasa (36-60)", "Lansia (60+)"],
    )
    df["fare_quartile"] = pd.qcut(
        df["fare"], 4, labels=["Q1 termurah", "Q2", "Q3", "Q4 termahal"]
    )
    df["status"] = np.where(df["survived"] == 1, "Selamat", "Tidak selamat")
    catatan.append(
        "Fitur baru: family_size, age_group, fare_quartile, status."
    )

    assert df[["age", "fare", "embarked"]].isna().sum().sum() == 0
    return df, catatan


# ---------------------------------------------------------------------------
# 3. Analisis
# ---------------------------------------------------------------------------
def analisis(df):
    """Jalankan groupby, pivot, korelasi, dan uji statistik; kembalikan dict hasil."""
    hasil = {}
    hasil["overall"] = df["survived"].mean()
    hasil["by_sex"] = df.groupby("sex", observed=True)["survived"].agg(["count", "mean"])
    hasil["by_class"] = df.groupby("pclass")["survived"].agg(["count", "mean"])
    hasil["pivot_sex_class"] = df.pivot_table(
        values="survived", index="sex", columns="pclass",
        aggfunc="mean", observed=True,
    )
    hasil["by_age_group"] = df.groupby("age_group", observed=True)["survived"].agg(["count", "mean"])
    hasil["by_family"] = df.groupby("family_size")["survived"].agg(["count", "mean"])
    hasil["by_embark"] = df.groupby("embark_town", observed=True)["survived"].agg(["count", "mean"])
    hasil["fare_by_class"] = df.groupby("pclass")["fare"].agg(["median", "mean", "max"])
    hasil["korelasi"] = df[["survived", "pclass", "age", "sibsp", "parch", "fare", "family_size"]].corr()
```

```
    # uji statistik sederhana (stretch goal)
    tabel_sex = pd.crosstab(df["sex"], df["survived"])
    chi2, p_sex, _, _ = stats.chi2_contingency(tabel_sex)
    tabel_cls = pd.crosstab(df["pclass"], df["survived"])
    chi2_c, p_cls, _, _ = stats.chi2_contingency(tabel_cls)
    fare_sel = df.loc[df["survived"] == 1, "fare"]
    fare_tdk = df.loc[df["survived"] == 0, "fare"]
    u, p_fare = stats.mannwhitneyu(fare_sel, fare_tdk, alternative="two-sided")
    hasil["uji"] = pd.DataFrame(
        {
            "uji": ["Chi-square sex x survived", "Chi-square pclass x survived",
                    "Mann-Whitney fare (selamat vs tidak)"],
            "statistik": [chi2, chi2_c, u],
            "p_value": [p_sex, p_cls, p_fare],
        }
    )
    return hasil


# ---------------------------------------------------------------------------
# 4. Visualisasi
# ---------------------------------------------------------------------------
def _simpan(fig, nama):
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    fig.savefig(FIG_DIR / f"{nama}.png", dpi=150, bbox_inches="tight")


def _plot_sex_class(ax, df):
    sns.barplot(data=df, x="pclass", y="survived", hue="sex", errorbar=None, ax=ax)
    ax.set_title("Tingkat keselamatan per kelas & jenis kelamin")
    ax.set_xlabel("Kelas tiket"); ax.set_ylabel("Proporsi selamat")
    ax.set_ylim(0, 1); ax.legend(title="Jenis kelamin")
    for c in ax.containers:
        ax.bar_label(c, fmt="%.2f", padding=2)


def _plot_age(ax, df):
    for status, warna in WARNA.items():
        sns.histplot(df.loc[df["status"] == status, "age"], bins=25, kde=True,
                     stat="density", color=warna, alpha=0.45, label=status, ax=ax)
    ax.set_title("Distribusi usia berdasarkan keselamatan")
    ax.set_xlabel("Usia (tahun)"); ax.set_ylabel("Kepadatan"); ax.legend()


def _plot_fare(ax, df):
    sns.boxplot(data=df, x="pclass", y="fare", ax=ax, color="#8ecae6")
    ax.set_yscale("log")
    ax.set_title("Harga tiket per kelas (skala log)")
    ax.set_xlabel("Kelas tiket"); ax.set_ylabel("Fare (log)")


def _plot_corr(ax, hasil):
    sns.heatmap(hasil["korelasi"], annot=True, fmt=".2f", cmap="coolwarm",
                center=0, ax=ax, cbar=False, annot_kws={"size": 8})
    ax.set_title("Korelasi antar variabel numerik")


def _plot_family(ax, hasil):
    d = hasil["by_family"].reset_index()
    sns.barplot(data=d, x="family_size", y="mean", ax=ax, color="#457b9d")
    ax.set_title("Tingkat keselamatan menurut ukuran keluarga")
    ax.set_xlabel("Ukuran keluarga (termasuk diri sendiri)")
    ax.set_ylabel("Proporsi selamat"); ax.set_ylim(0, 1)
    for i, (n, m) in enumerate(zip(d["count"], d["mean"])):
        ax.text(i, m + 0.02, f"n={n}", ha="center", fontsize=8)


def _plot_age_sex(ax, df):
    sns.barplot(data=df, x="age_group", y="survived", hue="sex", errorbar=None, ax=ax)
    ax.set_title("Tingkat keselamatan per kelompok usia & jenis kelamin")
    ax.set_xlabel("Kelompok usia"); ax.set_ylabel("Proporsi selamat")
    ax.set_ylim(0, 1); ax.tick_params(axis="x", rotation=20); ax.legend(title="")


def visualisasi(df, hasil):
```

```
    """Buat 6 grafik terpisah + 1 dashboard 2x3 (PNG & PDF)."""
    plot_df = [
        ("01_keselamatan_kelas_sex", _plot_sex_class, (df,)),
        ("02_distribusi_usia", _plot_age, (df,)),
        ("03_harga_tiket_per_kelas", _plot_fare, (df,)),
        ("04_korelasi", _plot_corr, (hasil,)),
        ("05_ukuran_keluarga", _plot_family, (hasil,)),
        ("06_usia_dan_sex", _plot_age_sex, (df,)),
    ]
    for nama, fungsi, args in plot_df:
        fig, ax = plt.subplots(figsize=(8, 5))
        fungsi(ax, *args)
        fig.tight_layout()
        _simpan(fig, nama)
        plt.close(fig)

    fig, axes = plt.subplots(2, 3, figsize=(20, 11))
    for ax, (_, fungsi, args) in zip(axes.ravel(), plot_df):
        fungsi(ax, *args)
    fig.suptitle("Dashboard EDA Titanic (891 penumpang)", fontsize=18, y=1.0)
    fig.tight_layout()
    _simpan(fig, "00_dashboard")
    fig.savefig(FIG_DIR / "00_dashboard.pdf", bbox_inches="tight")
    plt.close(fig)


# ---------------------------------------------------------------------------
# 5. Insight + export
# ---------------------------------------------------------------------------
def tulis_insight(df, hasil):
    """Susun insight berbasis angka hasil analisis (ditulis ke insights.md)."""
    s = hasil["by_sex"]["mean"]
    c = hasil["by_class"]["mean"]
    pv = hasil["pivot_sex_class"]
    ag = hasil["by_age_group"]
    fam = hasil["by_family"]
    uji = hasil["uji"].set_index("uji")["p_value"]
    kor = hasil["korelasi"]["survived"].drop("survived").sort_values()
    sendiri = fam.loc[1, "mean"]
    kecil = df.loc[df["family_size"].between(2, 4), "survived"].mean()
    besar = df.loc[df["family_size"] >= 5, "survived"].mean()
    n_besar = int((df["family_size"] >= 5).sum())

    baris = [
        f"1. **Jenis kelamin adalah pembeda terkuat.** {s['female']:.1%} penumpang "
        f"perempuan selamat, dibanding {s['male']:.1%} laki-laki "
        f"(selisih {(s['female'] - s['male']) * 100:.0f} poin persentase; "
        f"chi-square p = {uji['Chi-square sex x survived']:.1e}).",
        f"2. **Kelas tiket memengaruhi peluang selamat.** Kelas 1: {c[1]:.1%}, "
        f"kelas 2: {c[2]:.1%}, kelas 3: {c[3]:.1%} "
        f"(chi-square p = {uji['Chi-square pclass x survived']:.1e}). "
        f"Korelasi pclass dengan survived = {kor['pclass']:.2f}.",
        f"3. **Efek kelas dan jenis kelamin saling menguatkan.** Perempuan kelas 1 "
        f"selamat {pv.loc['female', 1]:.1%}, sedangkan laki-laki kelas 3 hanya "
        f"{pv.loc['male', 3]:.1%}. Namun perempuan kelas 3 ({pv.loc['female', 3]:.1%}) "
        f"masih lebih baik daripada laki-laki kelas 1 ({pv.loc['male', 1]:.1%}).",
        f"4. **Anak-anak diprioritaskan.** Kelompok usia 0-12 tahun selamat "
        f"{ag.loc['Anak (0-12)', 'mean']:.1%} (n={int(ag.loc['Anak (0-12)', 'count'])}), "
        f"lebih tinggi daripada dewasa muda 19-35 tahun "
        f"({ag.loc['Dewasa muda (19-35)', 'mean']:.1%}) dan lansia 60+ "
        f"({ag.loc['Lansia (60+)', 'mean']:.1%}).",
        f"5. **Ukuran keluarga berhubungan non-linier dengan keselamatan.** "
        f"Penumpang sendirian selamat {sendiri:.1%}, keluarga 2-4 orang {kecil:.1%}, "
        f"dan keluarga >= 5 orang hanya {besar:.1%} (n={n_besar}).",
        f"6. **Harga tiket berkorelasi positif dengan keselamatan.** Median fare "
        f"kelas 1 = {hasil['fare_by_class'].loc[1, 'median']:.1f} vs kelas 3 = "
        f"{hasil['fare_by_class'].loc[3, 'median']:.1f}; penumpang yang selamat "
        f"membayar lebih mahal (Mann-Whitney p = {uji['Mann-Whitney fare (selamat vs tidak)']:.1e}; "
        f"korelasi fare-survived = {kor['fare']:.2f}).",
    ]
    return "\n\n".join(baris) + "\n"


def export(df, hasil, catatan, insight):
    """Simpan data bersih, tabel ringkasan, Excel multi-sheet, dan teks insight."""
```

```
    CLEAN_CSV.parent.mkdir(parents=True, exist_ok=True)
    REPORT_DIR.mkdir(parents=True, exist_ok=True)
    df.to_csv(CLEAN_CSV, index=False)

    ringkasan = hasil["by_class"].join(
        hasil["fare_by_class"], rsuffix="_fare"
    ).rename(columns={"count": "jumlah", "mean": "proporsi_selamat"})
    ringkasan.round(4).to_csv(REPORT_DIR / "ringkasan.csv")
    hasil["pivot_sex_class"].round(4).to_csv(REPORT_DIR / "pivot_sex_kelas.csv")

    with pd.ExcelWriter(REPORT_DIR / "laporan_titanic.xlsx") as xl:
        ringkasan.round(4).to_excel(xl, sheet_name="Per kelas")
        hasil["by_sex"].round(4).to_excel(xl, sheet_name="Per sex")
        hasil["pivot_sex_class"].round(4).to_excel(xl, sheet_name="Pivot sex x kelas")
        hasil["by_age_group"].round(4).to_excel(xl, sheet_name="Per usia")
        hasil["by_family"].round(4).to_excel(xl, sheet_name="Per keluarga")
        hasil["korelasi"].round(3).to_excel(xl, sheet_name="Korelasi")
        hasil["uji"].to_excel(xl, sheet_name="Uji statistik", index=False)
        pd.DataFrame({"catatan_cleaning": catatan}).to_excel(
            xl, sheet_name="Catatan cleaning", index=False
        )
    teks = insight.replace("**", "") + "\nBatasan:\n" + "\n".join(f"- {b}" for b in BATASAN) + "\n"
    (REPORT_DIR / "insight.txt").write_text(teks, encoding="utf-8")


def main():
    print("PERTANYAAN ANALISIS")
    print("\n".join(PERTANYAAN))
    print()
    df_raw = load_data()
    inspeksi(df_raw)

    print("\n" + "=" * 60 + "\n2. CLEANING\n" + "=" * 60)
    df, catatan = cleaning(df_raw)
    for c in catatan:
        print("-", c)
    print("Shape setelah cleaning:", df.shape)

    print("\n" + "=" * 60 + "\n3. ANALISIS\n" + "=" * 60)
    hasil = analisis(df)
    print(f"Tingkat keselamatan keseluruhan: {hasil['overall']:.1%}")
    for kunci in ("by_sex", "by_class", "pivot_sex_class", "by_age_group",
                  "by_family", "by_embark", "fare_by_class", "uji"):
        print(f"\n[{kunci}]")
        if kunci == "uji":
            print(hasil[kunci].to_string(
                formatters={"statistik": "{:.2f}".format, "p_value": "{:.2e}".format}))
        else:
            print(hasil[kunci].round(4).to_string())

    visualisasi(df, hasil)
    insight = tulis_insight(df, hasil)
    export(df, hasil, catatan, insight)

    print("\n" + "=" * 60 + "\n4. INSIGHT\n" + "=" * 60)
    print(insight)
    print("Batasan:")
    print("\n".join(f"- {b}" for b in BATASAN))
    print("\nSelesai. File tersimpan di data/processed/, figures/, dan output/.")


if __name__ == "__main__":
    main()
```

Jalur B — API PyPI & npm → Analisis → Visualisasi

> Contoh pengerjaan lengkap untuk **Modul 6 (versi 2), Jalur B**. Data nyata diambil **langsung dari API publik** PyPI dan npm pada **2026-10-02**; angka dihitung oleh `src/main.py`.

1. Latar belakang & motivasi

Library populer dirilis dengan ritme yang sangat berbeda. Memahami ritme itu membantu memilih library dan merencanakan pembaruan dependensi. Proyek ini memakai API publik sebagai sumber data agar pipeline-nya mencerminkan pekerjaan nyata (ambil → simpan mentah → olah → visualisasi).

2. Pertanyaan analisis

1. Library mana yang paling aktif dirilis per tahun (bukan sekadar total sepanjang masa)?
2. Apakah ritme rilis library Python (PyPI) berbeda dari library JavaScript (npm) pada sampel ini?
3. Seberapa besar pre-release/build otomatis memengaruhi gambaran jumlah rilis?

3. Data (sumber, ukuran, cara dapat, etika)

- **Sumber:** PyPI JSON API (`https://pypi.org/pypi/<paket>/json`) untuk pandas, numpy, matplotlib, seaborn, scipy, requests, flask, django, fastapi, scikit-learn; npm registry (`https://registry.npmjs.org/<paket>`) untuk react, vue, express, lodash, axios.
- **Ukuran:** 15 paket, 5.703 rilis mentah → 5.697 baris bersih → 2.390 rilis stabil dipakai untuk analisis irama.
- **Cara dapat:** `requests.get` dengan `timeout`, 3× percobaan dengan jeda bertambah, cache file (24 jam), dan fallback ke cache lama bila API gagal. Hanya field yang diperlukan disimpan (respons npm penuh bisa >7 MB).
- **Waktu pengambilan:** 2026-10-02 (tersimpan pada `fetched_at` di tiap cache `data/raw/cache/*.json`).
- **Etika:** API publik tanpa autentikasi; diberi `User-Agent`, satu permintaan per paket, dan cache agar tidak membebani server.

4. Metode (cleaning, analisis, tools)

- **Ambil data:** fungsi generik `ambil_json(url, cache_file, ...)`; gabungkan dua sumber ke satu DataFrame.
- **Cleaning:** parsing waktu UTC; buang 6 baris tanpa tanggal valid dan 0 duplikat; tandai 3.299 versi pre-release (aturan PEP 440 untuk PyPI, tanda `-` untuk semver npm) dan 9 versi yanked; analisis irama memakai rilis stabil non-yanked.
- **Analisis:** jumlah rilis, median/P90 jarak antar rilis, rilis per tahun (dinormalisasi umur paket), pivot paket × tahun, perbandingan ekosistem, tren ukuran distribusi PyPI.
- **Tools:** requests, pandas, numpy, matplotlib, seaborn, packaging, openpyxl.

5. Hasil utama (temuan + rujukan figure)

1. Django punya rilis stabil terbanyak (377), tetapi fastapi paling aktif per tahun (40.2 rilis/tahun) sedangkan seaborn paling jarang (2.0/tahun). Jumlah total dipengaruhi umur paket, sehingga rilis per tahun lebih adil untuk membandingkan.

2. Jarak antar rilis sangat bervariasi. Median tercepat: Django (0.1 hari), terlama: seaborn (93.2 hari). Untuk Django, 52% interval antar rilis kurang dari 1 hari karena beberapa cabang dukungan dirilis bersamaan (rilis keamanan), sehingga mediannya mendekati nol. Pandas memiliki 114 rilis stabil dengan median 35.5 hari.
3. Perbandingan ekosistem: median rilis per paket per tahun di PyPI = 6.1 vs npm = 11.7; median jarak antar rilis PyPI = 18.4 hari vs npm = 5.8 hari (sampel hanya 10 paket PyPI dan 5 paket npm, jadi bukan generalisasi ekosistem).
4. Tahun 2014 adalah tahun dengan rilis stabil terbanyak (203 rilis dari seluruh paket yang dipantau; express sendiri menyumbang 100); pada tahun penuh terakhir (2025) total rilis = 172.
5. Proporsi pre-release berbeda jauh: PyPI 9.0% vs npm 76.3% dari seluruh versi terdaftar. Angka npm didominasi react: 95% versinya pre-release (2819 versi, 70% di antaranya build canary/experimental/nightly/dev).
6. Paket dengan rilis terakhir paling lama: seaborn (981 hari sebelum tanggal pengambilan data), sedangkan paling baru: fastapi (2 hari).
7. Ukuran distribusi NumPy (median per rilis) berubah dari 126.60 MB (2015) menjadi 901.74 MB (2026); median jumlah file per rilis naik dari 13 menjadi 72 (makin banyak wheel per platform/versi Python).

Rujukan figure: butir 1 → `figures/01_jumlah_rilis_per_paket.png`; butir 2 → `02_median_interval_rilis.png`; butir 3 → `02` dan `04_kumulatif_rilis.png`; butir 4 → `03_heatmap_rilis_per_tahun.png`; butir 7 → `05_ukuran_distribusi_pypi.png`.

Tabel: `output/ringkasan.csv`, `output/laporan_rilis.xlsx`; teks lengkap: `output/insight.txt`.

6. Visualisasi (daftar figure + arti singkat)

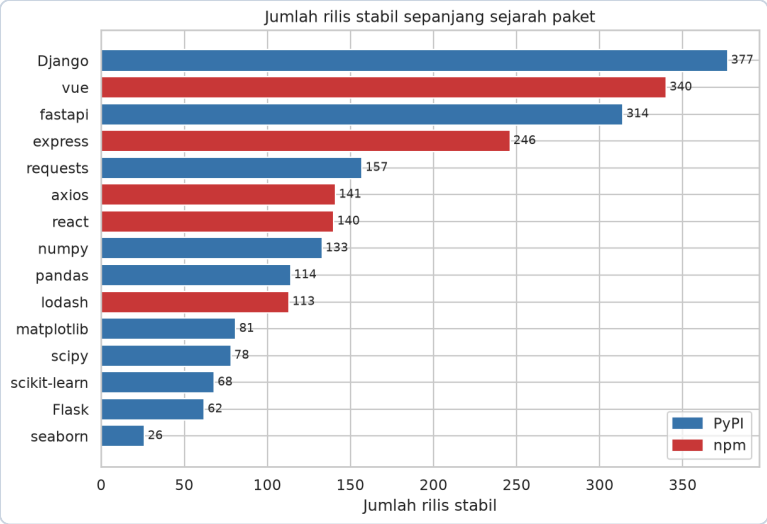
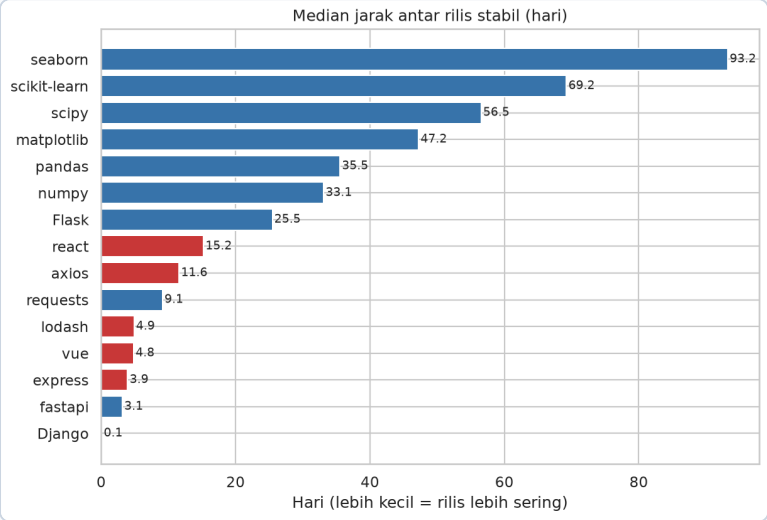
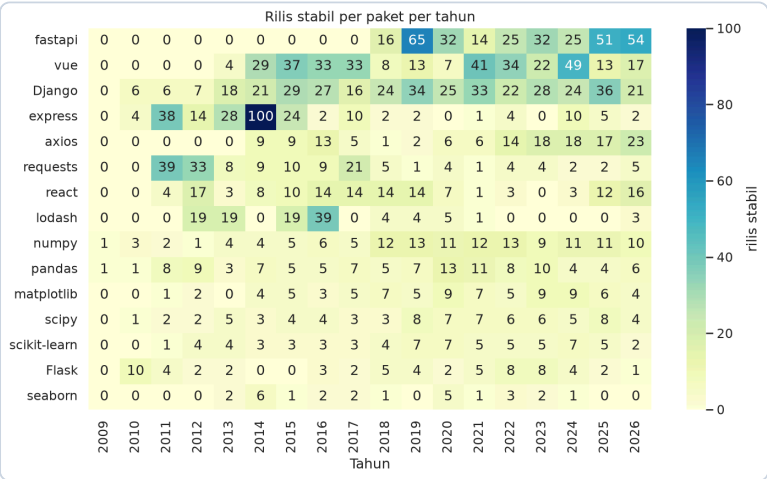
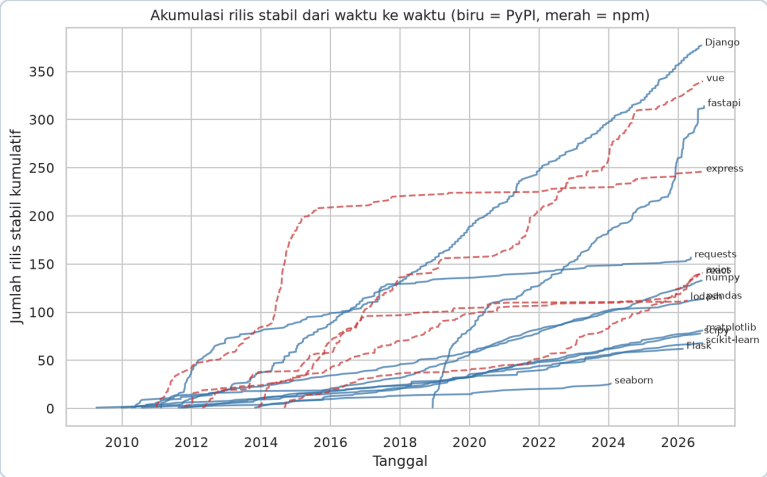
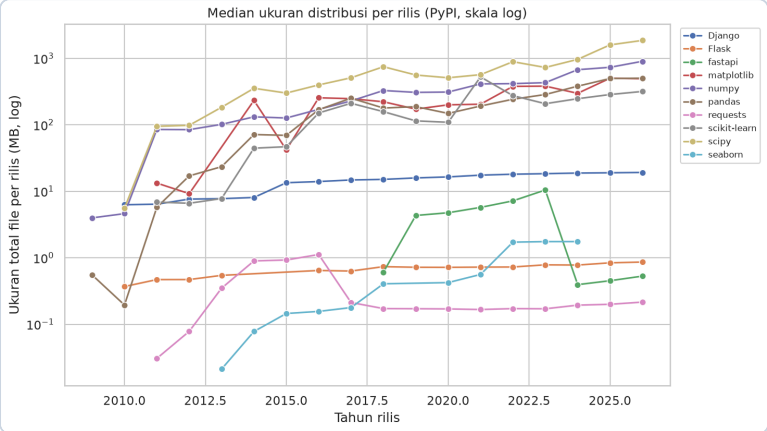
Figure	Arti singkat
	Django dan vue punya rilis stabil terbanyak; seaborn paling sedikit.
	Median jarak antar rilis berkisar dari hitungan jam (Django) hingga ±93 hari (seaborn).
	Lonjakan khas: express (100 rilis di 2014) dan fastapi yang aktif sejak 2018.

Figure	Arti singkat
	fastapi dan Django naik paling curam; seaborn hampir datar.
	Ukuran total file per rilis membesar karena makin banyak wheel per platform.

7. Batasan

- Sampel hanya 10 paket PyPI dan 5 paket npm — bukan mewakili seluruh ekosistem.
- Jumlah rilis mencerminkan kebijakan versi tiap proyek (mis. rilis keamanan serentak di Django, build canary di react), bukan kualitas atau popularitas.
- Data adalah snapshot pada waktu pengambilan; hasil dapat berbeda jika dijalankan ulang di waktu lain.
- Waktu rilis PyPI memakai upload file pertama per versi; npm memakai stempel waktu publish versi.
- Tanpa data unduhan, hubungan antara ritme rilis dan popularitas tidak dapat disimpulkan.

8. Kesimpulan & kerja lanjutan

Ritme rilis sangat bergantung pada gaya pengelolaan proyek: fastapi dan vue dirilis sangat sering, library ilmiah (numpy, pandas, scipy) rilis tiap beberapa minggu, sedangkan seaborn jarang. Pada sampel ini npm lebih sering merilis daripada PyPI, tetapi dengan 15 paket itu tidak boleh digeneralisasi. Menghitung "rilis per tahun" dan memisahkan pre-release/build otomatis mengubah gambaran secara signifikan.

Kerja lanjutan: tambahkan data unduhan; tambah paket/ekosistem; bedakan rilis mayor/minor/patch; jadwalkan pengambilan berkala.

9. Cara menjalankan

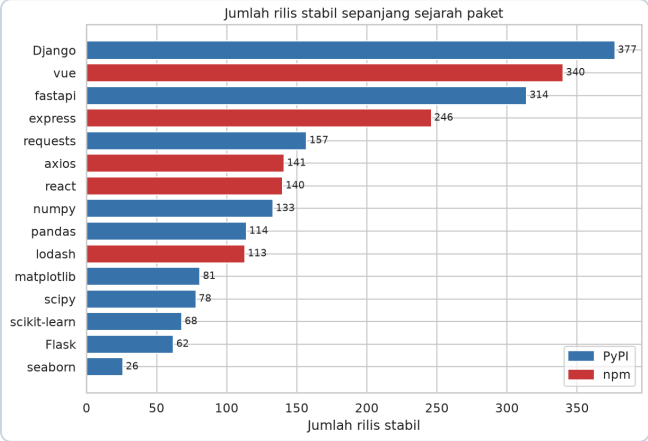
```
pip install -r requirements.txt
python src/main.py            # memakai cache bila < 24 jam
python src/main.py --refresh  # paksa ambil ulang dari API
```

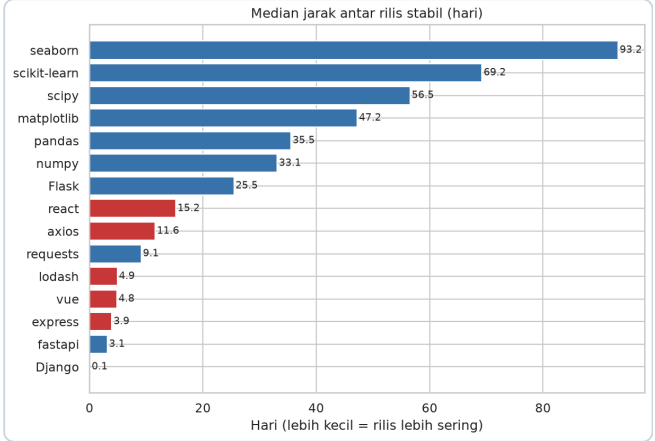
Butuh internet untuk pengambilan pertama. Log eksekusi: `output/run_log.txt`.

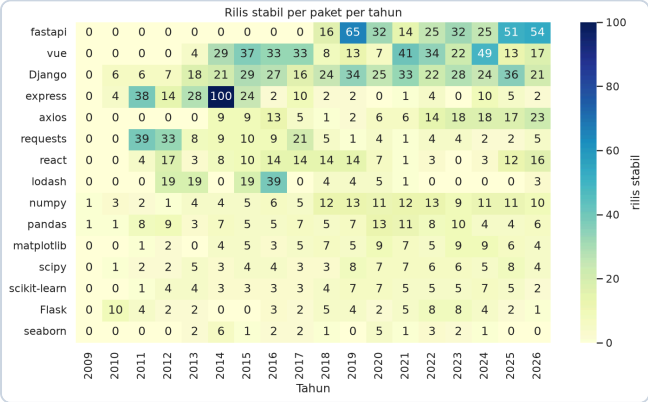
10. Struktur folder

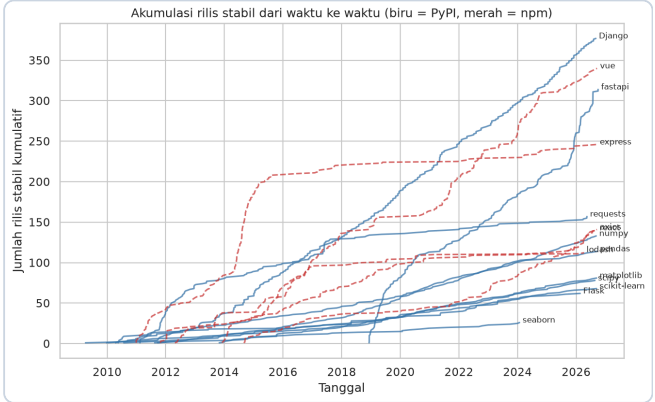
```
jalur_b_api_pypi_npm/
├── README.md
├── requirements.txt
├── data/
│   ├── raw/cache/               # respons API versi ringkas (JSON per paket)
│   ├── raw/rilis_mentah.csv     # semua rilis gabungan, belum dibersihkan
│   └── processed/rilis_bersih.csv
├── src/main.py
├── figures/                     # 5 PNG
└── output/
    ├── ringkasan.csv  laporan_rilis.xlsx
    ├── insight.txt
    └── run_log.txt
```

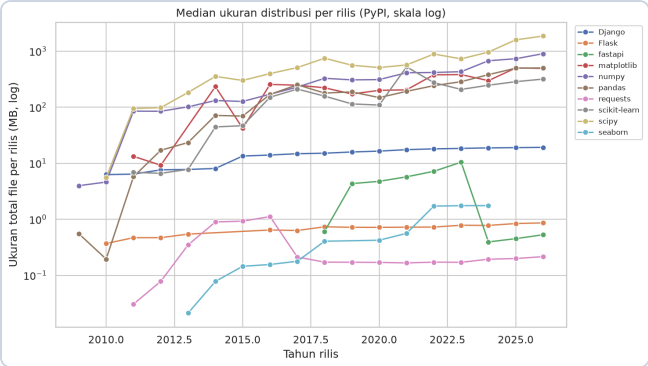
Grafik hasil











Kode sumber

jalur_b_api_pypi_npm/src/main.py

```
"""Capstone Modul 6 (v2) — Jalur B: API -> analisis -> visualisasi (PyPI + npm).

Pertanyaan: seperti apa ritme rilis library populer Python (PyPI) dibanding
JavaScript (npm)? Pipeline: ambil data (API + cache) -> DataFrame -> cleaning
-> analisis -> visualisasi -> simpan data mentah & bersih -> insight.

Pemakaian:
    python src/main.py             # pakai cache jika masih segar (< 24 jam)
    python src/main.py --refresh   # paksa ambil ulang dari API
"""
import argparse
import json
import time
from datetime import datetime, timezone
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import requests
import seaborn as sns
from packaging.version import InvalidVersion, Version

BASE_DIR = Path(__file__).resolve().parent.parent
CACHE_DIR = BASE_DIR / "data" / "raw" / "cache"
PROCESSED_DIR = BASE_DIR / "data" / "processed"
FIG_DIR = BASE_DIR / "figures"
REPORT_DIR = BASE_DIR / "output"

PYPI_PACKAGES = ["pandas", "numpy", "matplotlib", "seaborn", "scipy",
                 "requests", "flask", "django", "fastapi", "scikit-learn"]
NPM_PACKAGES = ["react", "vue", "express", "lodash", "axios"]
CACHE_TTL_JAM = 24
HEADERS = {"User-Agent": "capstone-bootcamp/1.0 (contoh edukasi)"}
WARNA = {"PyPI": "#3776ab", "npm": "#cb3837"}
PERTANYAAN = [
    "1. Library mana yang paling aktif dirilis per tahun (bukan sekadar total sepanjang masa)?",
    "2. Apakah ritme rilis library Python (PyPI) berbeda dari library JavaScript (npm) pada sampel ini?",
    "3. Seberapa besar pre-release/build otomatis memengaruhi gambaran jumlah rilis?",
]
BATASAN = [
    "Sampel hanya 10 paket PyPI dan 5 paket npm — bukan mewakili seluruh ekosistem.",
    "Jumlah rilis mencerminkan kebijakan versi tiap proyek (mis. rilis keamanan serentak di Django, build canary
di react), bukan kualitas atau popularitas.",
    "Data adalah snapshot pada waktu pengambilan; hasil dapat berbeda jika dijalankan ulang di waktu lain.",
    "Waktu rilis PyPI memakai upload file pertama per versi; npm memakai stempel waktu publish versi.",
    "Tanpa data unduhan, hubungan antara ritme rilis dan popularitas tidak dapat disimpulkan.",
]

sns.set_theme(style="whitegrid")
```

```python
# ---------------------------------------------------------------------------
# 1. Pengambilan data (generik + caching + error handling)
# ---------------------------------------------------------------------------
def ambil_json(url, cache_file, refresh=False, retry=3, timeout=30):
    """Ambil JSON dari URL dengan cache file dan retry.

    Urutan: cache segar -> API (retry, backoff) -> cache lama (fallback) -> None.
    Mengembalikan (data, sumber) dengan sumber 'cache' | 'api' | 'cache-lama'.
    """
    if cache_file.exists() and not refresh:
        umur = time.time() - cache_file.stat().st_mtime
        if umur < CACHE_TTL_JAM * 3600:
            return json.loads(cache_file.read_text(encoding="utf-8")), "cache"

    for percobaan in range(1, retry + 1):
        try:
            resp = requests.get(url, headers=HEADERS, timeout=timeout)
            resp.raise_for_status()
            return resp.json(), "api"
        except (requests.exceptions.RequestException, ValueError) as err:
            print(f"  ! percobaan {percobaan}/{retry} gagal untuk {url}: "
                  f"{type(err).__name__}")
            time.sleep(1.5 * percobaan)

    if cache_file.exists():
        print("  ! API gagal, memakai cache lama.")
        return json.loads(cache_file.read_text(encoding="utf-8")), "cache-lama"
    return None, "gagal"


def _ringkas_pypi(data):
    """Ambil hanya field yang dibutuhkan dari respons PyPI (hemat ruang)."""
    info = data["info"]
    rilis = []
    for versi, files in data["releases"].items():
        if not files:
            continue
        rilis.append({
            "version": versi,
            "upload_time": min(f["upload_time_iso_8601"] for f in files),
            "n_files": len(files),
            "size_bytes": sum(f["size"] for f in files),
            "yanked": any(f.get("yanked") for f in files),
        })
    return {"name": info["name"], "latest": info["version"],
            "summary": info.get("summary"),
            "requires_python": info.get("requires_python"), "releases": rilis}


def _ringkas_npm(data):
    """Ambil hanya field yang dibutuhkan dari respons registry npm."""
    waktu = {v: t for v, t in data["time"].items()
             if v not in ("created", "modified")}
    return {"name": data["name"], "latest": data["dist-tags"]["latest"],
            "summary": data.get("description"),
            "releases": [{"version": v, "upload_time": t} for v, t in waktu.items()]}


def kumpulkan_data(refresh=False):
    """Ambil semua paket dari kedua sumber; kembalikan (list_ringkasan, log)."""
    CACHE_DIR.mkdir(parents=True, exist_ok=True)
    hasil, log = [], []
    sumber = ([("PyPI", p, f"https://pypi.org/pypi/{p}/json", _ringkas_pypi)
               for p in PYPI_PACKAGES]
              + [("npm", p, f"https://registry.npmjs.org/{p}", _ringkas_npm)
                 for p in NPM_PACKAGES])
    for eko, paket, url, ringkas in sumber:
        cache_file = CACHE_DIR / f"{eko.lower()}_{paket}.json"
        mentah, asal = ambil_json(url, cache_file, refresh=refresh)
        if mentah is None:
            print(f"- {eko}/{paket}: GAGAL (dilewati)")
            log.append({"ekosistem": eko, "paket": paket, "asal": "gagal"})
            continue
        if asal == "api":  # simpan versi ringkas sebagai cache
```

```
            ringkasan = ringkas(mentah)
            ringkasan["fetched_at"] = datetime.now(timezone.utc).isoformat(timespec="seconds")
            cache_file.write_text(json.dumps(ringkasan), encoding="utf-8")
        else:
            ringkasan = mentah  # cache sudah berbentuk ringkas
        ringkasan["ekosistem"] = eko
        hasil.append(ringkasan)
        log.append({"ekosistem": eko, "paket": paket, "asal": asal,
                    "n_rilis_mentah": len(ringkasan["releases"])})
        print(f"- {eko}/{paket}: {len(ringkasan['releases'])} rilis ({asal})")
    return hasil, pd.DataFrame(log)


# ---------------------------------------------------------------------------
# 2. DataFrame + cleaning
# ---------------------------------------------------------------------------
def _prerelease(versi):
    """True jika versi pre-release (alpha/beta/rc/dev, atau '-' pada semver)."""
    try:
        return Version(versi).is_prerelease
    except InvalidVersion:
        return "-" in versi


def buat_dataframe(ringkasan):
    """Gabungkan data dua sumber menjadi satu DataFrame rilis."""
    baris = []
    for p in ringkasan:
        for r in p["releases"]:
            baris.append({"ekosistem": p["ekosistem"], "paket": p["name"],
                          "version": r["version"], "upload_time": r["upload_time"],
                          "n_files": r.get("n_files"),
                          "size_bytes": r.get("size_bytes"),
                          "yanked": r.get("yanked", False)})
    return pd.DataFrame(baris)


def cleaning(df):
    """Bersihkan DataFrame rilis; kembalikan (df_bersih, catatan)."""
    catatan = [f"Data mentah: {len(df)} baris, {df['paket'].nunique()} paket."]
    df = df.copy()
    df["released_at"] = pd.to_datetime(df["upload_time"], utc=True, errors="coerce")
    n_bad = int(df["released_at"].isna().sum())
    df = df.dropna(subset=["released_at"])
    catatan.append(f"{n_bad} baris tanpa tanggal valid dibuang.")

    n_dup = int(df.duplicated(subset=["paket", "version"]).sum())
    df = df.drop_duplicates(subset=["paket", "version"])
    catatan.append(f"{n_dup} duplikat (paket, versi) dibuang.")

    df["prerelease"] = df["version"].apply(_prerelease)
    df["yanked"] = df["yanked"].fillna(False).astype(bool)
    df["size_mb"] = df["size_bytes"] / 1e6
    df["tahun"] = df["released_at"].dt.year
    catatan.append(
        f"{int(df['prerelease'].sum())} versi pre-release dan "
        f"{int(df['yanked'].sum())} versi yanked ditandai; analisis irama rilis "
        "memakai rilis stabil non-yanked saja."
    )
    df = df.sort_values(["ekosistem", "paket", "released_at"]).reset_index(drop=True)
    stabil = df[~df["prerelease"] & ~df["yanked"]]
    catatan.append(f"Rilis stabil yang dianalisis: {len(stabil)} baris.")
    return df.drop(columns=["upload_time"]), catatan


# ---------------------------------------------------------------------------
# 3. Analisis
# ---------------------------------------------------------------------------
def analisis(df, ringkasan):
    """Hitung metrik irama rilis per paket dan per ekosistem."""
    stabil = df[~df["prerelease"] & ~df["yanked"]].copy()
    stabil["interval_hari"] = stabil.groupby("paket")["released_at"].diff().dt.total_seconds() / 86400
    acuan = max(pd.Timestamp(p["fetched_at"]) for p in ringkasan if "fetched_at" in p)
    tahun_acuan = acuan.year
```

```
    per_paket = stabil.groupby(["ekosistem", "paket"]).agg(
        rilis_stabil=("version", "count"),
        rilis_pertama=("released_at", "min"),
        rilis_terakhir=("released_at", "max"),
        median_interval_hari=("interval_hari", "median"),
        p90_interval_hari=("interval_hari", lambda s: s.quantile(0.9)),
    ).reset_index()
    per_paket["umur_tahun"] = (acuan - per_paket["rilis_pertama"]).dt.days / 365.25
    per_paket["rilis_per_tahun"] = per_paket["rilis_stabil"] / per_paket["umur_tahun"]
    per_paket["hari_sejak_rilis_terakhir"] = (acuan - per_paket["rilis_terakhir"]).dt.days
    versi_terbaru = {p["name"]: p["latest"] for p in ringkasan}
    per_paket["versi_terbaru"] = per_paket["paket"].map(versi_terbaru)
    per_paket = per_paket.sort_values("rilis_per_tahun", ascending=False)

    per_tahun = (stabil.pivot_table(index="paket", columns="tahun", values="version",
                                    aggfunc="count", fill_value=0))
    per_eko = stabil.groupby("ekosistem").agg(
        paket=("paket", "nunique"), rilis_stabil=("version", "count"),
        median_interval_hari=("interval_hari", "median"))
    per_eko["rilis_per_paket_per_tahun"] = per_paket.groupby("ekosistem")["rilis_per_tahun"].median()

    pre = df.groupby("ekosistem")["prerelease"].mean().rename("persen_prerelease")
    pypi_ukuran = (df[(df["ekosistem"] == "PyPI") & ~df["prerelease"]]
                   .groupby(["paket", "tahun"])["size_mb"].median().reset_index())
    return {"stabil": stabil, "per_paket": per_paket, "per_tahun": per_tahun,
            "per_eko": per_eko, "pre": pre, "pypi_ukuran": pypi_ukuran,
            "acuan": acuan, "tahun_acuan": tahun_acuan}


# ---------------------------------------------------------------------------
# 4. Visualisasi
# ---------------------------------------------------------------------------
def _simpan(fig, nama):
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    fig.savefig(FIG_DIR / f"{nama}.png", dpi=150, bbox_inches="tight")
    plt.close(fig)


def visualisasi(hasil):
    """Buat 5 grafik: jumlah rilis, interval, heatmap tahunan, kumulatif, ukuran."""
    pp = hasil["per_paket"]

    fig, ax = plt.subplots(figsize=(9, 6))
    d = pp.sort_values("rilis_stabil")
    ax.barh(d["paket"], d["rilis_stabil"], color=d["ekosistem"].map(WARNA))
    ax.set_title("Jumlah rilis stabil sepanjang sejarah paket")
    ax.set_xlabel("Jumlah rilis stabil")
    for i, v in enumerate(d["rilis_stabil"]):
        ax.text(v + 2, i, str(v), va="center", fontsize=9)
    ax.legend(handles=[plt.Rectangle((0, 0), 1, 1, color=c) for c in WARNA.values()],
              labels=list(WARNA), loc="lower right")
    _simpan(fig, "01_jumlah_rilis_per_paket")

    fig, ax = plt.subplots(figsize=(9, 6))
    d = pp.sort_values("median_interval_hari")
    ax.barh(d["paket"], d["median_interval_hari"], color=d["ekosistem"].map(WARNA))
    ax.set_title("Median jarak antar rilis stabil (hari)")
    ax.set_xlabel("Hari (lebih kecil = rilis lebih sering)")
    for i, v in enumerate(d["median_interval_hari"]):
        ax.text(v + 0.3, i, f"{v:.1f}", va="center", fontsize=9)
    _simpan(fig, "02_median_interval_rilis")

    fig, ax = plt.subplots(figsize=(11, 6))
    pt = hasil["per_tahun"]
    pt = pt.loc[pp["paket"]]  # urut sesuai rilis per tahun
    sns.heatmap(pt, annot=True, fmt="d", cmap="YlGnBu", cbar_kws={"label": "rilis stabil"}, ax=ax)
    ax.set_title("Rilis stabil per paket per tahun"); ax.set_xlabel("Tahun"); ax.set_ylabel("")
    _simpan(fig, "03_heatmap_rilis_per_tahun")

    fig, ax = plt.subplots(figsize=(10, 6))
    st = hasil["stabil"].copy()
    st["kumulatif"] = st.groupby("paket").cumcount() + 1
    for eko, grup in st.groupby("ekosistem"):
        for paket, g in grup.groupby("paket"):
            ax.plot(g["released_at"], g["kumulatif"], color=WARNA[eko], alpha=0.75,
```

```
                    linestyle="-" if eko == "PyPI" else "--", label=f"{paket}")
            ax.annotate(paket, (g["released_at"].iloc[-1], g["kumulatif"].iloc[-1]),
                        fontsize=8, xytext=(3, 0), textcoords="offset points")
    ax.set_title("Akumulasi rilis stabil dari waktu ke waktu (biru = PyPI, merah = npm)")
    ax.set_xlabel("Tanggal"); ax.set_ylabel("Jumlah rilis stabil kumulatif")
    _simpan(fig, "04_kumulatif_rilis")

    fig, ax = plt.subplots(figsize=(10, 6))
    uk = hasil["pypi_ukuran"]
    sns.lineplot(data=uk, x="tahun", y="size_mb", hue="paket", marker="o", ax=ax)
    ax.set_yscale("log")
    ax.set_title("Median ukuran distribusi per rilis (PyPI, skala log)")
    ax.set_xlabel("Tahun rilis"); ax.set_ylabel("Ukuran total file per rilis (MB, log)")
    ax.legend(bbox_to_anchor=(1.01, 1), loc="upper left", fontsize=8)
    _simpan(fig, "05_ukuran_distribusi_pypi")


# ---------------------------------------------------------------------------
# 5. Insight + simpan
# ---------------------------------------------------------------------------
def tulis_insight(hasil, df):
    """Susun insight dari angka hasil analisis."""
    pp = hasil["per_paket"].set_index("paket")
    eko = hasil["per_eko"]
    pt = hasil["per_tahun"]
    teraktif = pp["rilis_per_tahun"].idxmax()
    terjarang = pp["rilis_per_tahun"].idxmin()
    tercepat = pp["median_interval_hari"].idxmin()
    terlama = pp["median_interval_hari"].idxmax()
    terbanyak = pp["rilis_stabil"].idxmax()
    pandas_n = pp.loc["pandas", "rilis_stabil"]
    py_med = eko.loc["PyPI", "rilis_per_paket_per_tahun"]
    js_med = eko.loc["npm", "rilis_per_paket_per_tahun"]
    pre = hasil["pre"]
    tahun_penuh = hasil["tahun_acuan"] - 1
    puncak_tahun = pt.sum().idxmax()
    basi = pp["hari_sejak_rilis_terakhir"].sort_values(ascending=False)
    uk = hasil["pypi_ukuran"]
    uk_np = uk[uk["paket"] == "numpy"].set_index("tahun")["size_mb"]

    cepat_hari = hasil["stabil"].loc[hasil["stabil"]["paket"] == tercepat, "interval_hari"]
    persen_serentak = (cepat_hari < 1).mean()
    pre_paket = df.groupby("paket")["prerelease"].mean().sort_values(ascending=False)
    pre_top = pre_paket.index[0]
    pre_top_df = df[(df["paket"] == pre_top) & df["prerelease"]]
    persen_build = pre_top_df["version"].str.contains("canary|experimental|nightly|dev").mean()
    tahun_max = int(uk_np.index.max())
    files_np = df[(df["paket"] == "numpy") & ~df["prerelease"]].groupby("tahun")["n_files"].median()

    baris = [
        f"1. **{terbanyak} punya rilis stabil terbanyak ({pp.loc[terbanyak, 'rilis_stabil']}), "
        f"tetapi {teraktif} paling aktif per tahun** ({pp.loc[teraktif, 'rilis_per_tahun']:.1f} "
        f"rilis/tahun) sedangkan {terjarang} paling jarang ({pp.loc[terjarang, 'rilis_per_tahun']:.1f}/tahun). "
        f"Jumlah total dipengaruhi umur paket, sehingga rilis per tahun lebih adil untuk membandingkan.",
        f"2. **Jarak antar rilis sangat bervariasi.** Median tercepat: {tercepat} "
        f"({pp.loc[tercepat, 'median_interval_hari']:.1f} hari), terlama: {terlama} "
        f"({pp.loc[terlama, 'median_interval_hari']:.1f} hari). "
        f"Untuk {tercepat}, {persen_serentak:.0%} interval antar rilis kurang dari 1 hari karena beberapa cabang "
        f"dukungan dirilis bersamaan (rilis keamanan), sehingga mediannya mendekati nol. "
        f"Pandas memiliki {pandas_n} rilis stabil dengan median {pp.loc['pandas', 'median_interval_hari']:.1f} 
hari.",
        f"3. **Perbandingan ekosistem:** median rilis per paket per tahun di PyPI = {py_med:.1f} "
        f"vs npm = {js_med:.1f}; median jarak antar rilis PyPI = "
        f"{eko.loc['PyPI', 'median_interval_hari']:.1f} hari vs npm = {eko.loc['npm', 'median_interval_hari']:.1f} 
hari "
        f"(sampel hanya {int(eko.loc['PyPI','paket'])} paket PyPI dan {int(eko.loc['npm','paket'])} paket npm, "
        f"jadi bukan generalisasi ekosistem).",
        f"4. **Tahun {puncak_tahun} adalah tahun dengan rilis stabil terbanyak** "
        f"({int(pt.sum()[puncak_tahun])} rilis dari seluruh paket yang dipantau; "
        f"{pt[puncak_tahun].idxmax()} sendiri menyumbang {int(pt[puncak_tahun].max())}); "
        f"pada tahun penuh terakhir ({tahun_penuh}) total rilis = {int(pt.sum().get(tahun_penuh, 0))}.",
        f"5. **Proporsi pre-release berbeda jauh:** PyPI {pre['PyPI']:.1%} vs npm {pre['npm']:.1%} dari "
        f"seluruh versi terdaftar. Angka npm didominasi {pre_top}: {pre_paket[pre_top]:.0%} versinya "
        f"pre-release ({len(pre_top_df)} versi, {persen_build:.0%} di antaranya build "
        f"canary/experimental/nightly/dev).",
```

```
        f"6. **Paket dengan rilis terakhir paling lama:** {basi.index[0]} "
        f"({int(basi.iloc[0])} hari sebelum tanggal pengambilan data), "
        f"sedangkan paling baru: {basi.index[-1]} ({int(basi.iloc[-1])} hari).",
    ]
    if 2015 in uk_np.index and uk_np.index.max() > 2015:
        baris.append(
            f"7. **Ukuran distribusi NumPy** (median per rilis) berubah dari {uk_np[2015]:.2f} MB (2015) "
            f"menjadi {uk_np.iloc[-1]:.2f} MB ({tahun_max}); median jumlah file per rilis naik dari "
            f"{files_np[2015]:.0f} menjadi {files_np[tahun_max]:.0f} (makin banyak wheel per platform/versi
Python)."
        )
    return "\n\n".join(baris) + "\n"


def simpan(df_mentah, df, hasil, catatan, log, insight):
    """Simpan data mentah, data bersih, ringkasan, Excel, dan insight."""
    for d in (PROCESSED_DIR, REPORT_DIR):
        d.mkdir(parents=True, exist_ok=True)
    df_mentah.to_csv(BASE_DIR / "data" / "raw" / "rilis_mentah.csv", index=False)
    df_simpan = df.copy()  # format waktu seragam agar mudah dibaca ulang pd.read_csv
    df_simpan["released_at"] = df_simpan["released_at"].dt.strftime("%Y-%m-%dT%H:%M:%SZ")
    df_simpan.to_csv(PROCESSED_DIR / "rilis_bersih.csv", index=False)
    pp = hasil["per_paket"].copy()
    for k in ("rilis_pertama", "rilis_terakhir"):
        pp[k] = pp[k].dt.strftime("%Y-%m-%d")
    pp.round(2).to_csv(REPORT_DIR / "ringkasan.csv", index=False)
    with pd.ExcelWriter(REPORT_DIR / "laporan_rilis.xlsx") as xl:
        pp.round(2).to_excel(xl, sheet_name="Per paket", index=False)
        hasil["per_tahun"].to_excel(xl, sheet_name="Rilis per tahun")
        hasil["per_eko"].round(2).to_excel(xl, sheet_name="Per ekosistem")
        log.to_excel(xl, sheet_name="Log pengambilan", index=False)
        pd.DataFrame({"catatan_cleaning": catatan}).to_excel(xl, sheet_name="Catatan cleaning", index=False)
    teks = insight.replace("**", "") + "\nBatasan:\n" + "\n".join(f"- {b}" for b in BATASAN) + "\n"
    (REPORT_DIR / "insight.txt").write_text(teks, encoding="utf-8")


def main():
    parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    parser.add_argument("--refresh", action="store_true", help="abaikan cache, ambil ulang dari API")
    args = parser.parse_args()

    print("PERTANYAAN ANALISIS")
    print("\n".join(PERTANYAAN))
    print()

    print("=" * 60 + "\n1. PENGAMBILAN DATA\n" + "=" * 60)
    ringkasan, log = kumpulkan_data(refresh=args.refresh)
    if not ringkasan:
        raise SystemExit("Tidak ada data yang berhasil diambil dan tidak ada cache.")

    print("\n" + "=" * 60 + "\n2. CLEANING\n" + "=" * 60)
    df_mentah = buat_dataframe(ringkasan)
    print("Shape mentah:", df_mentah.shape)
    df, catatan = cleaning(df_mentah)
    for c in catatan:
        print("-", c)
    print("Shape bersih:", df.shape)

    print("\n" + "=" * 60 + "\n3. ANALISIS\n" + "=" * 60)
    hasil = analisis(df, ringkasan)
    print("Tanggal acuan (waktu pengambilan data):", hasil["acuan"].date())
    kolom = ["ekosistem", "paket", "rilis_stabil", "median_interval_hari",
             "rilis_per_tahun", "hari_sejak_rilis_terakhir", "versi_terbaru"]
    print(hasil["per_paket"][kolom].round(2).to_string(index=False))
    print("\nPer ekosistem:")
    print(hasil["per_eko"].round(2).to_string())
    print("\nProporsi pre-release:")
    print(hasil["pre"].round(3).to_string())

    visualisasi(hasil)
    insight = tulis_insight(hasil, df)
    simpan(df_mentah, df, hasil, catatan, log, insight)
    print("\n" + "=" * 60 + "\n4. INSIGHT\n" + "=" * 60)
    print(insight)
    print("Batasan:")
```

```
    print("\n".join(f"- {b}" for b in BATASAN))
    print("\nSelesai. Lihat data/processed/, figures/, dan output/.")


if __name__ == "__main__":
    main()
```

Jalur C — Pipeline Paper arXiv

> Contoh pengerjaan untuk **Modul 6 (versi 2), Jalur C**.
>
> ⚠️ **Data pada contoh ini adalah SAMPEL FIKTIF, bukan hasil arXiv sebenarnya.** API arXiv (`export.arxiv.org`) diblokir di lingkungan tempat contoh ini dibuat (HTTP 403), sehingga program otomatis memakai `data/raw/arxiv_sample.xml` yang dibuat acak oleh `src/buat_sampel.py` (tautan memakai `example.org`). Kode, alur, dan format output adalah yang akan Anda pakai; **angka dan temuan pada output ini hanya untuk memahami pipeline**. Jalankan di komputer dengan internet untuk mendapat hasil nyata.

1. Latar belakang & motivasi

Mahasiswa pascasarjana sering perlu memetakan literatur: seberapa cepat topik berkembang, kata kunci apa yang dominan, dan siapa penulis yang berulang. Proyek ini membangun pipeline kecil untuk itu dari metadata arXiv.

2. Pertanyaan analisis

1. Bagaimana jumlah paper bertopik deep learning + education berkembang dari tahun ke tahun?
2. Kata/topik apa yang paling sering muncul di judul?
3. Siapa penulis yang paling sering muncul dan seberapa terkonsentrasi penulisnya?

3. Data (sumber, ukuran, cara dapat, etika)

- **Sumber (mode live):** arXiv API `http://export.arxiv.org/api/query` dengan query `all:"deep learning" AND all:education`, diurutkan `submittedDate` menurun.
- **Ukuran:** 3 halaman × 10 paper = 30 paper (≥ 20 sesuai scope minimum), paginasi dengan parameter `start`.
- **Cara dapat:** `requests.get` dengan `timeout`; jeda 3 detik antar halaman; XML mentah tiap halaman disimpan ke `data/raw/arxiv_halaman_N.xml` begitu berhasil (agar tidak perlu mengulang bila nanti gagal).
- **Fallback:** bila API gagal, program memakai data sampel fiktif dan **menandainya** pada judul grafik dan awal `insight.txt`.
- **Etika:** hanya metadata publik; jeda antar request sesuai pedoman arXiv; tidak ada scraping halaman web.

4. Metode (cleaning, analisis, tools)

- **Cleaning:** buang judul duplikat (1 pada sampel ini) dan baris tanpa tanggal valid; bentuk kolom `tahun` dan `n_penulis`.
- **Analisis:** jumlah paper per tahun, frekuensi kata judul (≥ 3 huruf, tanpa stopword), penulis teratas, rata-rata penulis per paper, konsentrasi tiga penulis teratas.
- **Tools:** requests, feedparser, pandas, matplotlib, seaborn.

5. Hasil utama (temuan + rujukan figure)

Perhatian: dihitung dari data sampel fiktif.

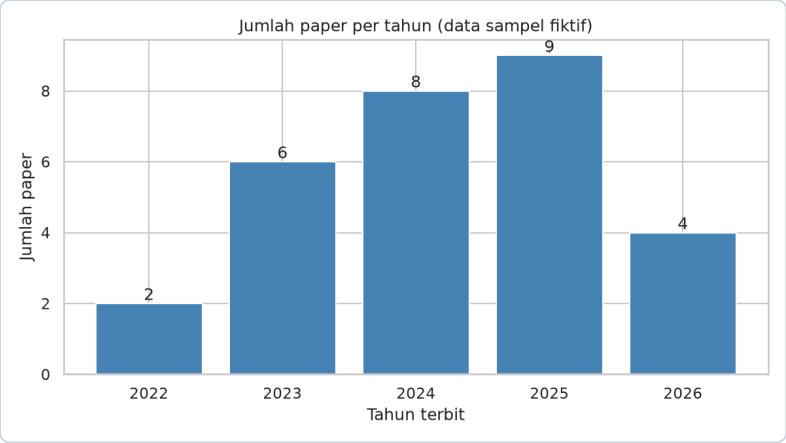
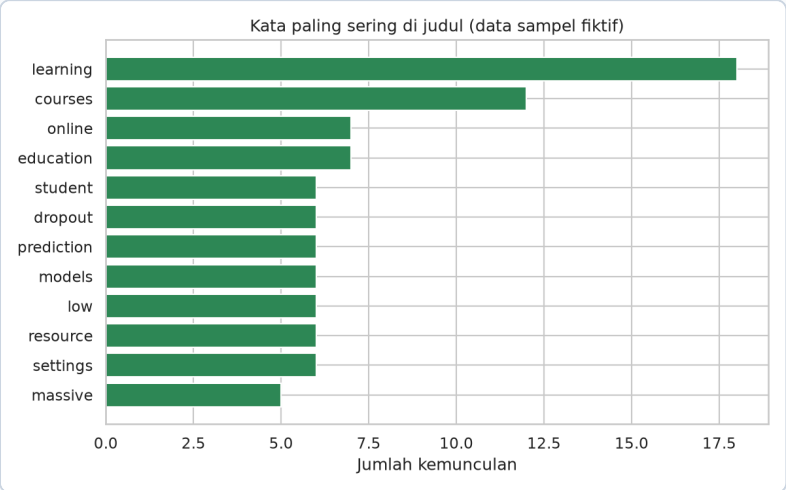
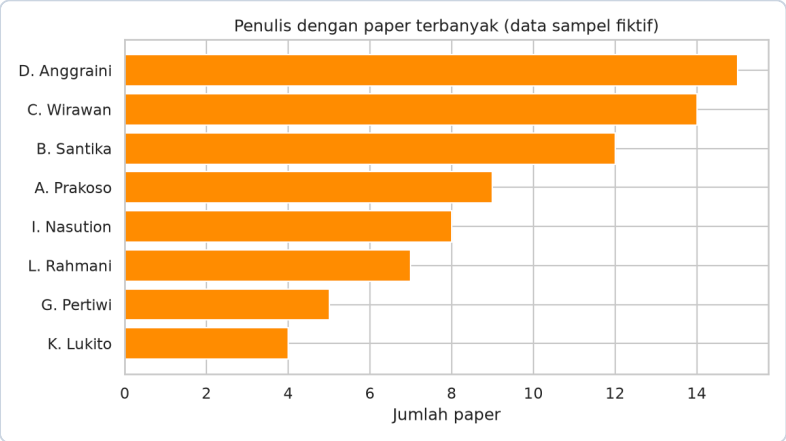
1. Korpus: 29 paper, terbit 2022-10-05 s.d. 2026-08-20.

2. Tahun dengan paper terbanyak: 2025 (9 paper); tahun 2022 hanya 2, sehingga jumlah pada tahun penuh terakhir (2025) = 9 (≈ 4.5× tahun pertama).
3. Kata judul paling sering: 'learning' (18×) dan 'courses' (12×) — menandakan fokus topik pada kata tersebut.
4. Rata-rata 2.9 penulis per paper; penulis teratas D. Anggraini muncul di 15 paper.
5. Tiga penulis teratas menyumbang 41 dari 85 kemunculan nama penulis (48%) — indikasi tingkat konsentrasi penulis pada korpus ini.

Rujukan figure: butir 2 → `figures/01_paper_per_tahun.png`; butir 3 → `figures/02_kata_judul.png`; butir 4–5 → `figures/03_penulis_teratas.png`.

Tabel: `output/ringkasan.csv`, `output/kata_judul.csv`, `output/penulis_teratas.csv`; teks lengkap: `output/insight.txt`.

6. Visualisasi (daftar figure + arti singkat)

Figure	Arti singkat
	Jumlah paper per tahun terbit (judul grafik menandai bahwa ini data sampel fiktif).
	Kata yang paling sering muncul pada judul.
	Penulis dengan jumlah paper terbanyak pada korpus.

7. Batasan

- Hanya metadata (judul, penulis, tanggal) — isi paper tidak dianalisis; kata judul tidak sama dengan topik sebenarnya.
- Hasil bergantung pada query: query berbeda akan menghasilkan korpus berbeda (bias query).
- Ukuran korpus kecil (30 paper) sehingga pola per tahun dan penulis tidak stabil secara statistik.

- Satu nama penulis dapat ditulis berbeda antar paper (tidak ada disambiguasi penulis).
- Tanggal 'published' adalah tanggal submit versi pertama, bukan tanggal publikasi jurnal.
- **Khusus contoh ini:** seluruh data fiktif; tidak ada kesimpulan tentang literatur sebenarnya yang dapat diambil.

8. Kesimpulan & kerja lanjutan

Pipeline lengkap dan teruji untuk alur paginasi → metadata → analisis waktu/kata/penulis → visualisasi → insight, termasuk penanganan kegagalan API dengan fallback yang dilabeli jelas. Untuk hasil nyata, jalankan dengan internet.

Kerja lanjutan: tambah analisis tren kata per tahun; gunakan abstrak (bukan hanya judul); bandingkan dua query; deduplikasi nama penulis.

9. Cara menjalankan

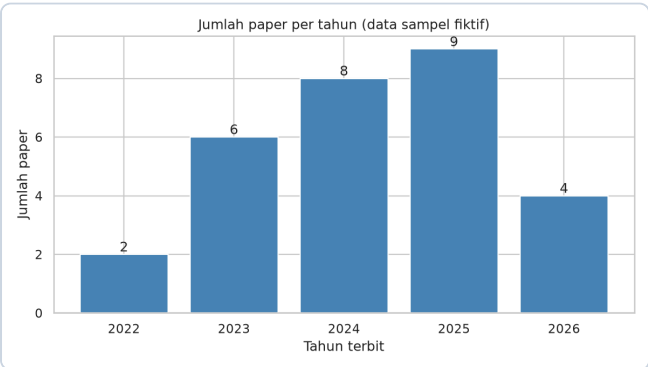
```
pip install -r requirements.txt
python src/main.py            # coba API arXiv; bila gagal pakai sampel fiktif
python src/main.py --offline  # langsung pakai sampel fiktif
python src/buat_sampel.py     # (opsional) membuat ulang data sampel fiktif
```

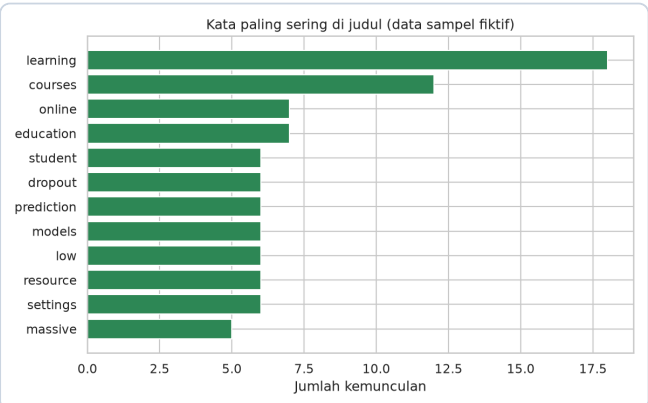
Log eksekusi (API gagal → fallback): `output/run_log.txt`.

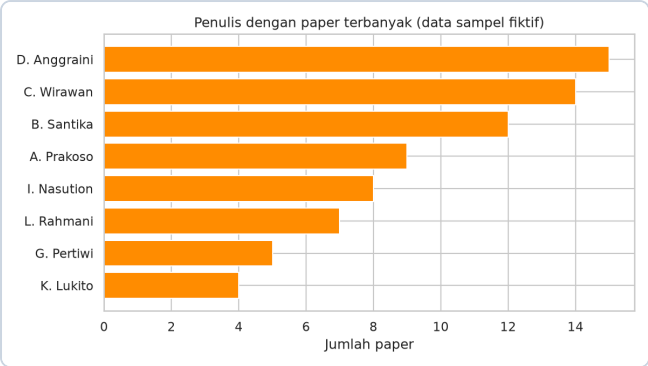
10. Struktur folder

```
jalur_c_arxiv/
├── README.md
├── requirements.txt
├── data/
│   ├── raw/arxiv_sample.xml      # SAMPEL FIKTIF (fallback)
│   ├── raw/paper_mentah.csv
│   └── processed/paper_bersih.csv
├── src/
│   ├── main.py
│   └── buat_sampel.py
├── figures/                      # 3 PNG
└── output/
    ├── ringkasan.csv  kata_judul.csv  penulis_teratas.csv
    ├── insight.txt
    └── run_log.txt
```

Grafik hasil







Kode sumber

jalur_c_arxiv/src/main.py

```
"""Capstone Modul 6 (v2) — Jalur C: arXiv / literatur + ringkasan kuantitatif.

Alur: ambil paper (paginasi + jeda + error handling) -> DataFrame -> cleaning ->
analisis (per tahun, kata judul, penulis teratas) -> visualisasi -> insight.

Sumber data:
    python src/main.py               # live ke API arXiv; jika gagal, pakai sampel fiktif
    python src/main.py --offline     # langsung pakai data sampel fiktif (tanpa internet)

PENTING: bila API tidak dapat dijangkau, program memakai data/raw/arxiv_sample.xml
yang FIKTIF (lihat src/buat_sampel.py). Hasil dari sampel hanya demonstrasi pipeline,
bukan temuan tentang literatur sebenarnya; insight.txt akan memuat peringatan itu.
"""
import argparse
import re
import time
from collections import Counter
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import feedparser
import matplotlib.pyplot as plt
import pandas as pd
import requests
import seaborn as sns

BASE_DIR = Path(__file__).resolve().parent.parent
RAW_DIR = BASE_DIR / "data" / "raw"
PROC_DIR = BASE_DIR / "data" / "processed"
FIG_DIR = BASE_DIR / "figures"
OUT_DIR = BASE_DIR / "output"
SAMPEL = RAW_DIR / "arxiv_sample.xml"

API = "http://export.arxiv.org/api/query"
QUERY = 'all:"deep learning" AND all:education'
PER_HALAMAN = 10
JUMLAH_HALAMAN = 3          # total 30 paper (>= 20 sesuai scope minimum)
JEDA_DETIK = 3              # hormati server arXiv

PERTANYAAN = [
    "1. Bagaimana jumlah paper bertopik deep learning + education berkembang dari tahun ke tahun?",
    "2. Kata/topik apa yang paling sering muncul di judul?",
    "3. Siapa penulis yang paling sering muncul dan seberapa terkonsentrasi penulisnya?",
]
STOP = {"the", "a", "an", "of", "and", "for", "in", "on", "to", "with", "from", "using", "via", "by", "at"}
sns.set_theme(style="whitegrid")


# ---------------------------------------------------------------------------
# 1. Ambil data
# ---------------------------------------------------------------------------
def fetch_halaman(query, start, n, timeout=20):
```

```
    """Ambil satu halaman dari API arXiv; kembalikan teks Atom, atau None jika gagal."""
    params = {"search_query": query, "start": start, "max_results": n,
              "sortBy": "submittedDate", "sortOrder": "descending"}
    try:
        r = requests.get(API, params=params, timeout=timeout)
        r.raise_for_status()
        return r.text
    except requests.exceptions.RequestException as err:
        print(f"  ! halaman start={start} gagal: {type(err).__name__}")
        return None


def parse_feed(teks):
    """Ubah teks Atom menjadi list dict metadata paper."""
    feed = feedparser.parse(teks)
    baris = []
    for e in feed.entries:
        terbit = e.get("published", "")[:10]
        baris.append({
            "title": " ".join(e.title.split()),
            "authors": "; ".join(a.name for a in e.get("authors", [])),
            "published": terbit,
            "link": e.get("link", ""),
        })
    return baris


def ambil_live():
    """Ambil JUMLAH_HALAMAN halaman dengan jeda; simpan XML mentah tiap halaman."""
    semua = []
    for i in range(JUMLAH_HALAMAN):
        print(f"Mengambil halaman {i + 1}/{JUMLAH_HALAMAN} ...")
        teks = fetch_halaman(QUERY, i * PER_HALAMAN, PER_HALAMAN)
        if teks is None:
            return None            # gagal -> biarkan pemanggil memakai cadangan
        RAW_DIR.mkdir(parents=True, exist_ok=True)
        (RAW_DIR / f"arxiv_halaman_{i + 1}.xml").write_text(teks, encoding="utf-8")
        semua.extend(parse_feed(teks))
        if i < JUMLAH_HALAMAN - 1:
            time.sleep(JEDA_DETIK)
    return semua


def ambil_sampel():
    """Baca data sampel fiktif; potong sesuai ukuran halaman agar sama dengan mode live."""
    baris = parse_feed(SAMPEL.read_text(encoding="utf-8"))
    return baris[: PER_HALAMAN * JUMLAH_HALAMAN]


def muat_data(offline):
    """Kembalikan (DataFrame, sumber) dengan sumber 'arXiv (live)' atau 'sampel fiktif'."""
    if not offline:
        data = ambil_live()
        if data:
            return pd.DataFrame(data), "arXiv (live)"
        print("API arXiv tidak dapat dijangkau -> memakai DATA SAMPEL FIKTIF.")
    return pd.DataFrame(ambil_sampel()), "sampel fiktif"


# ---------------------------------------------------------------------------
# 2. Cleaning, analisis
# ---------------------------------------------------------------------------
def cleaning(df):
    """Bersihkan metadata; kembalikan (df, catatan)."""
    catatan = [f"Data awal: {len(df)} baris."]
    df = df.copy()
    n_dup = int(df.duplicated(subset=["title"]).sum())
    df = df.drop_duplicates(subset=["title"])
    catatan.append(f"{n_dup} judul duplikat dibuang.")
    df["published"] = pd.to_datetime(df["published"], errors="coerce")
    n_bad = int(df["published"].isna().sum())
    df = df.dropna(subset=["published"])
    catatan.append(f"{n_bad} baris tanpa tanggal valid dibuang.")
    df["tahun"] = df["published"].dt.year
    df["n_penulis"] = df["authors"].str.count(";") + 1
```

```
    return df.reset_index(drop=True), catatan


def kata_judul(df, n=12):
    """Frekuensi kata judul (huruf, >= 3 karakter, tanpa stopword)."""
    hitung = Counter()
    for judul in df["title"]:
        for w in re.findall(r"[a-zA-Z]{3,}", judul.lower()):
            if w not in STOP:
                hitung[w] += 1
    return hitung.most_common(n)


def penulis_teratas(df, n=8):
    """Penulis dengan jumlah kemunculan terbanyak."""
    hitung = Counter(a.strip() for s in df["authors"] for a in s.split(";") if a.strip())
    return hitung.most_common(n)


def analisis(df):
    """Hitung ringkasan kuantitatif."""
    return {
        "per_tahun": df.groupby("tahun").size().rename("jumlah_paper"),
        "kata": kata_judul(df),
        "penulis": penulis_teratas(df),
        "rata_penulis": df["n_penulis"].mean(),
        "rentang": (df["published"].min().date(), df["published"].max().date()),
    }


# ---------------------------------------------------------------------------
# 3. Visualisasi
# ---------------------------------------------------------------------------
def visualisasi(hasil, sumber):
    """Simpan 3 figure: paper per tahun, kata judul, penulis teratas."""
    FIG_DIR.mkdir(parents=True, exist_ok=True)
    cat = " (data sampel fiktif)" if sumber == "sampel fiktif" else ""

    fig, ax = plt.subplots(figsize=(8, 4.5))
    pt = hasil["per_tahun"]
    ax.bar(pt.index.astype(str), pt.values, color="steelblue")
    for i, v in enumerate(pt.values):
        ax.text(i, v + 0.1, str(v), ha="center")
    ax.set_title("Jumlah paper per tahun" + cat)
    ax.set_xlabel("Tahun terbit"); ax.set_ylabel("Jumlah paper")
    fig.tight_layout(); fig.savefig(FIG_DIR / "01_paper_per_tahun.png", dpi=150); plt.close(fig)

    fig, ax = plt.subplots(figsize=(8, 5))
    kata, n = zip(*hasil["kata"])
    ax.barh(list(kata)[::-1], list(n)[::-1], color="seagreen")
    ax.set_title("Kata paling sering di judul" + cat)
    ax.set_xlabel("Jumlah kemunculan")
    fig.tight_layout(); fig.savefig(FIG_DIR / "02_kata_judul.png", dpi=150); plt.close(fig)

    fig, ax = plt.subplots(figsize=(8, 4.5))
    nama, n = zip(*hasil["penulis"])
    ax.barh(list(nama)[::-1], list(n)[::-1], color="darkorange")
    ax.set_title("Penulis dengan paper terbanyak" + cat)
    ax.set_xlabel("Jumlah paper")
    fig.tight_layout(); fig.savefig(FIG_DIR / "03_penulis_teratas.png", dpi=150); plt.close(fig)


# ---------------------------------------------------------------------------
# 4. Insight + simpan
# ---------------------------------------------------------------------------
def tulis_insight(df, hasil, sumber):
    """Susun insight (>= 5 poin) + batasan; tulis ke output/insight.txt."""
    pt = hasil["per_tahun"]
    tahun_max = pt.idxmax()
    kata = dict(hasil["kata"])
    (k1, n1), (k2, n2) = hasil["kata"][0], hasil["kata"][1]
    pen = hasil["penulis"]
    total_penulis = sum(len(s.split(";")) for s in df["authors"])
    top3 = sum(n for _, n in pen[:3])
    tahun_penuh = pt[pt.index < pt.index.max()]
```

```
    naik = (tahun_penuh.iloc[-1] / tahun_penuh.iloc[0]) if len(tahun_penuh) > 1 and tahun_penuh.iloc[0] else
float("nan")
    baris = [
        f"1. Korpus: {len(df)} paper, terbit {hasil['rentang'][0]} s.d. {hasil['rentang'][1]}.",
        f"2. Tahun dengan paper terbanyak: {tahun_max} ({int(pt.max())} paper); tahun {tahun_penuh.index[0]} hanya
{int(tahun_penuh.iloc[0])}, "
        f"sehingga jumlah pada tahun penuh terakhir ({tahun_penuh.index[-1]}) = {int(tahun_penuh.iloc[-1])} (≈
{naik:.1f}× tahun pertama).",
        f"3. Kata judul paling sering: '{k1}' ({n1}×) dan '{k2}' ({n2}×) — menandakan fokus topik pada kata
tersebut.",
        f"4. Rata-rata {hasil['rata_penulis']:.1f} penulis per paper; penulis teratas {pen[0][0]} muncul di
{pen[0][1]} paper.",
        f"5. Tiga penulis teratas menyumbang {top3} dari {total_penulis} kemunculan nama penulis ({top3 /
total_penulis:.0%}) "
        "— indikasi tingkat konsentrasi penulis pada korpus ini.",
    ]
    batasan = [
        "Hanya metadata (judul, penulis, tanggal) — isi paper tidak dianalisis; kata judul tidak sama dengan topik
sebenarnya.",
        "Hasil bergantung pada query: query berbeda akan menghasilkan korpus berbeda (bias query).",
        "Ukuran korpus kecil (30 paper) sehingga pola per tahun dan penulis tidak stabil secara statistik.",
        "Satu nama penulis dapat ditulis berbeda antar paper (tidak ada disambiguasi penulis).",
        "Tanggal 'published' adalah tanggal submit versi pertama, bukan tanggal publikasi jurnal.",
    ]
    kepala = ""
    if sumber == "sampel fiktif":
        kepala = ("PERINGATAN: angka di bawah dihitung dari DATA SAMPEL FIKTIF (data/raw/arxiv_sample.xml), "
                  "bukan hasil pencarian arXiv sebenarnya. Gunakan untuk memahami alur pipeline saja.\n\n")
    teks = kepala + "\n".join(baris) + "\n\nBatasan:\n" + "\n".join(f"- {b}" for b in batasan) + "\n"
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    (OUT_DIR / "insight.txt").write_text(teks, encoding="utf-8")
    return teks


def simpan(df, hasil):
    """Simpan data bersih dan ringkasan CSV."""
    PROC_DIR.mkdir(parents=True, exist_ok=True)
    OUT_DIR.mkdir(parents=True, exist_ok=True)
    df.assign(published=df["published"].dt.strftime("%Y-%m-%d")).to_csv(PROC_DIR / "paper_bersih.csv",
index=False)
    ring = hasil["per_tahun"].reset_index()
    ring["persen"] = (ring["jumlah_paper"] / ring["jumlah_paper"].sum() * 100).round(1)
    ring.to_csv(OUT_DIR / "ringkasan.csv", index=False)
    pd.DataFrame(hasil["kata"], columns=["kata", "jumlah"]).to_csv(OUT_DIR / "kata_judul.csv", index=False)
    pd.DataFrame(hasil["penulis"], columns=["penulis", "jumlah_paper"]).to_csv(OUT_DIR / "penulis_teratas.csv",
index=False)


def main():
    parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    parser.add_argument("--offline", action="store_true", help="pakai data sampel fiktif tanpa memanggil API")
    args = parser.parse_args()

    print("PERTANYAAN ANALISIS\n" + "\n".join(PERTANYAAN) + "\n")
    print("=" * 60 + "\n1. PENGAMBILAN DATA\n" + "=" * 60)
    df_mentah, sumber = muat_data(args.offline)
    print(f"Sumber data: {sumber}; {len(df_mentah)} paper mentah")
    RAW_DIR.mkdir(parents=True, exist_ok=True)
    df_mentah.to_csv(RAW_DIR / "paper_mentah.csv", index=False)

    print("\n" + "=" * 60 + "\n2. CLEANING\n" + "=" * 60)
    df, catatan = cleaning(df_mentah)
    for c in catatan:
        print("-", c)
    print("Shape bersih:", df.shape)
    print(df[["title", "published", "n_penulis"]].head(5).to_string(index=False))

    print("\n" + "=" * 60 + "\n3. ANALISIS\n" + "=" * 60)
    hasil = analisis(df)
    print("Paper per tahun:\n" + hasil["per_tahun"].to_string())
    print("\nKata judul teratas:", hasil["kata"])
    print("\nPenulis teratas:", hasil["penulis"])

    visualisasi(hasil, sumber)
    simpan(df, hasil)
```

```
    teks = tulis_insight(df, hasil, sumber)
    print("\n" + "=" * 60 + "\n4. INSIGHT\n" + "=" * 60)
    print(teks)
    print("Selesai. Lihat data/processed/, figures/, dan output/.")


if __name__ == "__main__":
    main()
```

Jalur D — Aplikasi Analisis Nilai Berbasis OOP

Nama

Capstone Python Intensive Bootcamp — contoh pengerjaan lengkap

Jalur Capstone

Jalur D — Aplikasi Analisis Nilai Berbasis OOP (level tinggi), dengan ketiga stretch goal: perbandingan dua prodi, mode interaktif terminal, dan simpan/muat state.

Deskripsi Singkat

Aplikasi berbasis kelas `AnalisisNilai` (OOP + Pandas) untuk mengelola data nilai 120 mahasiswa pada 3 program studi dan 5 mata kuliah: tambah data dengan validasi, statistik, filter, ranking, uji perbandingan prodi, 6 visualisasi + dashboard, export CSV/Excel, dan penyimpanan state. Dilengkapi 7 unit test.

Sumber Data

- **Data sintetis (fiktif)** buatan sendiri dengan `src/generate_data.py` (seed tetap = 2026, reproducible). Setiap prodi diberi "bonus" kemampuan pada mata kuliah tertentu sehingga ada pola yang dapat dianalisis. Nama dan nilai bukan data nyata.
- File: `data/raw/nilai_mahasiswa.csv`.

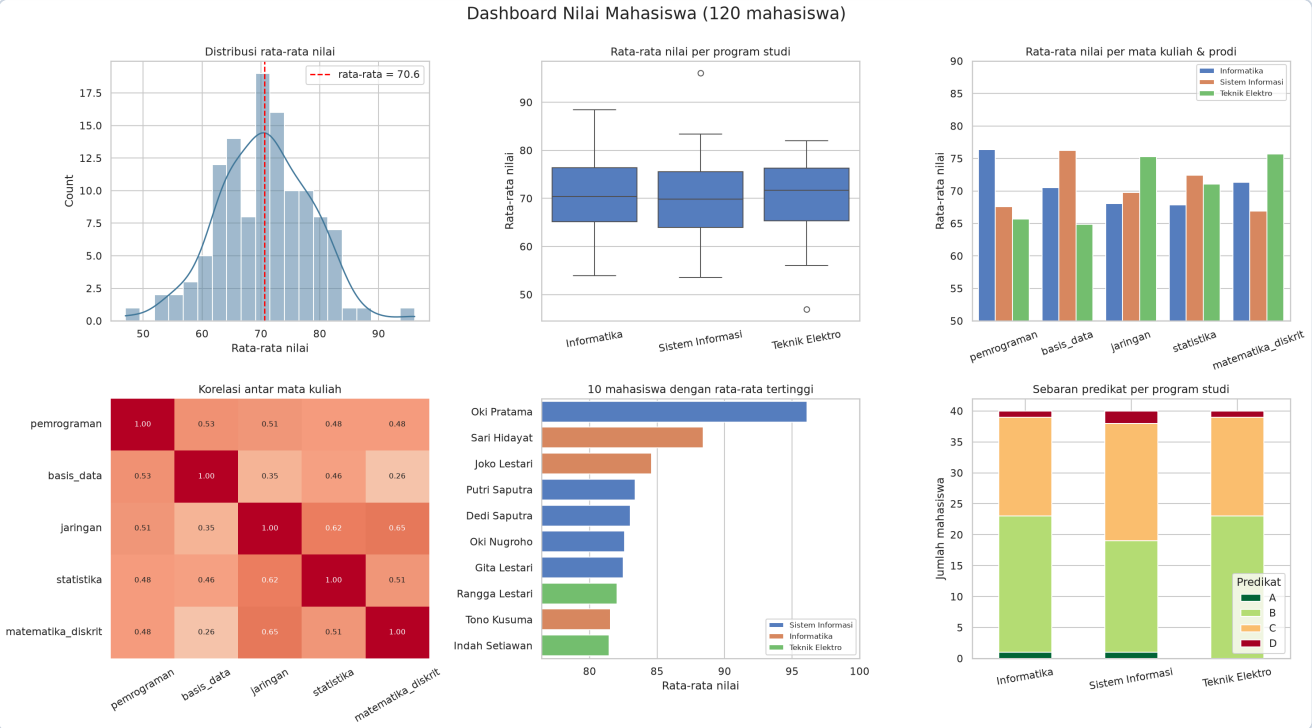
Cara Menjalankan

```
pip install -r requirements.txt
python src/generate_data.py           # (opsional) buat ulang data sintetis
python src/main.py                    # pipeline otomatis: semua fitur + output
python src/main.py --interaktif       # mode menu terminal
python -m unittest discover -s tests -v   # unit test
```

Log eksekusi tersimpan di `output/reports/`: `run_log.txt` (pipeline), `demo_interaktif_log.txt` (mode interaktif dengan input dari `data/demo_input.txt`), `test_log.txt` (unit test).

Hasil Utama / Insight

1. **Program studi dengan rata-rata tertinggi: Informatika** (70.84); selisih rata-rata antar prodi hanya 0.33 poin, sehingga perbedaan lebih terlihat pada mata kuliah daripada rata-rata keseluruhan.
2. **Setiap prodi unggul pada mata kuliah berbeda:** Informatika → pemrograman; Sistem Informasi → basis_data; Teknik Elektro → matematika_diskrit.
3. **Perbandingan Informatika vs Teknik Elektro (Welch t-test, alpha 5%):** mata kuliah dengan beda signifikan: pemrograman, basis_data, jaringan. Selisih terbesar pada pemrograman (+10.7 poin).
4. **Semua korelasi antar mata kuliah positif** (r antara 0.26 dan 0.65): terkuat jaringan–matematika_diskrit, terlemah basis_data–matematika_diskrit; mahasiswa yang kuat di satu mata kuliah cenderung kuat di lainnya (kemampuan umum).
5. **Mata kuliah paling bervariasi: pemrograman** (SD 11.7); paling seragam: jaringan (SD 9.3).
6. **Sebaran predikat:** A 1.7%, B 52.5%, C 42.5%, D 3.3%.



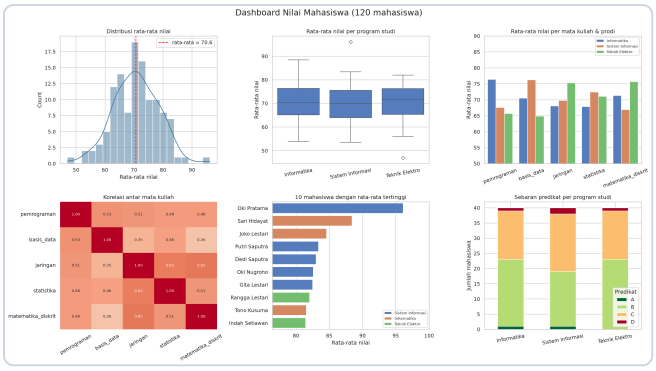
Struktur Folder

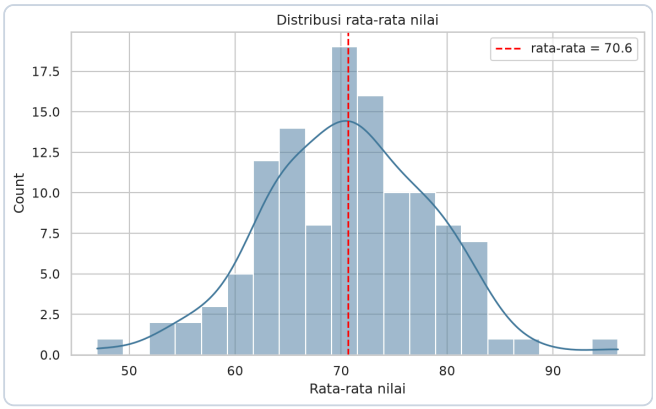
```
jalur_d_analisis_nilai_oop/
├── data/
│   ├── raw/nilai_mahasiswa.csv          # data sintetis
│   ├── processed/                       # data + rata_rata, state_aplikasi.json
│   └── demo_input.txt                   # skrip input untuk demo mode interaktif
├── src/
│   ├── analisis_nilai.py                # kelas AnalisisNilai
│   ├── generate_data.py                 # pembuat data sintetis
│   └── main.py                          # pipeline + mode interaktif
├── tests/test_analisis_nilai.py         # 7 unit test
├── output/
│   ├── figures/                         # 6 grafik + dashboard
│   └── reports/                         # CSV/Excel, perbandingan prodi, insights, log
├── requirements.txt
├── DOCUMENTATION.md
└── README.md
```

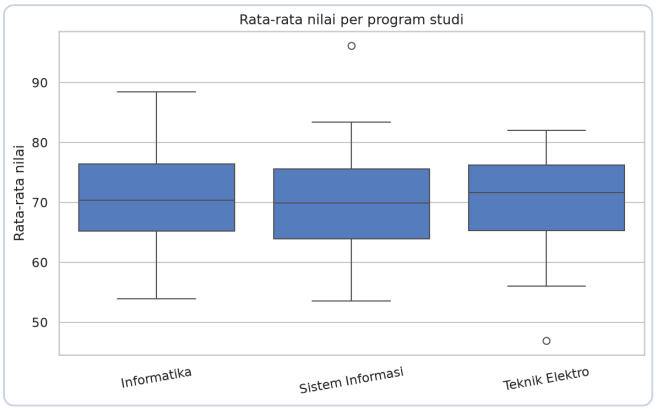
Pengembangan Selanjutnya (opsional)

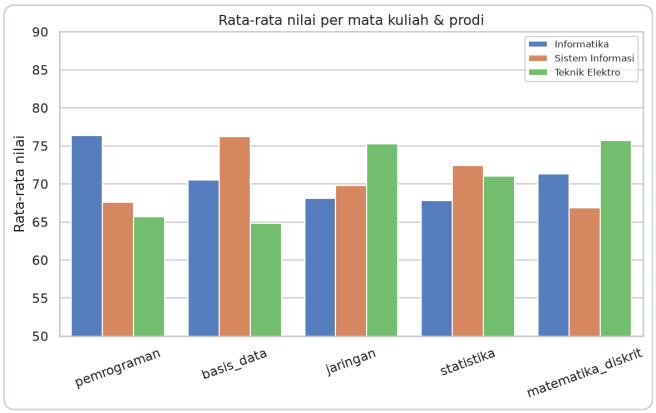
- Data per semester dan perhitungan IPK berbobot SKS.
- Antarmuka web sederhana (mis. Streamlit) di atas kelas yang sama.
- Deteksi mahasiswa berisiko (nilai turun/di bawah ambang).

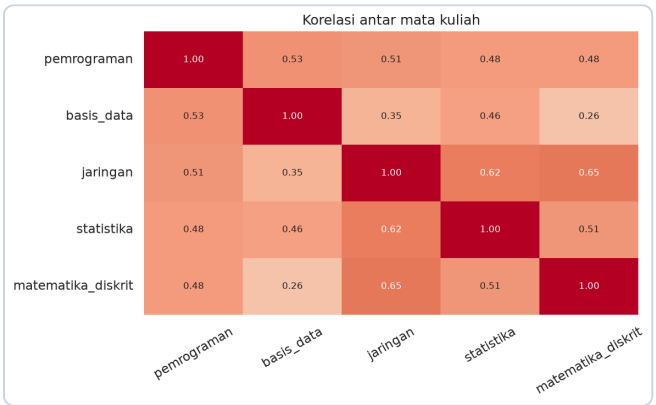
Grafik hasil

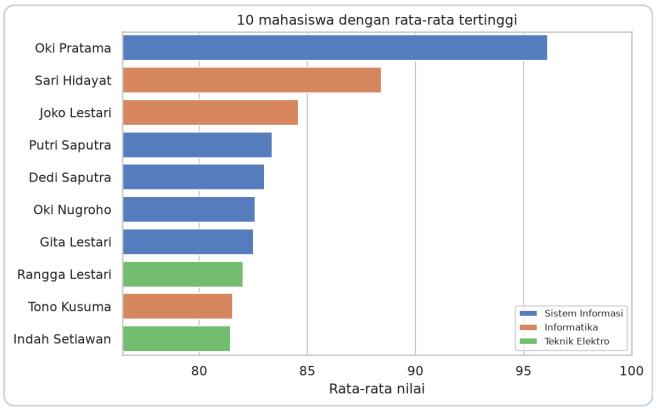


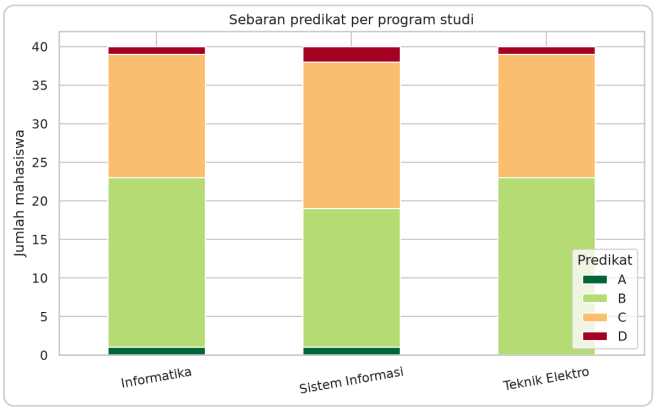












Kode sumber

`jalur_d_analisis_nilai_oop/src/analisis_nilai.py`

```python
"""Kelas AnalisisNilai — mini sistem analisis data akademik (OOP + Pandas).

Fitur: tambah data, statistik, filter, ranking, perbandingan dua prodi,
visualisasi, export CSV/Excel, serta simpan/muat state aplikasi (JSON).
"""
import json
from pathlib import Path

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from scipy import stats

MATKUL = ["pemrograman", "basis_data", "jaringan", "statistika", "matematika_diskrit"]
KOLOM_ID = ["nim", "nama", "prodi", "kelas"]
sns.set_theme(style="whitegrid", palette="muted")


class DataTidakValidError(ValueError):
    """Dilempar saat input data mahasiswa tidak memenuhi aturan validasi."""


class AnalisisNilai:
    """Mengelola data nilai mahasiswa dan menghasilkan analisis serta laporan.

    Attributes:
        df (pd.DataFrame): data nilai (kolom id + satu kolom per mata kuliah).
        matkul (list[str]): daftar mata kuliah yang dianalisis.
    """

    def __init__(self, data_path=None, matkul=None):
        self.matkul = list(matkul or MATKUL)
        if data_path:
            self.df = self._baca_csv(Path(data_path))
        else:
            self.df = pd.DataFrame(columns=KOLOM_ID + self.matkul)
        self._hitung_rata()

    # ------------------------------------------------------------------
    # Pemuatan & validasi
    # ------------------------------------------------------------------
    def _baca_csv(self, path):
        """Baca CSV dan validasi kolom serta rentang nilai (0-100)."""
        try:
            df = pd.read_csv(path, dtype={"nim": str})
        except FileNotFoundError as err:
            raise FileNotFoundError(f"File data tidak ditemukan: {path}") from err
        hilang = set(KOLOM_ID + self.matkul) - set(df.columns)
        if hilang:
            raise DataTidakValidError(f"Kolom wajib tidak ada: {sorted(hilang)}")
        if df["nim"].duplicated().any():
            raise DataTidakValidError("NIM duplikat ditemukan pada file data.")
        nilai = df[self.matkul]
        if nilai.isna().any().any() or ((nilai < 0) | (nilai > 100)).any().any():
            raise DataTidakValidError("Nilai harus terisi dan berada pada rentang 0-100.")
        return df

    def _hitung_rata(self):
        """Hitung kolom rata_rata dan predikat dari nilai mata kuliah."""
        if self.df.empty:
            self.df["rata_rata"] = pd.Series(dtype=float)
            self.df["predikat"] = pd.Series(dtype=str)
            return
        self.df["rata_rata"] = self.df[self.matkul].mean(axis=1).round(2)
        self.df["predikat"] = pd.cut(
            self.df["rata_rata"], bins=[-0.01, 54.99, 69.99, 84.99, 100],
            labels=["D", "C", "B", "A"]).astype(str)

    # ------------------------------------------------------------------
    # Tambah data
```

```
    # ------------------------------------------------------------------
    def tambah_mahasiswa(self, nama, prodi, kelas, nilai):
        """Tambah satu mahasiswa; `nilai` berupa dict {matkul: angka 0-100}.

        Mengembalikan NIM yang dibuat otomatis. Melempar DataTidakValidError
        jika nama kosong, mata kuliah tidak lengkap, atau nilai di luar 0-100.
        """
        if not str(nama).strip():
            raise DataTidakValidError("Nama tidak boleh kosong.")
        if set(nilai) != set(self.matkul):
            raise DataTidakValidError(f"Nilai harus lengkap untuk: {self.matkul}")
        for mk, angka in nilai.items():
            if not isinstance(angka, (int, float)) or not 0 <= angka <= 100:
                raise DataTidakValidError(f"Nilai {mk}={angka!r} harus angka 0-100.")
        nomor = 1 if self.df.empty else int(self.df["nim"].str[2:].astype(int).max()) + 1
        nim = f"26{nomor:04d}"
        baris = {"nim": nim, "nama": nama.strip(), "prodi": prodi.strip(),
                 "kelas": kelas.strip().upper(), **nilai}
        self.df = pd.concat([self.df, pd.DataFrame([baris])], ignore_index=True)
        self._hitung_rata()
        return nim

    # ------------------------------------------------------------------
    # Statistik, filter, ranking
    # ------------------------------------------------------------------
    def statistik(self):
        """Statistik deskriptif per mata kuliah (+ rata_rata)."""
        kolom = self.matkul + ["rata_rata"]
        return self.df[kolom].agg(["count", "mean", "median", "std", "min", "max"]).T.round(2)

    def statistik_per_prodi(self):
        """Rata-rata tiap mata kuliah per program studi."""
        return self.df.groupby("prodi")[self.matkul + ["rata_rata"]].mean().round(2)

    def filter(self, prodi=None, kelas=None, min_rata=None, predikat=None):
        """Kembalikan salinan DataFrame yang memenuhi semua kriteria yang diberikan."""
        hasil = self.df
        if prodi is not None:
            hasil = hasil[hasil["prodi"] == prodi]
        if kelas is not None:
            hasil = hasil[hasil["kelas"] == kelas.upper()]
        if min_rata is not None:
            hasil = hasil[hasil["rata_rata"] >= min_rata]
        if predikat is not None:
            hasil = hasil[hasil["predikat"] == predikat.upper()]
        return hasil.copy()

    def ranking(self, n=10, prodi=None):
        """n mahasiswa dengan rata-rata tertinggi (opsional per prodi)."""
        data = self.filter(prodi=prodi)
        top = data.sort_values(["rata_rata", "nim"], ascending=[False, True]).head(n)
        top = top[["nim", "nama", "prodi", "rata_rata", "predikat"]].reset_index(drop=True)
        top.index += 1
        top.index.name = "peringkat"
        return top

    def bandingkan_prodi(self, prodi_a, prodi_b):
        """Bandingkan rata-rata dua prodi per mata kuliah (Welch t-test + selisih)."""
        baris = []
        for mk in self.matkul + ["rata_rata"]:
            a = self.df.loc[self.df["prodi"] == prodi_a, mk]
            b = self.df.loc[self.df["prodi"] == prodi_b, mk]
            if len(a) < 2 or len(b) < 2:
                raise DataTidakValidError("Setiap prodi perlu minimal 2 mahasiswa.")
            t, p = stats.ttest_ind(a, b, equal_var=False)
            baris.append({"matkul": mk, f"rata_{prodi_a}": a.mean(),
                          f"rata_{prodi_b}": b.mean(), "selisih": a.mean() - b.mean(),
                          "t": t, "p_value": p, "signifikan_5%": p < 0.05})
        hasil = pd.DataFrame(baris).set_index("matkul")
        kolom_bulat = [c for c in hasil.columns if c not in ("p_value", "signifikan_5%")]
        hasil[kolom_bulat] = hasil[kolom_bulat].round(3)
        return hasil

    # ------------------------------------------------------------------
    # Visualisasi
```

```
    # ------------------------------------------------------------------
    def visualisasi(self, folder):
        """Simpan 6 grafik + dashboard 2x3 ke `folder`; kembalikan daftar nama file."""
        folder = Path(folder)
        folder.mkdir(parents=True, exist_ok=True)
        panel = [
            ("01_distribusi_rata_rata", self._p_hist),
            ("02_boxplot_per_prodi", self._p_box),
            ("03_rata_matkul_per_prodi", self._p_bar),
            ("04_korelasi_matkul", self._p_corr),
            ("05_top10", self._p_top),
            ("06_predikat_per_prodi", self._p_predikat),
        ]
        for nama, fungsi in panel:
            fig, ax = plt.subplots(figsize=(8, 5))
            fungsi(ax)
            fig.tight_layout()
            fig.savefig(folder / f"{nama}.png", dpi=150, bbox_inches="tight")
            plt.close(fig)
        fig, axes = plt.subplots(2, 3, figsize=(20, 11))
        for ax, (_, fungsi) in zip(axes.ravel(), panel):
            fungsi(ax)
        fig.suptitle(f"Dashboard Nilai Mahasiswa ({len(self.df)} mahasiswa)", fontsize=18, y=1.0)
        fig.tight_layout()
        fig.savefig(folder / "00_dashboard.png", dpi=150, bbox_inches="tight")
        plt.close(fig)
        return ["00_dashboard.png"] + [f"{n}.png" for n, _ in panel]

    def _p_hist(self, ax):
        sns.histplot(self.df["rata_rata"], bins=20, kde=True, ax=ax, color="#457b9d")
        ax.axvline(self.df["rata_rata"].mean(), color="red", ls="--",
                   label=f"rata-rata = {self.df['rata_rata'].mean():.1f}")
        ax.set_title("Distribusi rata-rata nilai"); ax.set_xlabel("Rata-rata nilai"); ax.legend()

    def _p_box(self, ax):
        sns.boxplot(data=self.df, x="prodi", y="rata_rata", ax=ax)
        ax.set_title("Rata-rata nilai per program studi")
        ax.set_xlabel(""); ax.set_ylabel("Rata-rata nilai"); ax.tick_params(axis="x", rotation=10)

    def _p_bar(self, ax):
        d = self.df.melt(id_vars="prodi", value_vars=self.matkul, var_name="matkul", value_name="nilai")
        sns.barplot(data=d, x="matkul", y="nilai", hue="prodi", errorbar=None, ax=ax)
        ax.set_title("Rata-rata nilai per mata kuliah & prodi")
        ax.set_xlabel(""); ax.set_ylabel("Rata-rata nilai"); ax.set_ylim(50, 90)
        ax.tick_params(axis="x", rotation=20); ax.legend(title="", fontsize=8)

    def _p_corr(self, ax):
        sns.heatmap(self.df[self.matkul].corr(), annot=True, fmt=".2f", cmap="coolwarm",
                    center=0, ax=ax, cbar=False, annot_kws={"size": 8})
        ax.set_title("Korelasi antar mata kuliah"); ax.tick_params(axis="x", rotation=30)

    def _p_top(self, ax):
        top = self.ranking(10).reset_index()
        sns.barplot(data=top, y="nama", x="rata_rata", hue="prodi", dodge=False, ax=ax)
        ax.set_title("10 mahasiswa dengan rata-rata tertinggi")
        ax.set_xlabel("Rata-rata nilai"); ax.set_ylabel("")
        ax.set_xlim(top["rata_rata"].min() - 5, 100); ax.legend(title="", fontsize=8)

    def _p_predikat(self, ax):
        tab = pd.crosstab(self.df["prodi"], self.df["predikat"]).reindex(
            columns=["A", "B", "C", "D"], fill_value=0)
        tab.plot(kind="bar", stacked=True, ax=ax, colormap="RdYlGn_r")
        ax.set_title("Sebaran predikat per program studi")
        ax.set_xlabel(""); ax.set_ylabel("Jumlah mahasiswa"); ax.tick_params(axis="x", rotation=10)
        ax.legend(title="Predikat")

    # ------------------------------------------------------------------
    # Export & state
    # ------------------------------------------------------------------
    def export(self, path_csv, path_xlsx=None):
        """Export data ke CSV; jika path_xlsx diberikan, buat laporan multi-sheet."""
        path_csv = Path(path_csv)
        path_csv.parent.mkdir(parents=True, exist_ok=True)
        self.df.to_csv(path_csv, index=False)
        if path_xlsx:
```

```
            with pd.ExcelWriter(path_xlsx) as xl:
                self.df.to_excel(xl, sheet_name="Data", index=False)
                self.statistik().to_excel(xl, sheet_name="Statistik")
                self.statistik_per_prodi().to_excel(xl, sheet_name="Per prodi")
                self.ranking(20).to_excel(xl, sheet_name="Top 20")
                self.df["predikat"].value_counts().rename("jumlah").to_frame().to_excel(
                    xl, sheet_name="Predikat")

    def simpan_state(self, path):
        """Simpan seluruh state (data + daftar matkul) ke file JSON."""
        path = Path(path)
        path.parent.mkdir(parents=True, exist_ok=True)
        state = {"matkul": self.matkul, "data": self.df.drop(columns=["rata_rata", "predikat"])
                 .to_dict(orient="records")}
        path.write_text(json.dumps(state, ensure_ascii=False, indent=1), encoding="utf-8")

    @classmethod
    def muat_state(cls, path):
        """Muat state dari JSON hasil simpan_state() dan kembalikan objek baru."""
        state = json.loads(Path(path).read_text(encoding="utf-8"))
        obj = cls(matkul=state["matkul"])
        obj.df = pd.DataFrame(state["data"], dtype=object)
        obj.df["nim"] = obj.df["nim"].astype(str)
        for mk in obj.matkul:
            obj.df[mk] = obj.df[mk].astype(float)
        obj._hitung_rata()
        return obj

    def __len__(self):
        return len(self.df)

    def __repr__(self):
        return f"AnalisisNilai({len(self)} mahasiswa, {len(self.matkul)} mata kuliah)"
```

jalur_d_analisis_nilai_oop/src/main.py

```
"""Capstone Jalur D — aplikasi analisis nilai akademik berbasis OOP.

Mode:
    python src/main.py              # pipeline otomatis (default)
    python src/main.py --interaktif # menu terminal (input dari keyboard)
"""
import argparse
import sys
from pathlib import Path

import pandas as pd

from analisis_nilai import MATKUL, AnalisisNilai, DataTidakValidError
from generate_data import buat_data

BASE_DIR = Path(__file__).resolve().parent.parent
RAW_CSV = BASE_DIR / "data" / "raw" / "nilai_mahasiswa.csv"
OUT_DIR = BASE_DIR / "output"
STATE = BASE_DIR / "data" / "processed" / "state_aplikasi.json"
pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)


def siapkan_data():
    """Buat data sintetis jika file mentah belum ada."""
    if not RAW_CSV.exists():
        RAW_CSV.parent.mkdir(parents=True, exist_ok=True)
        buat_data().to_csv(RAW_CSV, index=False)
        print(f"Data sintetis dibuat: {RAW_CSV.name}")


def pipeline():
    """Jalankan seluruh fitur secara otomatis dan simpan semua output."""
    siapkan_data()
    app = AnalisisNilai(RAW_CSV)
    print(app, "\n")
    print("=" * 60 + "\n1. STATISTIK\n" + "=" * 60)
    print(app.statistik().to_string())
```

```
    print("\nRata-rata per prodi:")
    print(app.statistik_per_prodi().to_string())
    print("\nSebaran predikat:")
    print(app.df["predikat"].value_counts().reindex(["A", "B", "C", "D"], fill_value=0).to_string())

    print("\n" + "=" * 60 + "\n2. FILTER\n" + "=" * 60)
    f1 = app.filter(prodi="Teknik Elektro", min_rata=80)
    print(f"Teknik Elektro dengan rata-rata >= 80: {len(f1)} mahasiswa")
    f2 = app.filter(predikat="A")
    print(f"Predikat A di seluruh prodi: {len(f2)} mahasiswa")
    print(f"Kelas B Sistem Informasi: {len(app.filter(prodi='Sistem Informasi', kelas='B'))} mahasiswa")

    print("\n" + "=" * 60 + "\n3. RANKING\n" + "=" * 60)
    print("Top 10 keseluruhan:")
    print(app.ranking(10).to_string())
    print("\nTop 3 per prodi:")
    for prodi in sorted(app.df["prodi"].unique()):
        print(f"\n{prodi}")
        print(app.ranking(3, prodi=prodi).to_string())

    print("\n" + "=" * 60 + "\n4. BANDINGKAN PRODI (Welch t-test)\n" + "=" * 60)
    banding = app.bandingkan_prodi("Informatika", "Teknik Elektro")
    print(banding.to_string(float_format=lambda x: f"{x:.4g}"))
    banding.to_csv(OUT_DIR / "reports" / "perbandingan_if_vs_te.csv")

    print("\n" + "=" * 60 + "\n5. VISUALISASI & EXPORT\n" + "=" * 60)
    file_gambar = app.visualisasi(OUT_DIR / "figures")
    print("Grafik:", ", ".join(file_gambar))
    app.export(OUT_DIR / "reports" / "laporan_nilai.csv", OUT_DIR / "reports" / "laporan_nilai.xlsx")
    app.df.to_csv(BASE_DIR / "data" / "processed" / "nilai_dengan_rata_rata.csv", index=False)
    print("Laporan CSV & Excel (5 sheet) disimpan di output/reports/")

    tulis_insight(app, banding)

    print("\n" + "=" * 60 + "\n7. TAMBAH DATA (dengan validasi)\n" + "=" * 60)
    nim = app.tambah_mahasiswa("Rina Kartika", "Informatika", "a",
                               dict(zip(MATKUL, [90, 85, 80, 88, 92])))
    print("Mahasiswa baru ditambahkan dengan NIM", nim)
    try:
        app.tambah_mahasiswa("Salah Input", "Informatika", "A",
                             dict(zip(MATKUL, [90, 85, 80, 88, 120])))
    except DataTidakValidError as err:
        print("Input ditolak:", err)

    print("\n" + "=" * 60 + "\n8. SIMPAN & MUAT STATE\n" + "=" * 60)
    app.simpan_state(STATE)
    app2 = AnalisisNilai.muat_state(STATE)
    sama = app2.df[["nim", "rata_rata"]].equals(app.df[["nim", "rata_rata"]])
    print(f"State disimpan ke {STATE.name}; dimuat ulang: {app2}; data identik: {sama}")



def tulis_insight(app, banding):
    """Hitung insight dari data dan simpan ke output/reports/insights.md."""
    df = app.df
    pp = app.statistik_per_prodi()
    terbaik_prodi = pp["rata_rata"].idxmax()
    korelasi = df[MATKUL].corr()
    pasangan = korelasi.where(~(korelasi.values == 1)).stack()
    kuat = pasangan.idxmax()
    lemah = pasangan.idxmin()
    sd = df[MATKUL].std()
    pred = df["predikat"].value_counts(normalize=True)
    unggul = {prodi: pp.loc[prodi, MATKUL].idxmax() for prodi in pp.index}
    sig = banding[banding["signifikan_5%"]].index.tolist()
    baris = [
        f"1. **Program studi dengan rata-rata tertinggi: {terbaik_prodi}** "
        f"({pp.loc[terbaik_prodi, 'rata_rata']:.2f}); selisih rata-rata antar prodi hanya "
        f"{pp['rata_rata'].max() - pp['rata_rata'].min():.2f} poin, sehingga perbedaan lebih terlihat "
        f"pada mata kuliah daripada rata-rata keseluruhan.",
        "2. **Setiap prodi unggul pada mata kuliah berbeda:** "
        + "; ".join(f"{p} → {m}" for p, m in unggul.items()) + ".",
        f"3. **Perbandingan Informatika vs Teknik Elektro (Welch t-test, alpha 5%):** "
        f"mata kuliah dengan beda signifikan: {', '.join(sig) if sig else 'tidak ada'}. "
        f"Selisih terbesar pada {banding['selisih'].abs().drop('rata_rata').idxmax()} "
```

```
        f"({banding.loc[banding['selisih'].abs().drop('rata_rata').idxmax(), 'selisih']:+.1f} poin).",
        f"4. **Semua korelasi antar mata kuliah positif** (r antara {pasangan[lemah]:.2f} dan "
        f"{pasangan[kuat]:.2f}): terkuat {kuat[0]}–{kuat[1]}, terlemah {lemah[0]}–{lemah[1]}; "
        f"mahasiswa yang kuat di satu mata kuliah cenderung kuat di lainnya (kemampuan umum).",
        f"5. **Mata kuliah paling bervariasi: {sd.idxmax()}** (SD {sd.max():.1f}); "
        f"paling seragam: {sd.idxmin()} (SD {sd.min():.1f}).",
        f"6. **Sebaran predikat:** A {pred.get('A', 0):.1%}, B {pred.get('B', 0):.1%}, "
        f"C {pred.get('C', 0):.1%}, D {pred.get('D', 0):.1%}.",
    ]
    teks = "# Insight — Analisis Nilai Mahasiswa\n\n" + "\n\n".join(baris) + "\n"
    (OUT_DIR / "reports").mkdir(parents=True, exist_ok=True)
    (OUT_DIR / "reports" / "insights.md").write_text(teks, encoding="utf-8")
    print("\n" + "=" * 60 + "\n6. INSIGHT\n" + "=" * 60)
    print("\n\n".join(baris))


def tanya(prompt):
    """input() yang ikut menampilkan jawaban bila stdin bukan terminal (mis. demo/log)."""
    jawaban = input(prompt)
    if not sys.stdin.isatty():
        print(jawaban)
    return jawaban


def interaktif():
    """Menu terminal sederhana untuk mencoba fitur secara interaktif."""
    siapkan_data()
    app = AnalisisNilai.muat_state(STATE) if STATE.exists() else AnalisisNilai(RAW_CSV)
    menu = ("\n=== ANALISIS NILAI ===\n1. Statistik\n2. Ranking top 5\n3. Filter by prodi\n"
            "4. Tambah mahasiswa\n5. Simpan & keluar")
    while True:
        print(menu)
        try:
            pilih = tanya("Pilih (1-5): ").strip()
        except EOFError:
            print("\nInput habis, keluar.")
            break
        if pilih == "1":
            print(app.statistik_per_prodi().to_string())
        elif pilih == "2":
            print(app.ranking(5).to_string())
        elif pilih == "3":
            prodi = tanya("Nama prodi: ").strip()
            hasil = app.filter(prodi=prodi)
            print(f"{len(hasil)} mahasiswa ditemukan.")
            if len(hasil):
                print(hasil[["nim", "nama", "rata_rata", "predikat"]].head(5).to_string(index=False))
        elif pilih == "4":
            try:
                nama = tanya("Nama: ")
                prodi = tanya("Prodi: ")
                kelas = tanya("Kelas (A/B): ")
                nilai = {mk: float(tanya(f"  nilai {mk}: ")) for mk in MATKUL}
                print("Ditambahkan dengan NIM", app.tambah_mahasiswa(nama, prodi, kelas, nilai))
            except (ValueError, DataTidakValidError) as err:
                print("Gagal:", err)
        elif pilih == "5":
            app.simpan_state(STATE)
            print("State disimpan. Sampai jumpa!")
            break
        else:
            print("Pilihan tidak dikenal.")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__.splitlines()[0])
    parser.add_argument("--interaktif", action="store_true", help="jalankan mode menu terminal")
    args = parser.parse_args()
    interaktif() if args.interaktif else pipeline()
```

jalur_d_analisis_nilai_oop/src/generate_data.py

```
"""Membuat data nilai mahasiswa buatan sendiri (sintetis, reproducible).
```

```
Data ini FIKTIF — dibuat dengan seed tetap agar hasil analisis dapat diulang.
Jalankan: python src/generate_data.py
"""
from pathlib import Path

import numpy as np
import pandas as pd

BASE_DIR = Path(__file__).resolve().parent.parent
OUT = BASE_DIR / "data" / "raw" / "nilai_mahasiswa.csv"

MATKUL = ["pemrograman", "basis_data", "jaringan", "statistika", "matematika_diskrit"]
PRODI = {
    # bonus rata-rata nilai per mata kuliah (urutan = MATKUL)
    "Informatika": [6, 3, 0, 0, 2],
    "Sistem Informasi": [0, 6, 0, 3, -3],
    "Teknik Elektro": [-4, -3, 6, 2, 5],
}
DEPAN = ["Ani", "Budi", "Citra", "Dedi", "Eka", "Fajar", "Gita", "Hadi", "Indah", "Joko",
         "Kiki", "Lina", "Made", "Nina", "Oki", "Putri", "Rangga", "Sari", "Tono", "Umar"]
BELAKANG = ["Pratama", "Wijaya", "Santoso", "Lestari", "Hidayat", "Saputra", "Kusuma",
            "Nugroho", "Rahayu", "Setiawan"]


def buat_data(n_per_prodi=40, seed=2026):
    """Hasilkan DataFrame nilai: kemampuan dasar + bonus prodi + noise."""
    rng = np.random.default_rng(seed)
    baris = []
    nomor = 1
    for prodi, bonus in PRODI.items():
        for _ in range(n_per_prodi):
            dasar = rng.normal(70, 8)  # kemampuan umum mahasiswa
            nilai = np.clip(dasar + np.array(bonus) + rng.normal(0, 6, len(MATKUL)), 0, 100)
            baris.append({
                "nim": f"26{nomor:04d}",
                "nama": f"{rng.choice(DEPAN)} {rng.choice(BELAKANG)}",
                "prodi": prodi,
                "kelas": rng.choice(["A", "B"]),
                **dict(zip(MATKUL, nilai.round(1))),
            })
            nomor += 1
    return pd.DataFrame(baris)


if __name__ == "__main__":
    OUT.parent.mkdir(parents=True, exist_ok=True)
    df = buat_data()
    df.to_csv(OUT, index=False)
    print(f"{len(df)} baris disimpan ke {OUT}")
```

jalur_d_analisis_nilai_oop/tests/test_analisis_nilai.py

```
"""Unit test sederhana untuk AnalisisNilai (jalankan: python -m unittest discover -s tests)."""
import sys
import tempfile
import unittest
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "src"))

from analisis_nilai import MATKUL, AnalisisNilai, DataTidakValidError  # noqa: E402
from generate_data import buat_data  # noqa: E402


def app_contoh():
    tmp = Path(tempfile.mkdtemp()) / "data.csv"
    buat_data(n_per_prodi=10, seed=1).to_csv(tmp, index=False)
    return AnalisisNilai(tmp)


class TestAnalisisNilai(unittest.TestCase):
    def test_jumlah_dan_rata_rata(self):
        app = app_contoh()
        self.assertEqual(len(app), 30)
```

```
        baris = app.df.iloc[0]
        self.assertAlmostEqual(baris["rata_rata"], round(baris[MATKUL].mean(), 2))

    def test_tambah_valid_dan_nim_berurutan(self):
        app = app_contoh()
        nim = app.tambah_mahasiswa("Tes", "Informatika", "a", dict.fromkeys(MATKUL, 80))
        self.assertEqual(nim, "260031")
        self.assertEqual(len(app), 31)
        self.assertEqual(app.df.iloc[-1]["kelas"], "A")

    def test_tambah_tidak_valid(self):
        app = app_contoh()
        with self.assertRaises(DataTidakValidError):
            app.tambah_mahasiswa("Tes", "Informatika", "A", dict.fromkeys(MATKUL, 101))
        with self.assertRaises(DataTidakValidError):
            app.tambah_mahasiswa("  ", "Informatika", "A", dict.fromkeys(MATKUL, 80))
        with self.assertRaises(DataTidakValidError):
            app.tambah_mahasiswa("Tes", "Informatika", "A", {"pemrograman": 80})
        self.assertEqual(len(app), 30)

    def test_filter_dan_ranking(self):
        app = app_contoh()
        hasil = app.filter(prodi="Informatika")
        self.assertTrue((hasil["prodi"] == "Informatika").all())
        top = app.ranking(5)
        self.assertEqual(len(top), 5)
        self.assertTrue(top["rata_rata"].is_monotonic_decreasing)

    def test_state_bolak_balik(self):
        app = app_contoh()
        path = Path(tempfile.mkdtemp()) / "state.json"
        app.simpan_state(path)
        baru = AnalisisNilai.muat_state(path)
        self.assertEqual(len(baru), len(app))
        self.assertTrue((baru.df["rata_rata"].values == app.df["rata_rata"].values).all())

    def test_file_tidak_ada(self):
        with self.assertRaises(FileNotFoundError):
            AnalisisNilai("tidak_ada.csv")

    def test_bandingkan_prodi(self):
        app = app_contoh()
        hasil = app.bandingkan_prodi("Informatika", "Teknik Elektro")
        self.assertIn("p_value", hasil.columns)
        self.assertEqual(len(hasil), len(MATKUL) + 1)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```